

What is a nitrate measurement worth? Fisher information and goal-oriented monitoring network design in alluvial aquifers

Dulian Zeqiraj

Department of Energy Resources, Faculty of Geology and Mining, Polytechnic University of Tirana, Rruga e Elbasanit, Tirana 1001, Albania

Correspondence: dulian.zeqiraj@fgjm.edu.al

Highlights

- On a 24-well nitrate snapshot no interpolator beats the network mean out of sample
- 24 nitrate wells constrain 0.0753 of one direction in a 150-mode porosity field
- For porosity, one optimally sited nitrate sample is worth 0.09 of a pumping test
- Weighting the parameter space leaves D-optimal sites unchanged, which is proved
- Goal orientation cuts the targeted variance 10.0 times further than D-optimality

Abstract

Monitoring networks are extended where drilling is convenient and the value of the resulting data is asserted rather than computed. This paper computes it for the Fushë-Kuqe alluvial aquifer in northwestern Albania, a network of 180 points of which 43 carry a pumping-test hydraulic conductivity and 31 a nitrate concentration. Four results follow. First, the nitrate field is not predictable at this density: inverse-distance weighting at three exponents, ordinary kriging at five correlation lengths and five linear models all fail to beat the network mean under leave-one-out cross-validation, the best reaching $R^2 = +0.16$; a transient transport model with a loading field estimated from the data reaches 0.48 in sample and -0.04 out of sample. Second, this is a property of the physics, not a modelling failure. Under a uniform regional Darcy flux, effective porosity cancels between the seepage velocity and the mechanical dispersion coefficient, so a steady concentration field is blind to it; only the transient carries information, and here the 24 nitrate wells constrain the equivalent of 0.0753 of one direction in a 150-dimensional parameter space, supplying 0.038 nats. Third, a corollary drawn in recent work on this aquifer, that one concentration observation is worth about three pumping tests, comes out inverted when it is computed from the prior and the forward operator rather than argued from kernel extent: the factor is 0.09, stable at 0.02 to 0.33 across the measurement errors tested. Fourth, where design does pay, goal orientation is what makes it pay. Weighting the parameter space by vulnerability leaves the classical D-optimal site unchanged for any weighting, which is proved in two lines; targeting instead the variance of the vulnerability-weighted contaminant arrival time reduces that variance by 30.7 per cent against 3.1 per cent for D-optimality, with sites a median 7.9 km apart. The same divergence is reproduced on 44 public USGS wells of the Mississippi River Valley alluvial aquifer. Code and data are released in full.

Keywords: optimal experimental design; Fisher information; monitoring network; effective porosity; DRASTIC vulnerability; alluvial aquifer; cross-validation

1. Introduction

1.1 The value of a monitoring well is rarely computed

Regional groundwater monitoring networks grow by accretion. Wells are added where access is easy, where a utility already owns land, or where an earlier campaign left a borehole, and the information content of the resulting network is discussed qualitatively if at all. The theory needed to do better is old. Lindley (1956) supplied the measure, Fedorov (1972) and Pukelsheim (2006) the design criteria, Chaloner and Verdinelli (1995) the Bayesian synthesis. Hydrogeology has had task-driven and geostatistical versions for two decades (Herrera and Pinder, 2005; Nowak et al., 2010; Leube et al., 2012), and the infinite-dimensional Bayesian treatment is now well developed (Alexanderian, 2021), including goal-oriented variants (Attia et al., 2018). What remains uncommon is a computation on a real network that answers, in units a manager recognises, what the data already in hand are worth and where the next well should go.

This paper is that computation for one aquifer. Its first result is negative, and the negative result is what makes the rest worth doing.

1.2 A claim worth checking

Recent work on this aquifer coupled DRASTIC vulnerability assessment to a Biased Global Random Walk transport model and reported nitrate prediction at $R^2 = 0.868$ against 564 space-time measurements spanning 300 days (Zeqiraj et al., 2026); a companion study developed a correlation-aware fusion of two porosity estimation pathways (Zeqiraj and Beqiraj, 2026).

A note on what can and cannot be compared. The field record available for the present study is a single snapshot: 31 nitrate values with no time dimension, ranging 1.38 to 24.74 mg L⁻¹ with a mean of 6.96. That is not the evidence base behind the 564 space-time measurements, and this paper makes no claim about the validity of a number computed on data it does not hold. What it does claim concerns the snapshot, and concerns a corollary drawn from that earlier work which is a statement about physics rather than about any particular dataset: because a concentration measurement integrates along a transport path while a pumping test samples a radius of influence, one well-placed concentration observation was argued to be worth roughly three additional pumping tests.

That corollary is testable from the prior and the forward operator alone, without any concentration data at all, and it is what Section 6.4 computes. It comes out inverted.

1.3 Contributions

1. A cross-validated assessment of what a 24-well nitrate snapshot predicts, benchmarked against interpolators that assume no physics at all. This is the control that decides whether a poor fit is the model's fault or the data's, and it is the step whose absence makes an in-sample R^2 misleading.
2. An identifiability statement with its magnitude. Under a uniform regional flux the steady concentration field does not depend on effective porosity, because porosity cancels between the seepage velocity and the mechanical dispersion coefficient. Identifiability lives in the transient, and how much of it there is on these data is computed rather than asserted.

3. A negative result about weighted design criteria. Reweighting the parameter space by a vulnerability field, by any weighting whatever, leaves the D-optimal site unchanged. Criteria proposed in the form of a vulnerability-weighted D-optimality therefore cannot do what they are meant to do. The proof is two lines.
4. A criterion that does work in its place: goal orientation on the vulnerability-weighted contaminant arrival time, with a closed-form rank-one gain, and two diagnostics computed on the real sensitivity kernels, an information horizon and a redundancy measure.
5. A computed information-equivalence factor between a concentration observation and a pumping test, with a sensitivity analysis over every assumption it depends on.
6. A reported pathology of greedy design on smooth information fields, and the constraint that removes it.
7. An independent replication on a public USGS dataset from another continent.

Everything is released as one reproducible package (Section 10).

2. Study area and data

2.1 The aquifer

The Fushë-Kuqe aquifer occupies the coastal plain of the Mat river in northwestern Albania. The primary hydrogeological description (Cenameri and Beqiraj, 2016) gives a confined multilayer alluvial system recharged by bed infiltration from the Mat in the north and the Droja in the south, discharging to the Adriatic along its western margin, with regional groundwater flow from northeast to southwest and a seawater wedge advancing in the opposite sense. The gravel and sand framework thickens from 5 to 10 m in the east to 180 to 200 m in the west beneath a silt and clay cover of 30 to 40 m, reaching about 270 m of alluvium in the Mat delta. Roughly 15 per cent of the upper aquifer carries a measurable seawater fraction, up to about 5.5 per cent near the coast.

Two features of that description matter here. The flow direction is a single regional azimuth, northeast to southwest, not the rotating field used in some earlier treatments of the same system. And the aquifer is confined and layered, so the depth-averaged two-dimensional representation used below is an idealisation that has to be declared rather than assumed away.

2.2 What the field workbook contains

One workbook, compiled from Albanian Geological Survey and IGJEUM records, holds 180 monitoring points with Gauss-Krüger coordinates (Krasovsky 1940, zone 4) and a DRASTIC index for every one; 131 of them fall inside a 3411-vertex digitised boundary enclosing 349.1 km². Of the 180 points, 43 carry a pumping-test hydraulic conductivity together with depth to water, a lithological description, a soil class and a vadose-zone class, and 31 carry a nitrate concentration. Every well with nitrate also has a conductivity. Inside the boundary there are 32 wells with conductivity and 24 with nitrate.

Figure 1 maps the system and Table 1 gives the inventory. Conductivity runs from 50 to 200 m d⁻¹, mean 130.7, standard deviation 59.7. Nitrate runs from 1.38 to 24.74 mg L⁻¹, mean 6.96, standard deviation 6.23, below the 50 mg L⁻¹ European limit everywhere but well above a natural background. DRASTIC indices run from 87 to 189, mean 136.7.

Table 1. What the field workbook holds. Every entry is read from the file by `fk_load_data.m` and printed at the start of every run.

Quantity	Value
Monitoring points with a DRASTIC index	180
Of which inside the digitised boundary	131
With a pumping-test hydraulic conductivity	43
With a nitrate concentration	31
Inside the boundary, with conductivity	32
Inside the boundary, with nitrate	24
Aquifer boundary vertices	3411
Digitised area (km ²)	349.1
Hydraulic conductivity (m d ⁻¹)	50 to 200, mean 130.7, sd 59.7
Nitrate (mg L ⁻¹)	1.38 to 24.74, mean 6.96, sd 6.23
DRASTIC index	87 to 189, mean 136.7, sd 23.5
Lithological classes of the K wells (coarse/medium/fine)	13 / 22 / 8
ANOVA of K on the lithological label	$\eta^2 = 0.040$, $F(2,40) = 0.83$, $p = 0.44$
Exponential variogram of $\log_{10}K$	nugget 0.0000, partial sill 0.0590, range 2057 m

2.3 What it does not contain, and one thing it contradicts

Three absences shape the method. There is no laboratory determination of effective porosity anywhere in the dataset, which is why the porosity prior is imported rather than derived (Section 4.1). There is no digital elevation model, and the column recording water level is a depth below ground rather than a head, so a hydraulic gradient cannot be obtained from the workbook and has to be derived from a published mean seepage velocity. And there is no time series of nitrate: every concentration is a single sample.

One contradiction governs a modelling choice and is therefore reported rather than smoothed over. The lithological descriptor attached to the 43 conductivity wells sorts them into coarse, medium and fine gravel with 13, 22 and 8 members. Those three groups are statistically indistinguishable in conductivity: one-way analysis of variance gives $\eta^2 = 0.040$, $F(2,40) = 0.83$, $p = 0.44$. Fine gravel includes wells at 200 m d⁻¹ and coarse gravel a well at 50 m d⁻¹. Facies-keyed parameter assignment, which published treatments of this aquifer use, is therefore not supported by this workbook, and the prior of Section 4.1 is keyed instead on the measurement that does discriminate.

2.4 An independent public dataset

For replication the study uses 44 wells of the Mississippi Alluvial Plain with hydraulic conductivity and transmissivity from slug tests (Pugh, 2023), to which DRASTIC ratings have been attached. Conductivity there spans 0.91 to 122.2 m d⁻¹, mean 29.5, an order of magnitude

below Fushë-Kuqe, and DRASTIC indices 115 to 187. Every DRASTIC index is recomputed in the code from its seven ratings and the standard weights, and the run aborts if any fails to reproduce; 0 failed.

3. Is the nitrate field predictable at all?

Before a transport model is judged on how well it reproduces measured nitrate, the same question has to be put to methods that assume no physics. If simple interpolation cannot predict a withheld well, no transport model can be expected to, and a good in-sample fit then measures only the number of parameters.

Leave-one-out cross-validation was run on the 24 nitrate wells inside the boundary for inverse-distance weighting at three exponents, ordinary kriging with an exponential variogram at five correlation lengths from 1 to 16 km, and five linear regressions on the available covariates. The criterion is the coefficient of determination against the honest baseline, the network mean.

Every interpolator is worse than the mean (Table 2, Figure 2a). Kriging gives R^2 between -0.27 and -0.11 across the whole range of correlation lengths, inverse-distance weighting between -0.74 and -0.11, and the DRASTIC index alone -0.19. The only method that beats the mean is a plain linear trend in easting and northing, at +0.16, and adding DRASTIC to that trend makes it worse.

Table 2. Leave-one-out cross-validation on the 24 nitrate wells inside the aquifer boundary. The baseline is the network mean, whose R^2 is zero by construction; a negative value means the method is worse than quoting the mean.

Method	Leave-one-out R^2
Inverse distance, power 1	-0.110
Inverse distance, power 2	-0.612
Inverse distance, power 3	-0.744
Ordinary kriging, range 1 km	-0.209
Ordinary kriging, range 2 km	-0.275
Ordinary kriging, range 4 km	-0.271
Ordinary kriging, range 8 km	-0.199
Ordinary kriging, range 16 km	-0.108
Linear regression on X,Y	+0.159
Linear regression on X,Y,DRASTIC	+0.041
Linear regression on X,Y,DRASTIC,K,depth	-0.004
Linear regression on DRASTIC	-0.188
Linear regression on depth to water	-0.039
Transient transport model	-0.037
Network mean (baseline)	0.000

The reason is visible in the correlations. Nitrate correlates with northing at -0.55 and with the along-flow coordinate at +0.58, but with the DRASTIC index at only +0.12 and with conductivity at -0.24. The field is organised by where nitrogen enters, and at a mean well spacing of kilometres the pattern of entry is not resolved.

This is the fact the rest of the paper rests on. It says nothing about a model fitted to a longer record; it says that on the snapshot in hand, an in-sample fit is the only kind of fit available, and a transport model evaluated only in sample on data of this density has not been tested.

4. Bayesian framework

4.1 The porosity prior is imported, not re-derived

The Dual-Pathway fusion of lithological and hydraulic porosity estimates for this aquifer is already published together with its correlation-aware variance (Zeqiraj et al., 2026; Zeqiraj and Beqiraj, 2026). This paper consumes that result rather than repeating it. Two numbers are taken from it: a network mean effective porosity of 0.282 and a network range of 0.22 to 0.35.

Because the lithological label does not discriminate conductivity in this dataset, the spatial pattern of the prior mean is keyed on $\log_{10} K$,

$$n_e^{prior}(x_i) = \bar{n}_e + s_n \frac{\log_{10} K_i - \overline{\log_{10} K}}{\text{sd}(\log_{10} K)},$$

clipped to the published range, with $\bar{n}_e = 0.282$ and $s_n = 0.020$, the latter the published range read as a ± 3.2 standard deviation span over a 43-well network. Nothing in this construction is fitted to any result of the present paper.

The prior standard deviation follows the correlation-aware combination of the cited work. Under independent pathway errors the fused standard deviation is 0.0324; with the published facies correlations 0.15, 0.35 and 0.58 it becomes 0.0347, 0.0375 and 0.0402. Since the facies labels do not separate this dataset, the intermediate value 0.0375 is used throughout, and ignoring the correlation would understate the prior spread by 15.6 per cent.

The spatial structure comes from the measurement that samples the facies architecture directly (Figure 3b). An exponential variogram fitted by weighted least squares to the 43 measured $\log_{10} K$ values gives a nugget of 0.0000, a partial sill of 0.0590 and a range of 2057 m. The prior covariance is $B(x, x') = \sigma(x)\sigma(x')\exp(-|x - x'|/\ell)$ with ℓ that range, and the porosity field is expanded in its Karhunen-Loève basis,

$$n_e(x) = n_e^{prior}(x) + \sum_{k=1}^m \theta_k \phi_k(x), \quad \theta \sim N(0, \text{diag} \lambda),$$

truncated at $m = 150$ modes holding 78 per cent of the prior variance, the basis built by the Nyström method on a strided subset of the active cells.

4.2 Why porosity is invisible at steady state

The domain is discretised on a grid aligned with the regional flow azimuth, which makes the dispersion tensor diagonal in grid coordinates and removes the cross-derivative terms. The grid

has 389 by 474 cells of 100 m with 34887 active, covering 348.9 km² against 349.1 km² for the digitised polygon. Advection is first-order upwind, whose numerical dispersion is equivalent to a dispersivity of half a cell, 50 m, or a tenth of the baseline α_L . The dispersivity sweep of Section 6.8 spans several times that error, so the conclusions do not rest on the discretisation.

The gradient is not measurable from the workbook and is derived from the published mean seepage velocity of 2.1 m d⁻¹ and the measured mean conductivity, $i = v\bar{n}_e/\bar{K} = 4.53\text{e-}03$, giving a uniform regional Darcy flux $q_0 = 0.5922$ m d⁻¹. The depth-averaged transport equation is

$$n_e \frac{\partial C}{\partial t} + q_0 \frac{\partial C}{\partial \xi} = \frac{\partial}{\partial \xi} \left(\alpha_L q_0 \Gamma \frac{\partial C}{\partial \xi} \right) + \frac{\partial}{\partial \eta} \left(\alpha_T q_0 \Gamma \frac{\partial C}{\partial \eta} \right) + m_s(x),$$

with ξ and η the along- and across-flow coordinates and $\Gamma(x)$ the DRASTIC amplification of dispersion published for this aquifer, here between 1.00 and 1.50.

Written this way the identifiability question answers itself. The seepage velocity is $v = q_0/n_e$ and the mechanical dispersion coefficient is $D_L = \alpha_L v$, so the product $n_e D_L = \alpha_L q_0$ contains no porosity, and neither does the advective term once the flux rather than the velocity is taken uniform. Effective porosity survives only in the storage term. At steady state that term vanishes and the concentration field is blind to porosity. What porosity controls is how far a front has moved in a given time.

The transient is physically warranted here. At the published mean seepage velocity a traverse of the aquifer takes of the order of half a century, comparable with the history of intensive agricultural nitrogen loading, so the nitrate field cannot be assumed to have equilibrated.

4.3 The loading field is estimated, not assumed

A first attempt fixed the spatial shape of the loading to a power of the DRASTIC index and fitted one amplitude and one onset time. It failed. Across the eight exponents from 0 to 3 the best in-sample R^2 was 0.04, at exponent 1.00. The reason is the one Section 3 gives: the nitrate pattern follows position, not vulnerability, so a loading shape fixed a priori by vulnerability cannot reproduce it.

The loading is therefore expanded in a small set of large-scale spatial functions,

$$m_s(x) = \sum_j \beta_j \psi_j(x), \quad \psi \in \{1, \xi, \eta, \xi^2, \eta^2, \xi\eta, V\},$$

with ξ, η and the DRASTIC index V normalised to $[-1, 1]$. The transport equation is linear in the source, so the response to each basis function is computed once; it is autonomous with a constant source, so one march at fixed step gives the solution at every candidate onset time. The coefficients then follow from linear least squares at each onset time and the search reduces to a scan. Where least squares drives the loading negative, over 35 per cent of the domain, it is clipped and the amplitude refitted on the clipped shape, which is what the reported fit uses.

4.4 Fisher information and an exact discrete Jacobian

Differentiating the discrete transport system with respect to θ_k gives

$$\left(\frac{M}{\Delta t} + A \right) C_k^n = \frac{M}{\Delta t} C_k^{n-1} - \text{diag}(\varphi_k V_{\text{cell}}) \frac{C^n - C^{n-1}}{\Delta t},$$

the same left-hand side as the base march. One sparse factorisation therefore serves the base solve and all 150 modes, and the result is the exact derivative of the discrete system rather than a finite difference. Checked against finite differences at three step sizes the residual falls as a power 1.00 of the step, reaching 3.9e-04 at the smallest, which is the behaviour a correct derivative has and a coincidence does not.

The Fisher information decomposes additively,

$$I_{total} = \underbrace{\text{diag}(1/\lambda)}_{I_{prior}} + \underbrace{\sigma_{obs}^{-2} J^T J}_{I_{obs}},$$

and the posterior covariance is $\Sigma = I_{total}^{-1}$.

5. Design criteria

5.1 Weighting the parameter space does nothing to D-optimality

A recurring proposal is to weight a design criterion by a vulnerability field so that the design prefers vulnerable ground. For D-optimality this cannot work, and the reason is worth stating precisely because the construction looks reasonable.

Proposition 1. Let W be any fixed symmetric positive-definite weighting of the parameter space and let $\psi = W^{1/2}\theta$. The site maximising the D-optimal gain is the same under θ and under ψ .

Proof. Under the reparameterisation $I_\psi = W^{-1/2} I W^{-1/2}$ and $J_\psi(x) = J(x) W^{-1/2}$, so

$$\log \det (I_\psi + \sigma^{-2} J_\psi^T J_\psi) = \log \det (I + \sigma^{-2} J^T J) - \log \det W,$$

and the second term does not depend on x . The argmax is unchanged. $\square$

By the matrix determinant lemma the classical gain from one observation is

$$\Delta_D(x) = \log (1 + \sigma_{obs}^{-2} J(x) \Sigma J(x)^T),$$

and the set function $A \mapsto \log \det (I_{prior} + \sigma^{-2} \sum_{a \in A} J_a^T J_a)$ is monotone and submodular, so greedy selection carries the $(1 - 1/e)$ guarantee of Nemhauser et al. (1978), as exploited for sensor placement by Krause et al. (2008).

5.2 Goal orientation does change the design

What a protection-zone delineation depends on is not the porosity field but the time a contaminant takes to reach the discharge boundary, and it depends on that most where the aquifer is vulnerable. With a uniform flux the residual travel time from x is $\tau(x) = L(x)n_e(x)/q_0$ with $L(x)$ the remaining distance along the flow direction, so a porosity error maps linearly onto an arrival-time error. Define the vulnerability-weighted mean arrival-time error

$$g(\theta) = \frac{\sum_x w(x) L(x) \delta n_e(x)}{q_0 \sum_x w(x)} = c^T \theta, \quad w(x) = \left(\frac{V(x)}{\bar{V}} \right)^{\gamma_v},$$

a linear functional with $c = \Phi^T (wL)/(q_0 \sum w)$. The design minimises $\text{Var}(g) = c^T \Sigma c$.

Proposition 2. One observation at x reduces that variance by

$$\Delta_g(x) = \frac{\sigma_{obs}^{-2}(c^\top \Sigma J(x)^\top)^2}{1 + \sigma_{obs}^{-2} J(x) \Sigma J(x)^\top}.$$

Proof. Sherman-Morrison applied to $\Sigma^+ = (\Sigma^{-1} + \sigma^{-2} J^\top J)^{-1}$ gives $c^\top \Sigma^+ c = c^\top \Sigma c - (c^\top \Sigma J^\top)^2 / (\sigma^2 + J \Sigma J^\top)$. $\square$

Goal-oriented design in this sense is established in the inverse-problems literature (Attia et al., 2018) and has been applied to variable-density seawater intrusion in a companion preprint (Zeqiraj, 2026), where the objective is an adjoint expected information gain rather than a linear-functional variance. Unlike Proposition 1 this criterion is not invariant to the weighting: c enters the numerator and a different w selects a different direction in parameter space. Submodularity is a property of the log-determinant and is not claimed here; the greedy sequence is a heuristic and is reported as one.

5.3 Two diagnostics, and a pathology they expose

The prior covariance between the concentration at x and the porosity elsewhere,

$$\kappa_x(x') = \sum_k \lambda_k J(x, k) \varphi_k(x'),$$

is the kernel through which one observation sees the field. Its **information horizon** is the radius holding 90 per cent of its absolute mass. The **information correlation** between two candidate sites,

$$r_{info}(x_1, x_2) = \frac{J(x_1) \Sigma J(x_2)^\top}{\sqrt{J(x_1) \Sigma J(x_1)^\top J(x_2) \Sigma J(x_2)^\top}},$$

measures their redundancy.

The second diagnostic exposes a failure of greedy design that is not usually reported, and points at the scale that fixes it. On a 100 m grid whose information field varies over kilometres, the rank-one update at a chosen cell removes so little of the information at its neighbours that the next choice is the neighbour itself. Run without a constraint, the greedy D-optimal sequence here places its 5 wells within 283 m of one another at a maximum pairwise information correlation of 1.000: five wells buying one well's information.

The obvious remedy is a minimum separation, and the obvious scale to use is the variogram range, 2057 m. It is not enough. Imposing it lowers the maximum pairwise correlation only to 0.977, still within a few per cent of complete redundancy. The scale that has to be forbidden is the information horizon, 7500 m here, which is 3.6 times the variogram range; imposing that lowers the maximum pairwise correlation to 0.696. The reason is that redundancy between two observations is a property of the coherence of the observation kernel, not of the field: two wells inside one kernel width see the same thing however far apart the field decorrelates. Both constrained designs are reported below.

5.4 Concentration observation against pumping test

A pumping test constrains porosity at its own cell through the conductivity-to-porosity relation of Section 4.1, with standard deviation $\sigma_{n|K} = s_n(\sigma_K/K)/(\ln 10 \text{ sd}(\log_{10} K)) = 0.0038$. Its

Fisher contribution is rank one in the Karhunen-Loève basis with $\varphi(x)$ in place of $J(x)$, so the same closed form applies, and the information-equivalence factor is the ratio of best achievable gains,

$$R = \frac{\max_x \Delta_g^{conc}(x)}{\max_x \Delta_g^{pump}(x)},$$

the number of pumping tests one optimally placed concentration observation replaces.

6. Results

6.1 What the transport model achieves

With the loading expanded as in Section 4.3 the best fit has an onset time of 14.8 years, an in-sample RMSE of 4.46 mg L^{-1} and $R^2 = 0.48$. Dropping the DRASTIC column from the loading basis changes the unconstrained in-sample R^2 by 0.022, so vulnerability adds little once the regional trend is present. Leave-one-out cross-validation gives RMSE 6.32 mg L^{-1} and $R^2 = -0.04$.

The model therefore predicts a withheld well no better than the network mean, which is what Section 3 predicts for any method on these data. Eight parameters over 24 observations is enough to fit and not enough to generalise. The in-sample number alone would have read as a success.

6.2 How much the existing network is worth

Figure 4 shows what the assimilation buys, and the measure to quote is the degrees of freedom for signal (Rodgers, 2000), $\text{tr}(I_{obs}\Sigma)$, which counts how many directions of the parameter space the observations actually constrain. Out of 150 retained modes it is 0.0753. In log-determinant terms the 24 wells supply 0.038 nats, and averaged over the retained modes the porosity standard deviation falls by 0.17 per cent, from 0.03309 to 0.03304.

The trace ratio used in earlier work on this aquifer gives $5.1\text{e-}05$ for the observation share ($8.319\text{e-}02$ against $1.647\text{e+}03$), and is reported here for comparability only. It should not be the headline number: the trace of the prior information is dominated by the smallest prior variances, that is by the directions the prior already pins down, so it flatters the prior by construction. The two measures agree on the conclusion.

This is the quantitative form of Section 3, and it is a stronger statement than the cross-validation. The snapshot is not merely hard to extrapolate; measured against the prior it is very nearly uninformative about the quantity it is often assumed to constrain.

6.3 Designing more nitrate wells does not help much

The vulnerability-weighted mean residual arrival time under the prior is 22.5 years with a standard deviation of 246.3 days. Five additional concentration observations, optimally placed under either criterion, reduce that standard deviation to 238.9 days (D-optimal) and 238.8 days (goal-oriented), a reduction of 3.0 per cent. The two designs sit a median 224 m apart and differ in final variance by 0.1 per cent.

Figure 5 shows both designs. Both put their wells in ground whose mean DRASTIC index is 125.1 and 126.4 respectively, against a domain mean of 138.4: the locations where a concentration measurement is most informative about porosity are the less vulnerable ones, which is exactly the tension a goal-oriented criterion exists to resolve, and exactly the tension that cannot be resolved when there is no information to trade.

Under the wider separation constraint of one information horizon the figures are 242.9 and 242.8 days, so forcing the wells apart costs almost nothing here: the information being allocated is too small for its distribution to matter.

That the two criteria barely separate is not a defect of goal orientation. It follows from Section 6.2: when an observation type carries almost no information, there is almost nothing for a criterion to allocate, and every reasonable criterion returns nearly the same answer. The information horizon of a single concentration observation is 7500 m (range 6815 to 10504 m), 3.6 times the variogram range of 2057 m, which is another way of saying the same thing: the kernel is broad and flat.

6.4 One concentration observation is worth 0.09 pumping tests

The best single concentration observation reduces the variance of the weighted arrival time by $1.06\text{e}+03 \text{ d}^2$, the best single pumping test by $1.14\text{e}+04 \text{ d}^2$, giving $R = 0.09$.

This contradicts the factor of about three asserted for the same aquifer, and not marginally. The cause is identifiable. The earlier argument compares the spatial extent of the two sensitivity kernels and infers that the more extended one carries more information. Extent is not information. What enters the Fisher information is the squared amplitude of the kernel projected onto the directions the prior leaves uncertain, and a concentration observation in a nearly equilibrated field has a broad kernel of small amplitude. The pumping test constrains the field where it stands, with a porosity standard deviation of 0.0038, and on these data that is the better buy. Section 6.8 shows R stays at 0.02 to 0.33 across the measurement errors tested and 0.05 to 0.28 across the dispersivities, never approaching one.

6.5 Where design does pay

Concentration observations are not the only thing a campaign can buy. The other kind is a direct characterisation point: a cored borehole with a laboratory determination of effective porosity, or a downhole log calibrated to one. Section 2.3 noted that the present record contains none, and that is precisely why the placement question bites. A measurement that is expensive, scarce and strongly informative is the one whose siting is worth optimising; a cheap, abundant and weakly informative one is not. The operator is modelled as determining the field at its own location with standard deviation 0.020, a figure for a laboratory determination on an intact sample, and it is applied identically at both study sites so that no flow model enters the comparison.

With that operator the information is local, strong, and there is a great deal of it to allocate, and the choice of criterion matters a great deal.

At Fushë-Kuqe (Table 3, Figure 6a and 6b), five such points reduce the variance of the vulnerability-weighted mean by 3.1 per cent under D-optimality and 30.7 per cent under goal orientation, a factor of 10.0, and the two designs sit a median 7852 m apart. This is the separation Proposition 1 says cannot be obtained by weighting the D-criterion: it has to come

from changing what is being estimated. Figure 6a shows where the difference comes from. The D-optimal points sit on the domain boundary, which is where a smooth Gaussian prior is least constrained and therefore where the determinant gains most; the goal-oriented points move inland, to where the vulnerability weight and the remaining travel distance are both large. Boundary attraction is a known property of determinant criteria on bounded domains, and it is exactly the behaviour a decision-relevant objective has to overcome.

6.6 Independent replication

On the 44 wells of the Mississippi River Valley alluvial aquifer (Figure 6c), with the same local observation operator applied at both sites so that no flow model enters the comparison, the divergence is reproduced and is larger: 16.8 per cent variance reduction under D-optimality against 50.4 per cent under goal orientation, with a median separation of 59.5 km. Mean DRASTIC at the chosen sites is 160.2 for the goal-oriented design against 159.2 for the D-optimal one.

Table 3. Reduction in the variance of the vulnerability-weighted target under the two criteria, for 5 added observations. The first row uses the transport observation operator, which only the Albanian site supports; the last two use the local characterisation operator, which is identical at both sites.

Observation type and site	D-optimal (%)	Goal-oriented (%)	Median separation (m)
Transport observations, Fushe-Kuqe	5.9	6.0	224
Characterisation points, Fushe-Kuqe	3.1	30.7	7852
Characterisation points, MRVA	16.8	50.4	59506

The two sites differ by an order of magnitude in conductivity and by a continent in setting, and the conclusion transfers. That is the extent of the claim. The replication tests the design comparison, not the transport model, for which the MRVA dataset holds neither a flow field nor a solute record, and none was invented.

6.7 Posterior sampling, and what it does not show

The Gauss-Newton minimisation reaches its minimum in 12 iterations. The condition number of the Gauss-Newton Hessian is 62.7. Descent to a relative Riemannian gradient of 10^{-6} takes 1 iteration under the natural gradient against 686 under the Euclidean gradient.

That ratio should not be read as a general acceleration factor. On a quadratic model the natural gradient with unit step is Newton's method and terminates in one step by construction, so the comparison measures the conditioning of the problem and nothing about behaviour on a genuinely nonlinear cost. The often-quoted scaling by $\sqrt{\kappa}$, here $\sqrt{\kappa} = 7.9$, describes the Euclidean method's dependence on conditioning, not a property of the natural gradient, and the two should not be compared as though they were the same quantity.

Implicit sampling with 50 particles attains a normalised effective sample size (Figure 7a) of 0.989; sequential importance resampling from the prior with 500 particles attains 0.719. Both are high, and for the same reason the design comparison of Section 6.3 was flat: with the observation information as small as Section 6.2 finds, the posterior is close to the prior, the likelihood is nearly flat, and neither sampler is under strain. This is reported because the comparison is often presented as evidence of a method's power; on these data it is evidence of a weak likelihood.

6.8 Sensitivity of the conclusions

Every assumption marked as such in the configuration was varied one at a time. In the dispersivity sweep the fitted loading field and onset time are held at their baseline values, so what the sweep measures is the sensitivity of the information content to the transport parameter, not a refitting of the whole model; refitting at each dispersivity would confound the two.

Figure 7b to 7d and Table 4 collect the result. Nitrate measurement error from 0.5 to 2.0 mg L⁻¹ moves the observation share of the Fisher information over 1.3e-05 to 2.0e-04 and R over 0.02 to 0.33. Pumping-test relative error from 5 to 20 per cent moves R over 0.09 to 0.10. The vulnerability exponent γ_V from 0 to 3 moves the median separation of the two transport-operator designs only over 141 to 224 m, the two criteria staying within a tenth of a per cent of each other in final variance throughout. At $\gamma_V = 0$ the goal functional becomes the unweighted travel-time mean, which is still not the D-optimal objective, so a small separation remains; that it does not grow with γ_V is the same statement as Section 6.3, that there is too little information here for any criterion to allocate differently. Longitudinal dispersivity from 100 to 1000 m moves R over 0.05 to 0.28.

Table 4. Every assumption varied one at a time, and what moves.

Assumption varied	Range	R	Other effect
Nitrate measurement sd (mg L ⁻¹)	0.5 to 2	0.024 to 0.329	observation share 1.3e-05 to 2.0e-04
Pumping-test relative error	0.05 to 0.2	0.093 to 0.097	-
Longitudinal dispersivity (m)	100 to 1000	0.049 to 0.276	median separation 0 to 300 m
Vulnerability exponent	0 to 3	-	median separation 141 to 224 m, goal advantage 0.04 to 0.13%

No tested setting produces $R > 1$.

7. Discussion

7.1 What this corrects

Two things in the recent literature on this aquifer do not survive the computation, and a third needs qualifying.

The qualified one first. A nitrate prediction skill of $R^2 = 0.868$ is reported for this aquifer against 564 space-time measurements. The record available here is a snapshot of 31 values with no time dimension, so the two cannot be compared directly and no claim is made about that number. What can be said is what this snapshot supports: on it, nothing tested here beats the network mean out of sample, the best reaching +0.16, so a model evaluated only in sample on data of this density has not been tested against the possibility that it is fitting noise.

The facies-keyed parameter assignment is not supported by the workbook it is applied to, because in that workbook the lithological label and the measured conductivity are statistically independent ($p = 0.44$). This one is unqualified: it is a property of the file, checkable in one line.

The information-equivalence factor is 0.09, not three, and its practical direction is the reverse: for constraining effective porosity in this aquifer an additional pumping test buys more than an additional nitrate sample, by about a factor of 11. Nitrate sampling remains far cheaper per point and remains the right instrument for its own purposes, contaminant mapping and regulatory compliance among them. It is simply not an efficient instrument for porosity.

7.2 Why the negative results are the useful ones

A design calculation earns its keep where the intuition is wrong, and here the intuition that an integrating measurement must be informative fails for a reason that generalises. The Fisher information of an observation is the squared amplitude of its sensitivity projected onto the uncertain directions of the prior. A kernel can be broad and flat, sampling a large region and learning little from any of it, and a broad kernel is exactly what a regional concentration measurement in a slowly evolving plume has. The same reasoning says where concentration observations would be informative: in strongly transient plumes whose arrival time is still moving quickly. This aquifer is nearer that regime than a steady one, which is why R is not smaller still.

The second negative result, Proposition 1, is of the same kind. A weighted D-criterion looks like it should express a preference for vulnerable ground and it cannot, because D-optimality is invariant to exactly that operation. Getting the preference requires changing the estimand, not the metric.

7.3 Limitations

The flow field is a uniform regional flux with a single azimuth. This is the strongest simplification in the paper and it is forced by the absence of a head field. A calibrated multilayer flow model would change the sensitivity kernels and could change the placement, though the separation between the two design criteria under the local operator does not depend on the flow model at all.

The transport model is depth-averaged in a system the primary source describes as confined and multilayer with a thickness varying by a factor of twenty across the domain. Vertical structure is not represented.

The onset time of loading is estimated from 24 concentrations and is not independently constrained. It enters the porosity sensitivity directly, so the absolute magnitudes of Section 6.2 carry that uncertainty; the ratios of Sections 6.4 and 6.5 are less exposed.

The porosity prior is imported. If the published fusion is revised the prior moves and with it the posterior; the design machinery does not change.

Seawater intrusion affects roughly 15 per cent of the upper aquifer and is not represented. Chloride from the intruding wedge is a different tracer with a different source geometry, and the two chloride series available (Cenameri and Beqiraj, 2016) are too thin to constrain a field model. A variable-density treatment is the natural next step.

The goal functional is one linear functional. A vector-valued goal, for instance arrival times at several abstraction fields, would give a matrix criterion and possibly a different design.

7.4 What a manager should take from this

Three things, in the order they matter. The existing nitrate network should not be used to infer porosity, and a protection zone computed as though it does will be narrower than the evidence supports. If a choice must be made between one more pumping test and one more nitrate sample for the purpose of constraining transport timing, on these data the pumping test is worth about 11 of the nitrate samples. And when characterisation points are placed, the criterion should target the quantity the decision depends on, which on this aquifer reduces the relevant variance by a factor of 10.0 more than the conventional criterion does.

8. Conclusions

8. On a 24-well nitrate snapshot in a 349.1 km² alluvial aquifer, no interpolator and no transport model tested predicts a held-out well better than the network mean; the best leave-one-out R^2 reached by anything is +0.16.
9. Under a uniform regional flux the steady concentration field is independent of effective porosity. Identifiability lives in the transient, and on these data the nitrate observations constrain 0.0753 degrees of freedom out of 150, supplying 0.038 nats.
10. One optimally placed concentration observation is worth 0.09 pumping tests for this purpose, not three, across every assumption tested.
11. Weighting the parameter space by vulnerability leaves the D-optimal site unchanged, for any weighting. Designs claiming otherwise rely on an invariance they do not have.
12. Goal orientation on the vulnerability-weighted arrival time reduces the targeted variance by a factor of 10.0 more than D-optimality at Fushë-Kuqe and reproduces that divergence on 44 independent USGS wells.
13. Unconstrained greedy design on a smooth information field returns mutually redundant sites, here 5 wells within 283 m at information correlation 1.000. A minimum separation of one variogram range is not enough to break it (0.977); the scale that works is the information horizon (0.696).

9. Acknowledgements

The hydrogeological data were compiled from records of the Albanian Geological Survey and of the Institute of GeoSciences, Energy, Water and Environment at the Polytechnic University of Tirana. The Mississippi Alluvial Plain conductivity data are a public release of the U.S. Geological Survey.

10. Data availability

The complete package, comprising the field data in flat form, the MATLAB pipeline, the results file every number in this paper is quoted from, and the scripts that draw every figure, is at

- GitHub: <https://github.com/dulianzeqiraj/fushe-kuqe-oed>
- Zenodo (archived release): <https://doi.org/10.5281/zenodo.22726726>

One command reproduces everything reported here. The primary sources are cited in Section 2.

11. Declaration of competing interests

The author declares no competing interests.

12. CRediT author statement

Dulian Zeqiraj: conceptualization, methodology, software, formal analysis, investigation, data curation, writing, visualization.

Figure captions

Figure 1. The Fushë-Kuqe aquifer. (a) The 3411-vertex digitised boundary with all 180 monitoring points, the 43 carrying a pumping-test conductivity and the 31 carrying a nitrate concentration. (b) The DRASTIC vulnerability field kriged from all 180 points, with the regional flow direction reported by Cenameri and Beqiraj (2016). (c) The effective-porosity prior, keyed on measured $\log_{10} K$ and scaled to the published network statistics.

Figure 2. Nothing predicts a held-out nitrate well. (a) Leave-one-out R^2 for every method tested, against the network-mean baseline at zero. (b) Modelled against observed nitrate, in sample and under leave-one-out, with the 1:1 line.

Figure 3. Prior and loading. (a) The Karhunen-Loève spectrum of the prior covariance. (b) The empirical variogram of the 43 measured $\log_{10} K$ values with the fitted exponential model. (c) The loading field estimated from the 24 nitrate observations, with its fitted onset time.

Figure 4. What the existing network buys. (a) Modelled nitrate at the fitted onset time. (b) Prior standard deviation of effective porosity. (c) The reduction in that standard deviation produced by assimilating all 24 nitrate observations, on the same colour scale.

Figure 5. Design with transport observations. (a) The one-shot variance gain, with the sites the unconstrained greedy sequence returns: 5 wells within 283 m. (b) The two criteria under a minimum separation of 2057 m, over the vulnerability field. (c) The standard deviation of the vulnerability-weighted arrival time against the number of wells added.

Figure 6. Design with characterisation points, where the criterion matters. (a) The two designs at Fushë-Kuqe over the vulnerability field. (b) Variance of the goal functional relative to its prior value, at both sites and under both criteria. (c) The 44 USGS wells of the Mississippi Alluvial Plain with the two designs.

Figure 7. Sampling and sensitivity. (a) Normalised effective sample size for implicit sampling and for sequential importance resampling from the prior. (b) The information-equivalence factor against nitrate measurement error, with unity marked. (c) The same against longitudinal dispersivity. (d) Median separation of the two designs and the goal-oriented advantage against the vulnerability exponent.

Graphical abstract. The 24 nitrate wells of the network, the 0.0753 directions of a 150-dimensional porosity field they constrain, and the variance reduction the two design criteria achieve when the observation is a characterisation point instead.

References

Alexanderian, A., 2021. Optimal experimental design for infinite-dimensional Bayesian inverse problems governed by PDEs: a review. *Inverse Problems* 37(4), 043001. <https://doi.org/10.1088/1361-6420/abe10c>

Attia, A., Alexanderian, A., Saibaba, A.K., 2018. Goal-oriented optimal design of experiments for large-scale Bayesian linear inverse problems. *Inverse Problems* 34(9), 095009. <https://doi.org/10.1088/1361-6420/aad210>

Cenameri, S., Beqiraj, A., 2016. Assessment of seawater intrusion in Fushe Kuqe aquifer, Albania. *Bulletin of the Geological Society of Greece* 50(2), 665. <https://doi.org/10.12681/bgsg.11772>

Chaloner, K., Verdinelli, I., 1995. Bayesian experimental design: a review. *Statistical Science* 10(3), 273–304. <https://doi.org/10.1214/ss/1177009939>

Chorin, A.J., Morzfeld, M., Tu, X., 2010. Implicit particle filters for data assimilation. *Communications in Applied Mathematics and Computational Science* 5(2), 221–240. <https://doi.org/10.2140/camcos.2010.5.221>

Chorin, A.J., Tu, X., 2009. Implicit sampling for particle filters. *Proceedings of the National Academy of Sciences* 106(41), 17249–17254. <https://doi.org/10.1073/pnas.0909196106>

Fedorov, V.V., 1972. *Theory of Optimal Experiments*. Academic Press, New York.

Gelhar, L.W., Welty, C., Rehfeldt, K.R., 1992. A critical review of data on field-scale dispersion in aquifers. *Water Resources Research* 28(7), 1955–1974. <https://doi.org/10.1029/92WR00607>

Herrera, G.S., Pinder, G.F., 2005. Space-time optimization of groundwater quality sampling networks. *Water Resources Research* 41(12), W12407. <https://doi.org/10.1029/2004WR003626>

Krause, A., Singh, A., Guestrin, C., 2008. Near-optimal sensor placements in Gaussian processes: theory, efficient algorithms and empirical studies. *Journal of Machine Learning Research* 9(8), 235–284.

- Leube, P.C., Geiges, A., Nowak, W., 2012. Bayesian assessment of the expected data impact on prediction confidence in optimal sampling design. *Water Resources Research* 48(2), W02501. <https://doi.org/10.1029/2010WR010137>
- Lindley, D.V., 1956. On a measure of the information provided by an experiment. *The Annals of Mathematical Statistics* 27(4), 986–1005. <https://doi.org/10.1214/aoms/1177728069>
- Morzfeld, M., Tu, X., Atkins, E., Chorin, A.J., 2012. A random map implementation of implicit filters. *Journal of Computational Physics* 231(4), 2049–2066. <https://doi.org/10.1016/j.jcp.2011.11.022>
- Nemhauser, G.L., Wolsey, L.A., Fisher, M.L., 1978. An analysis of approximations for maximizing submodular set functions. *Mathematical Programming* 14(1), 265–294. <https://doi.org/10.1007/BF01588971>
- Nowak, W., de Barros, F.P.J., Rubin, Y., 2010. Bayesian geostatistical design: task-driven optimal site investigation when the geostatistical model is uncertain. *Water Resources Research* 46(3), W03535. <https://doi.org/10.1029/2009WR008312>
- Pugh, A.L., 2023. Hydraulic conductivity and transmissivity estimates from slug tests in wells within the Mississippi Alluvial Plain, Arkansas and Mississippi, 2020. U.S. Geological Survey Scientific Investigations Report 2023-5101. <https://doi.org/10.3133/sir20235101>
- Pukelsheim, F., 2006. *Optimal Design of Experiments*. Society for Industrial and Applied Mathematics, Philadelphia. <https://doi.org/10.1137/1.9780898719109>
- Rodgers, C.D., 2000. *Inverse Methods for Atmospheric Sounding: Theory and Practice*. World Scientific, Singapore. <https://doi.org/10.1142/3171>
- Zeqiraj, D., 2026. Goal-oriented Bayesian experimental design for variable-density seawater intrusion: adjoint expected information gain for joint monitoring and managed-aquifer-recharge placement. Preprint, SSRN. <https://doi.org/10.2139/ssrn.6866554>
- Zeqiraj, D., Beqiraj, A., 2026. Adaptive stacked ensemble fusion: Bayesian porosity inference in alluvial aquifers when the errors are not independent. *Journal of Contaminant Hydrology* 283, 105086. <https://doi.org/10.1016/j.jconhyd.2026.105086>
- Zeqiraj, D., Beqiraj, A., Jahja, A., Seitaj, B., Progni, F., Zeqo, E., 2026. Physical, biophysical, and chemical mechanisms of bidirectional coupling between DRASTIC vulnerability assessment and numerical transport modeling in heterogeneous aquifer systems. *Journal of Hazardous Materials Advances* 23, 101261. <https://doi.org/10.1016/j.hazadv.2026.101261>

Appendix A. MATLAB code

The listings below are the complete pipeline, in the order it runs them. They are reproduced verbatim from the released repository; the sixteen-character SHA-256 prefix given with each file identifies the exact version that produced every number in this paper. Nothing follows the listings.

File	Lines	SHA-256 prefix
fk_config.m	122	799015aec71adef

File	Lines	SHA-256 prefix
fk_load_data.m	63	2636d95be48a3679
fk_predictability.m	106	4be119fd350eebb9
fk_build_grid.m	67	dc9cd0083c88d2a7
fk_prior_field.m	199	d22e03def2a8007f
fk_forward.m	153	933610dda1c842af
fk_calibrate.m	217	00ac93fa2938cbfe
fk_sensitivity.m	132	b4093c40f3463b5b
fk_design.m	245	1214b65e46b106da
oed_local.m	71	35085400a56b59a9
fk_particles.m	151	12803914034129f3
mrva_replicate.m	146	4bd87060b4c23b70
fk_sweep.m	145	00194ccd2cfa0411
fk_write_results.m	203	14dda76387c28cdf
fk_figures.m	291	4d79a9be94d6296a
fk_prepare_data.m	154	d70f3c35185694d8
fk_fill_manuscript.m	363	4ecaff890d0ead90
fk_latex2omml.m	369	1c63f0bdb804d970
fk_xmlesc.m	9	f33903379adb07d1
fk_check_omml.m	48	0aaec357da997185
fk_md2docx.m	305	0c7fb33ead18c453
fk_zip_replace.m	91	0f13f458e8c3562c
fk_make_template.m	48	ee75001a80342d54
fk_build_docx.m	96	492467fda49b3b09
fk_verify_docx.m	150	afd10246b7fd16b2
fk_sha256.m	18	c475d86ff2bf570f
run_all.m	65	86ba7b72db52e4f4

A.1 `fk_config.m`

```
function cfg = fk_config()
%FK_CONFIG Every constant used by the pipeline, with its provenance.
%
% Nothing in this file is tuned to reproduce a target result. Each value is
% either (a) measured and present in data/, (b) taken from a cited published
% source, or (c) declared here as a modelling assumption and swept in the
% sensitivity analysis (see run_all.m, stage S).
%
% Provenance tags:
%   [DATA] read from the field workbook, nothing assumed
%   [PUB]  published value, citation given
%   [ASSUM] modelling assumption made in this paper, swept in sensitivity
```

```

% [DERIV] derived here from [DATA] and [PUB] quantities, formula given

cfg = struct();

% ----- paths ---
here = fileparts(mfilename('fullpath'));
cfg.dir_code    = here;
cfg.dir_data    = fullfile(here, '..', 'data');
cfg.dir_results = fullfile(here, '..', 'results');
cfg.dir_figures = fullfile(here, '..', 'figures');

% ----- numerical ---
cfg.dx = 100;          % [ASSUM] cell size, m. Advection is first-order
cfg.dy = 100;          % upwind, whose numerical dispersion is
                        % equivalent to a dispersivity of  $dx/2 =$ 
                        % 50 m, ten per cent of the baseline
                        %  $\alpha_L$  of 500 m. The dispersivity
                        % sweep spans that error several times
                        % over, so the conclusions do not rest
                        % on the discretisation.
cfg.n_kl = 150;        % [ASSUM] Karhunen-Loeve modes retained
cfg.nystrom_stride = 8; % [ASSUM] coarse-subgrid stride for Nystrom

% ----- aquifer / flow physics ---
cfg.b_aquifer = 40;    % [PUB] depth-averaged saturated thickness, m.
                        % Zeqiraj et al. 2026, J Hazard Mater Adv
                        % 23, 101261. The primary hydrogeological
                        % description (Cenameri & Beqiraj 2016)
                        % gives a confined multilayer system whose
                        % gravel-sand thickness grows from 5-10 m
                        % in the east to 180-200 m in the west; the
                        % single value is a depth-averaged 2-D
                        % idealisation and is stated as such.
cfg.v_mean_pub = 2.1;  % [PUB] mean seepage velocity, m/day, same source
cfg.alphaL = 500;      % [ASSUM] longitudinal dispersivity, m, from the
                        % Gelhar et al. (1992) scale dependence
                        %  $\alpha_L \sim 0.1 L$  at a transport scale of
                        % a few km. Swept over 100-1000 m.
cfg.alphaT_ratio = 0.1; % [PUB]  $\alpha_T = \alpha_L/10$ , Zheng & Bennett 2002
cfg.Dm = 1.9e-9 * 86400; % [PUB] molecular diffusion of nitrate,  $m^2/day$ 
                        % ( $1.9e-9 m^2/s$ , Li & Gregory 1974)

% Regional flow direction. The primary hydrogeological description of this
% aquifer (Cenameri & Beqiraj 2016, Bull. Geol. Soc. Greece 50(2), 665,
% doi:10.12681/bgsg.11772) gives flow from the Mat river outlet in the
% northeast to the Adriatic discharge boundary in the southwest, with the
% seawater wedge advancing in the opposite sense. A single regional azimuth
% is therefore used rather than the rotating field of earlier work.
cfg.azim_flow_deg = 225; % [PUB] flow towards SW (NE -> SW)
% Gradient magnitude is not measured: the workbook records depth to water, not
% head, and no DEM accompanies it. It is derived from the published mean
% seepage velocity and the measured mean K as  $i = v_{n_e} / K$ . [DERIV]

```

```

cfg.i_gradient = [];          % filled in fk_flow_field from measured K

% ----- DRASTIC-transport link -----
% Vulnerability amplification of the dispersion tensor, the form published in
% Zeqiraj et al. 2026 (J Hazard Mater Adv 23, 101261), Eq. (6).
cfg.alpha_V = 0.5;          % [ASSUM] amplitude, swept 0 - 1
cfg.gamma_V = 1.0;          % [ASSUM] exponent, swept 0.5 - 2

% ----- porosity prior -----
% Facies reference ranges for effective porosity, and the pathway error
% correlations, are taken from the fusion already published for this aquifer:
% Zeqiraj & Beqiraj 2026, J Contam Hydrol 283, 105086 (Table 1, Eq. 7)
% This paper does not re-derive them; it consumes them as the prior. [PUB]
cfg.facies_names = {'Gravel - coarse-grained', ...
                    'Gravel - medium-grained', ...
                    'Gravel - fine-grained'};
cfg.facies_short = {'coarse', 'medium', 'fine'};
cfg.ne_lo = [0.30, 0.28, 0.25]; % [PUB] lower end of reference range
cfg.ne_hi = [0.34, 0.32, 0.29]; % [PUB] upper end of reference range
cfg.rho_pathway = [0.15, 0.35, 0.58]; % [PUB] facies pathway-error correlation
cfg.sigma_VS = 0.042; % [PUB] Vukovic-Soro pathway sd
cfg.sigma_KC = 0.051; % [PUB] Kozeny-Carman pathway sd

% ----- observations -----
cfg.sigma_obs = 1.0; % [ASSUM] nitrate measurement sd, mg/L. Swept
% 0.5 - 2.0 mg/L.
cfg.sigma_K_rel = 0.10; % [ASSUM] relative sd of a pumping-test K, used
% only for the information-equivalence
% comparison. Swept 0.05 - 0.20.

% ----- calibration -----
cfg.cal_dt = 200; % time step of the calibration march, days
cfg.cal_nsteps = 400; % 80 000 days = 219 yr horizon, comfortably
% longer than any plausible loading history
cfg.cal_kappas = [0 0.25 0.5 0.75 1 1.5 2 3]; % DRASTIC loading exponents

% ----- design -----
cfg.n_new_wells = 5; % how many augmentation sites to place
cfg.gamma_Vdesign = 1.5; % [ASSUM] vulnerability weighting exponent in
% the D_V criterion. Swept 0 - 3.
cfg.horizon_frac = 0.90; % fraction of sensitivity-kernel mass defining
% the information horizon
cfg.sd_local = 0.02; % [ASSUM] sd of a direct porosity determination
% at a characterisation point, used by
% the local observation operator that
% both sites share
cfg.mrva_dx = 1000; % cell size of the MRVA grid, m (the well
% spacing there is tens of km)

% ----- particle step -----
cfg.gn_iters = 12; % Gauss-Newton iterations for the MAP estimate
cfg.max_desc = 20000; % cap on the descent-comparison iteration count

```

```

cfg.M_particles = 50;           % primary implicit-sampling particles
cfg.M_sir = 500;               % particles for the SIR comparison
cfg.seed = 20260912;          % fixed seed, reported in the paper

end

```

A.2 fk_load_data.m

```

function D = fk_load_data(cfg)
%FK_LOAD_DATA Read the field tables written by prepare_data.py.
%
% Returns a struct with the well tables, the aquifer boundary polygon and
% the facies index of every well that carries a pumping-test K. Nothing is
% imputed: wells without a measurement keep a NaN.

opts = detectImportOptions(fullfile(cfg.dir_data, 'wells_all.csv'), ...
    'TextType', 'string');
T = readtable(fullfile(cfg.dir_data, 'wells_all.csv'), opts);

D = struct();
D.well_id = string(T.Well_ID);
D.x = T.X_GaussKruger;
D.y = T.Y_GaussKruger;
D.K = T.K_m_per_day;           % m/day, NaN where not measured
D.NO3 = T.NO3_mg_per_L;        % mg/L, NaN where not measured
D.DRASTIC = T.DRASTIC_Index;    % dimensionless, all 180 wells
D.depth_w = T.Piezometric_Level_m; % depth to water, m
D.lith = string(T.Lithology);
D.inside = strcmp(string(T.Inside_Aquifer), "YES");

% facies index 1 coarse, 2 medium, 3 fine, 0 if the log gives no lithology
D.facies = zeros(numel(D.well_id), 1);
for f = 1:numel(cfg.facies_names)
    D.facies(D.lith == string(cfg.facies_names{f})) = f;
end

D.has_K = ~isnan(D.K);
D.has_NO3 = ~isnan(D.NO3);

B = readtable(fullfile(cfg.dir_data, 'boundary.csv'));
D.bx = B.X_GaussKruger;
D.by = B.Y_GaussKruger;

% ---- inventory, printed so that every count in the paper is traceable ----
D.n_total = numel(D.well_id);
D.n_inside = sum(D.inside);
D.n_K = sum(D.has_K);
D.n_NO3 = sum(D.has_NO3);
D.n_inside_K = sum(D.inside & D.has_K);
D.n_inside_NO3 = sum(D.inside & D.has_NO3);
D.n_boundary = numel(D.bx);

fprintf('[load] wells total %d, inside %d, with K %d, with NO3 %d\n', ...

```

```

        D.n_total, D.n_inside, D.n_K, D.n_NO3);
fprintf('[load] inside with K %d, inside with NO3 %d, boundary vertices %d\n', ..
.
        D.n_inside_K, D.n_inside_NO3, D.n_boundary);
for f = 1:3
    fprintf('[load] facies %-7s : %d wells with K\n', cfg.facies_short{f}, ...
        sum(D.has_K & D.facies == f));
end
fprintf('[load] K   range %.0f - %.0f m/day, mean %.1f, sd %.1f\n', ...
        min(D.K(D.has_K)), max(D.K(D.has_K)), ...
        mean(D.K(D.has_K)), std(D.K(D.has_K)));
fprintf('[load] NO3 range %.2f - %.2f mg/L, mean %.2f, sd %.2f\n', ...
        min(D.NO3(D.has_NO3)), max(D.NO3(D.has_NO3)), ...
        mean(D.NO3(D.has_NO3)), std(D.NO3(D.has_NO3)));
fprintf('[load] DRASTIC range %g - %g, mean %.2f, sd %.2f (n = %d)\n', ...
        min(D.DRASTIC), max(D.DRASTIC), mean(D.DRASTIC), std(D.DRASTIC), ...
        sum(~isnan(D.DRASTIC)));
end

```

A.3 fk_predictability.m

```

function Pr = fk_predictability(cfg, D)
%FK_PREDICTABILITY Is the measured nitrate field predictable at all?
%
% Before any transport model is blamed for a poor cross-validated fit, the
% question has to be asked of the data. This routine takes the nitrate
% values at the wells inside the aquifer and asks how well each of them can
% be predicted from the others by methods that make no physical assumption
% whatsoever: inverse-distance weighting, ordinary kriging over a wide range
% of correlation lengths, and linear regression on the available covariates.
%
% The number that matters is the leave-one-out coefficient of determination.
% A value at or below zero means the method does worse than predicting the
% network mean, which is the honest baseline.

ob = D.inside & D.has_NO3;
x = D.x(ob); y = D.y(ob); z = D.NO3(ob);
V = D.DRASTIC(ob); K = D.K(ob); dw = D.depth_w(ob);
n = numel(z);
sst = sum((z - mean(z)) .^ 2);
r2 = @(p) 1 - sum((z - p) .^ 2) / sst;
Pr.n = n;

% ----- inverse distance weights -----
Pr.idw_power = [1 2 3];
Pr.idw_R2 = zeros(1, 3);
for q = 1:3
    pw = Pr.idw_power(q);
    pred = zeros(n, 1);
    for i = 1:n
        m = true(n, 1); m(i) = false;
        d = max(hypot(x(m) - x(i), y(m) - y(i)), 1);
        w = d .^ (-pw);
    end
end

```

```

        pred(i) = sum(w .* z(m)) / sum(w);
    end
    Pr.idw_R2(q) = r2(pred);
    fprintf('[pred] leave-one-out inverse distance, power %d : R2 = %+.3f\n', ...
        pw, Pr.idw_R2(q));
end

% ----- ordinary kriging
Pr.krig_range = [1000 2000 4000 8000 16000];
Pr.krig_R2 = zeros(size(Pr.krig_range));
nug = 0.2 * var(z); sill = 0.8 * var(z);
for q = 1:numel(Pr.krig_range)
    rg = Pr.krig_range(q);
    pred = zeros(n, 1);
    for i = 1:n
        m = true(n, 1); m(i) = false;
        xs = x(m); ys = y(m); zs = z(m); k = numel(xs);
        Dd = hypot(xs - xs', ys - ys');
        Gm = nug + sill * (1 - exp(-Dd / rg));
        Gm(1:k+1:end) = 0;
        A = [Gm, ones(k, 1); ones(1, k), 0] + 1e-8 * eye(k + 1);
        g0 = nug + sill * (1 - exp(-hypot(xs - x(i), ys - y(i)) / rg));
        w = A \ [g0; 1];
        pred(i) = w(1:k)' * zs;
    end
    Pr.krig_R2(q) = r2(pred);
    fprintf('[pred] leave-one-out ordinary kriging, range %6d m : R2 = %+.3f\n',
        ...
        rg, Pr.krig_R2(q));
end

% ----- linear regressions --
sets = {'X,Y', [x y]}, ...
        {'X,Y,DRASTIC', [x y V]}, ...
        {'X,Y,DRASTIC,K,depth', [x y V K dw]}, ...
        {'DRASTIC', V}, ...
        {'depth to water', dw}};
Pr.lin_name = cell(1, numel(sets));
Pr.lin_R2 = zeros(1, numel(sets));
for q = 1:numel(sets)
    A = [ones(n, 1), sets{q}{2}];
    pred = zeros(n, 1);
    for i = 1:n
        m = true(n, 1); m(i) = false;
        b = A(m, :) \ z(m);
        pred(i) = A(i, :) * b;
    end
    Pr.lin_name{q} = sets{q}{1};
    Pr.lin_R2(q) = r2(pred);
    fprintf('[pred] leave-one-out linear on %-22s : R2 = %+.3f\n', ...
        sets{q}{1}, Pr.lin_R2(q));
end

```

```

% ----- what the field is organised by
% Reported because the choice of loading model in Section 4.3 turns on it: the
% nitrate pattern follows position far more closely than it follows the
% vulnerability index.
xi_w = x * cosd(90 - 225) + y * sind(90 - 225);
Pr.corr_northing = corr(z, y);
Pr.corr_easting  = corr(z, x);
Pr.corr_alongflow = corr(z, xi_w);
Pr.corr_drastic  = corr(z, V);
Pr.corr_K        = corr(z, K);
Pr.corr_depth    = corr(z, dw);
fprintf(['[pred] nitrate correlates with northing %+.2f, easting %+.2f, ' ...
        'along-flow %+.2f\n'], Pr.corr_northing, Pr.corr_easting, ...
        Pr.corr_alongflow);
fprintf(['[pred] nitrate correlates with DRASTIC %+.2f, K %+.2f, ' ...
        'depth to water %+.2f\n'], Pr.corr_drastic, Pr.corr_K, Pr.corr_depth);

Pr.best_R2 = max([Pr.idw_R2, Pr.krig_R2, Pr.lin_R2]);
fprintf(['[pred] best leave-one-out R2 achieved by any of these methods: ' ...
        '%+.3f\n'], Pr.best_R2);

end

```

A.4 fk_build_grid.m

```

function G = fk_build_grid(cfg, D)
%FK_BUILD_GRID Flow-aligned regular grid over the aquifer polygon.
%
% The grid axes are rotated onto the regional flow direction reported by
% Cenameri & Beqiraj (2016): northeast to southwest. Aligning the grid with
% the flow makes the dispersion tensor diagonal in grid coordinates, which
% removes the cross-derivative terms from the transport operator and with
% them the main source of discretisation error in a rotated-flow problem.
%
% Fields
%   G.exi, G.eta    unit vectors of the rotated frame in real-world coordinates
%   G.xa,  G.ya     real-world coordinates of the active cell centres
%   G.ia,  G.ja     rotated-frame (row, column) index of each active cell
%   G.idx          (ny x nx) map from grid position to active-cell number

az = cfg.azim_flow_deg;
th = deg2rad(90 - az);                % compass azimuth to mathematical angle
exi = [cos(th), sin(th)];             % unit vector along the flow
eta = [-sin(th), cos(th)];            % unit vector across the flow

% rotated coordinates of the boundary polygon
xi_b = D.bx * exi(1) + D.by * exi(2);
eta_b = D.bx * eta(1) + D.by * eta(2);

xi0 = min(xi_b); xi1 = max(xi_b);
et0 = min(eta_b); et1 = max(eta_b);

nx = ceil((xi1 - xi0) / cfg.dx);      % columns advance along the flow

```

```

ny = ceil((et1 - et0) / cfg.dy);          % rows advance across the flow

xic = xi0 + (0.5:1:(nx - 0.5)) * cfg.dx;
etc = et0 + (0.5:1:(ny - 0.5)) * cfg.dy;
[XI, ET] = meshgrid(xic, etc);

% back to real-world coordinates (the frame is orthonormal, so the inverse is
% the transpose)
XX = XI * exi(1) + ET * eta(1);
YY = XI * exi(2) + ET * eta(2);

in = inpolygon(XX, YY, D.bx, D.by);

G = struct();
G.exi = exi; G.eta = eta; G.theta = th;
G.nx = nx; G.ny = ny;
G.xi = xic; G.eta_c = etc;
G.X = XX; G.Y = YY;
G.mask = in;
G.idx = zeros(ny, nx);
G.idx(in) = 1:nnz(in);
G.n_active = nnz(in);
G.cell_area = cfg.dx * cfg.dy;
G.area_km2 = G.n_active * G.cell_area / 1e6;
G.xa = XX(in);
G.ya = YY(in);
[G.ia, G.ja] = find(in);                  % row (across flow), column (along flow)

% area of the digitised polygon itself, for the discretisation check
G.area_polygon_km2 = polyarea(D.bx, D.by) / 1e6;

fprintf(['[grid] flow azimuth %g deg, rotated grid %d x %d cells of %g m\n'], ...
        az, ny, nx, cfg.dx);
fprintf(['[grid] %d active cells, discretised area %.2f km2 ' ...
        '(polygon %.2f km2, difference %.2f%%)\n'], ...
        G.n_active, G.area_km2, G.area_polygon_km2, ...
        100 * abs(G.area_km2 - G.area_polygon_km2) / G.area_polygon_km2);
end

```

A.5 fk_prior_field.m

```

function P = fk_prior_field(cfg, D, G)
%FK_PRIOR_FIELD Porosity prior, vulnerability field and the KL basis.
%
% The porosity prior is NOT re-derived here. The Dual-Pathway fusion for this
% aquifer is already published (Zeqiraj et al. 2026, J Hazard Mater Adv 23,
% 101261; Zeqiraj & Beqiraj 2026, J Contam Hydrol 283, 105086). This routine
% consumes that result: the facies reference ranges and the pathway-error
% correlations are read from cfg, the measured K of each well places it inside
% its facies range, and the correlation-aware variance of the cited work sets
% the prior spread.
%
% Outputs

```

```

% P.ne_well    porosity prior at each of the 43 wells with a measured K
% P.sd_well    prior sd at each of those wells (facies dependent)
% P.sd_iv      the independence-assumption sd, for the comparison in the text
% P.vrange     correlation length fitted to the measured log10 K, m
% P.ne         kriged porosity prior on the grid
% P.sd         kriged prior sd on the grid
% P.V          kriged DRASTIC vulnerability on the grid
% P.Phi,P.lam  KL basis of the prior covariance (n_active x n_kl, n_kl x 1)

kk = D.has_K & D.facies > 0;
P.well_idx = find(kk);
nK = numel(P.well_idx);
xw = D.x(kk); yw = D.y(kk); Kw = D.K(kk); fw = D.facies(kk);

% ----- does the lithology label carry K? --
% The porosity prior would normally be keyed on facies. In this workbook the
% lithological descriptor and the pumping-test K are almost independent, so a
% facies-keyed prior would impose structure the data do not contain. The test
% is reported because it governs the choice made immediately below.
Kf_mean = zeros(1, 3); Kf_sd = zeros(1, 3); Kf_n = zeros(1, 3);
for f = 1:3
    s = (fw == f);
    Kf_n(f) = sum(s); Kf_mean(f) = mean(Kw(s)); Kf_sd(f) = std(Kw(s));
    fprintf('[prior] facies %-7s n=%2d K %3g-%3g mean %.1f sd %.1f m/day\n', ..
        .
        cfg.facies_short{f}, Kf_n(f), min(Kw(s)), max(Kw(s)), ...
        Kf_mean(f), Kf_sd(f));
end
grand = mean(Kw);
ss_between = sum(Kf_n .* (Kf_mean - grand).^2);
ss_total = sum((Kw - grand).^2);
P.eta2_facies_K = ss_between / ss_total;
dfb = 2; dfw = nK - 3;
F = (ss_between / dfb) / ((ss_total - ss_between) / dfw);
P.F_facies_K = F;
P.p_facies_K = 1 - fcdf(F, dfb, dfw);
fprintf(['[prior] one-way ANOVA of K on the lithology label: ' ...
    'eta^2 = %.4f, F(%d,%d) = %.3f, p = %.3f\n'], ...
    P.eta2_facies_K, dfb, dfw, F, P.p_facies_K);

% ----- prior mean at the wells --
% The label carries almost no information about K, so the spatial pattern of
% the prior is taken from the measurement that does: a monotone map of log10 K
% onto the published network porosity statistics. Both constants come from
% the fusion already published for this aquifer (network mean 0.282, network
% range 0.22-0.35; Zeqiraj et al. 2026, J Hazard Mater Adv 23, 101261,
% Table 3). Nothing here is fitted to any result of the present paper.
lK = log10(Kw);
z = (lK - mean(lK)) / std(lK);
P.ne_network_mean = 0.282; % [PUB]
P.ne_network_sd = 0.020; % [DERIV] published range 0.22-0.35 over
% a 43-well network is +-3.2 sd

```

```

ne_w = P.ne_network_mean + P.ne_network_sd * z;
ne_w = min(max(ne_w, 0.22), 0.35); % [PUB] published physical range
fprintf('[prior] prior mean from log10 K: %.4f - %.4f, network mean %.4f\n', ...
        min(ne_w), max(ne_w), mean(ne_w));

% ----- prior sd -----
% Independence (inverse-variance) combination, and the correlation-aware BLUE
% variance of Zeqiraj & Beqiraj (2026), Eq. (2). Because the facies labels do
% not separate this dataset, the network-representative medium-facies
% correlation is used, and rho is swept over the full published range in the
% sensitivity analysis.
s1 = cfg.sigma_VS; s2 = cfg.sigma_KC;
P.sd_iv = sqrt(s1^2 * s2^2 / (s1^2 + s2^2));
sd_facies = zeros(1, 3);
for f = 1:3
    r = cfg.rho_pathway(f);
    sd_facies(f) = sqrt((1 - r^2) * s1^2 * s2^2 / (s1^2 + s2^2 - 2*r*s1*s2));
end
P.rho_used = cfg.rho_pathway(2);
sd_w = sd_facies(2) * ones(nK, 1);
fprintf('[prior] sd under independence %.4f ; correlation-aware %.4f %.4f %.4f\n'
, ...
        P.sd_iv, sd_facies(1), sd_facies(2), sd_facies(3));
fprintf('[prior] prior sd used %.4f (rho = %.2f), inflation over IV %.1f%%\n', ..
.
        sd_facies(2), P.rho_used, 100 * (sd_facies(2) / P.sd_iv - 1));

P.ne_well = ne_w; P.sd_well = sd_w; P.sd_facies = sd_facies;
P.xw = xw; P.yw = yw; P.Kw = Kw; P.fw = fw;

% ----- correlation length from the measured K --
% The spatial structure of the porosity field is inherited from the facies
% architecture, which the measured log10 K samples directly. An exponential
% variogram is fitted to the 43 log10 K values by weighted least squares.
lk = log10(Kw);
[hbin, gbin, npair] = fk_empirical_variogram(xw, yw, lk, 14);
ok = npair >= 10 & isfinite(gbin);
sill0 = var(lk); range0 = 0.3 * max(hbin);
obj = @(p) sum(npair(ok)' .* (gbin(ok)' - ...
    (abs(p(1)) + abs(p(2)) * (1 - exp(-hbin(ok)' / abs(p(3)))))'.^2);
p = fminsearch(obj, [0.1*sill0, sill0, range0], ...
    optimset('Display', 'off', 'MaxFunEvals', 5000, 'MaxIter', 5000));
P.nugget = abs(p(1)); P.sill = abs(p(2)); P.vrange = abs(p(3));
P.vario_h = hbin; P.vario_g = gbin; P.vario_n = npair;
fprintf(['[prior] exponential variogram on log10 K: nugget %.4f, ' ...
        'partial sill %.4f, range %.0f m\n'], P.nugget, P.sill, P.vrange);

% ----- kriging -----
P.ne = fk_krige(xw, yw, ne_w, G.xa, G.ya, P.nugget, P.sill, P.vrange);
P.sd = fk_krige(xw, yw, sd_w, G.xa, G.ya, P.nugget, P.sill, P.vrange);
P.sd = max(P.sd, min(sd_facies)); % kriging cannot invent a smaller sd

```

```

dv = ~isnan(D.DRASTIC);
P.V = fk_krige(D.x(dv), D.y(dv), D.DRASTIC(dv), G.xa, G.ya, ...
               P.nugget, P.sill, P.vrange);
fprintf('[prior] gridded porosity %.4f - %.4f (mean %.4f)\n', ...
        min(P.ne), max(P.ne), mean(P.ne));
fprintf('[prior] gridded DRASTIC %.1f - %.1f (mean %.1f), from %d wells\n', ...
        min(P.V), max(P.V), mean(P.V), sum(dv));

% ----- KL basis of the prior -----
%  $B(x, x') = \text{sd}(x) \text{sd}(x') \exp(-|x - x'| / \text{range})$ . Nystrom on a strided subset of
% the active cells, then extended to every active cell.
na = G.n_active;
land = 1:cfg.nystrom_stride:na;
nl = numel(land);
Xl = G.xa(land); Yl = G.ya(land); Sl = P.sd(land);
Dl1 = hypot(Xl - Xl', Yl - Yl');
Cl1 = (Sl .* Sl') .* exp(-Dl1 / P.vrange);
Cl1 = (Cl1 + Cl1') / 2;
[U, Lam] = eig(Cl1, 'vector');
[Lam, ord] = sort(Lam, 'descend');
U = U(:, ord);
m = min(cfg.n_kl, sum(Lam > 0));
Lam = Lam(1:m); U = U(:, 1:m);

% Nystrom extension:  $\phi(x) = (1/\lambda) * \sum_j B(x, x_j) u_j * (nl/na \text{ scaling})$ 
% is absorbed by renormalising each mode to unit energy in the discrete metric)
Phi = zeros(na, m);
blk = 20000;
for a = 1:blk:na
    b = min(a + blk - 1, na);
    Dx1 = hypot(G.xa(a:b) - Xl', G.ya(a:b) - Yl');
    Cx1 = (P.sd(a:b) .* Sl') .* exp(-Dx1 / P.vrange);
    Phi(a:b, :) = Cx1 * (U ./ Lam');
end
for j = 1:m % scale so that var captured = Lam
    nrm = sqrt(Phi(:, j)' * Phi(:, j));
    if nrm > 0, Phi(:, j) = Phi(:, j) / nrm; end
end
P.Phi = Phi;
P.lam = Lam * (na / nl); % discrete-metric rescaling
P.kl_var_captured = sum(P.lam) / sum(P.sd.^2);
fprintf('[prior] KL basis: %d modes, %.1f%% of the prior variance\n', ...
        m, 100 * P.kl_var_captured);
end

% =====
function [hb, gb, nb] = fk_empirical_variogram(x, y, z, nbin)
n = numel(x);
[I, J] = find(triu(ones(n), 1));
h = hypot(x(I) - x(J), y(I) - y(J));
g = 0.5 * (z(I) - z(J)).^2;
edges = linspace(0, max(h) * 0.6, nbin + 1);

```

```

hb = zeros(nbin, 1); gb = nan(nbin, 1); nb = zeros(nbin, 1);
for k = 1:nbin
    s = h >= edges(k) & h < edges(k + 1);
    nb(k) = sum(s);
    if nb(k) > 0
        hb(k) = mean(h(s));
        gb(k) = mean(g(s));
    else
        hb(k) = 0.5 * (edges(k) + edges(k + 1));
    end
end
end

```

```

% =====
function zg = fk_krige(x, y, z, xg, yg, nugget, sill, range)
%FK_KRIGE Ordinary kriging with an exponential variogram.
n = numel(x);
Dd = hypot(x - x', y - y');
Gm = nugget + sill * (1 - exp(-Dd / range));
Gm(1:n+1:end) = 0;
A = [Gm, ones(n, 1); ones(1, n), 0];
A = A + 1e-10 * eye(n + 1);
zg = zeros(numel(xg), 1);
blk = 5000;
for a = 1:blk:numel(xg)
    b = min(a + blk - 1, numel(xg));
    Dg = hypot(xg(a:b) - x', yg(a:b) - y');
    Gg = nugget + sill * (1 - exp(-Dg / range));
    rhs = [Gg'; ones(1, b - a + 1)];
    w = A \ rhs;
    zg(a:b) = (w(1:n, :))' * z;
end
end

```

A.6 fk_forward.m

```

function F = fk_forward(cfg, G, P)
%FK_FORWARD Transport operator for the Fushe-Kuqe nitrate field.
%
% Physical setting. The aquifer is confined and multilayer, recharged from
% the Mat and Droja rivers and discharging to the Adriatic, with regional flow
% from northeast to southwest (Cenameri & Beqiraj 2016). At the published
% mean seepage velocity the traverse of the aquifer takes of order half a
% century, which is the same order as the history of intensive agricultural
% nitrogen loading in the catchment. The nitrate field is therefore treated
% as transient rather than steady: it is the response to an areal loading that
% switched on at an unknown time T0 before the sampling campaign.
%
% This matters for the whole paper. At steady state with a uniform Darcy
% flux the concentration field is almost independent of effective porosity,
% because  $n_e$  cancels between the seepage velocity and the dispersion
% coefficient. In the transient problem  $n_e$  survives in the storage term and
% sets how far the front has travelled, so a concentration measurement does

```

```

% carry information about n_e. The depth-averaged equation solved here is
%
%      n_e dC/dt + q0 dC/dxi = d/dxi (alphaL q0 Gam dC/dxi)
%                               + d/deta (alphaT q0 Gam dC/deta) + m_s(x)
%
% with q0 the uniform regional Darcy flux, Gam the DRASTIC amplification of
% dispersion published in Zeqiraj et al. (2026), and m_s the areal loading.
%
% F is a struct of everything that does not depend on n_e, plus function
% handles that build the n_e-dependent operator and run the time stepping.

na = G.n_active;

% ----- uniform regional flux ----
% The workbook records depth to water, not head, and carries no DEM, so the
% hydraulic gradient is not measurable from it. It is derived from the
% published mean seepage velocity and the measured mean K.
Kmean = mean(P.Kw);
F.i_gradient = cfg.v_mean_pub * P.ne_network_mean / Kmean;
F.q0 = Kmean * F.i_gradient; % m/day, uniform Darcy flux
fprintf(['[fwd] mean measured K %.1f m/day, published mean seepage velocity ' ...
        '%.1f m/day\n'], Kmean, cfg.v_mean_pub);
fprintf(['[fwd] implied regional gradient %.3e, uniform Darcy flux %.4f m/day\n',
        ...
        F.i_gradient, F.q0]);

% ----- DRASTIC dispersion amplifier --
Vn = (P.V - min(P.V)) / (max(P.V) - min(P.V));
F.Gam = 1 + cfg.alpha_V * Vn .^ cfg.gamma_V;
F.Vn = Vn;
fprintf(['[fwd] DRASTIC dispersion amplifier Gamma in [%.2f, %.2f]\n', ...
        min(F.Gam), max(F.Gam)]);

% ----- face connectivity table ---
% Neighbour in +xi (downstream, next column) and +eta (across flow, next row).
idx = G.idx;
ny = G.ny; nx = G.nx;
own = (1:na)';
[ia, ja] = deal(G.ia, G.ja);

nb_xi = zeros(na, 1);
ok = ja < nx;
lin = sub2ind([ny nx], ia(ok), ja(ok) + 1);
nb_xi(ok) = idx(lin);

nb_eta = zeros(na, 1);
ok2 = ia < ny;
lin2 = sub2ind([ny nx], ia(ok2) + 1, ja(ok2));
nb_eta(ok2) = idx(lin2);

F.own = own; F.nb_xi = nb_xi; F.nb_eta = nb_eta;
F.n_face_xi = nnz(nb_xi > 0);

```

```

F.n_face_eta = nnz(nb_eta > 0);
fprintf('[fwd] interior faces: %d along flow, %d across flow\n', ...
        F.n_face_xi, F.n_face_eta);

% ----- dispersion at faces --
aL = cfg.alphaL; aT = cfg.alphaT_ratio * cfg.alphaL;
% n_e * D = alpha * q0 * Gamma + n_e * Dm ; the molecular term is six orders
% of magnitude smaller than the mechanical one here and is retained only for
% completeness.
gam_f_xi = 0.5 * (F.Gam(own(nb_xi > 0)) + F.Gam(nb_xi(nb_xi > 0)));
gam_f_eta = 0.5 * (F.Gam(own(nb_eta > 0)) + F.Gam(nb_eta(nb_eta > 0)));
F.TL = aL * F.q0 * gam_f_xi * cfg.dy / cfg.dx; % transmissibility, m^2/day
F.TT = aT * F.q0 * gam_f_eta * cfg.dx / cfg.dy;
F.aL = aL; F.aT = aT;

% advective transmissibility across a downstream face (upwind, flow in +xi)
F.Aadv = F.q0 * cfg.dy; % m^2/day

% ----- source shape ---
F.Vratio = P.V / mean(P.V);

% ----- observations ----
F.cell_volume = cfg.dx * cfg.dy; % per unit thickness

% ----- operator builder --
F.build = @(ne) local_build(cfg, G, F, ne);
F.march = @(A, Mass, s, T0, nsteps) local_march(A, Mass, s, T0, nsteps);
end

% =====
function [A, Mass] = local_build(cfg, G, F, ne)
%LOCAL_BUILD Sparse operator A such that Mass dC/dt = -A C + s.
%
% A collects the upwind advective flux and the two dispersive fluxes. Rows
% are cells; the sign convention is that A C is the net outflow of mass.

na = G.n_active;
I = []; J = []; S = [];

% --- dispersion along the flow (xi) ---
k = find(F.nb_xi > 0);
o = F.own(k); n = F.nb_xi(k); T = F.TL;
I = [I; o; o; n; n];
J = [J; o; n; n; o];
S = [S; T; -T; T; -T];

% --- dispersion across the flow (eta) ---
k = find(F.nb_eta > 0);
o = F.own(k); n = F.nb_eta(k); T = F.TT;
I = [I; o; o; n; n];
J = [J; o; n; n; o];

```

```

S = [S; T; -T; T; -T];

% --- upwind advection, flow strictly in +xi ---
% Interior face: mass leaves the upstream cell, enters the downstream one.
k = find(F.nb_xi > 0);
o = F.own(k); n = F.nb_xi(k); a = F.Aadv;
I = [I; o; n];
J = [J; o; o];
S = [S; a * ones(numel(o), 1); -a * ones(numel(o), 1)];

% --- outflow faces: cells with no downstream neighbour lose mass freely ---
k = find(F.nb_xi == 0);
I = [I; F.own(k)];
J = [J; F.own(k)];
S = [S; a * ones(numel(k), 1)];

% Inflow faces (no upstream neighbour) carry C = 0, so they add nothing.

A = sparse(I, J, S, na, na);
Mass = spdiags(ne * F.cell_volume, 0, na, na);
end

% =====
function C = local_march(A, Mass, s, T0, nsteps)
%LOCAL_MARCH Backward-Euler march from C = 0 to time T0.
dt = T0 / nsteps;
Lhs = decomposition(Mass / dt + A, 'lu');
C = zeros(size(A, 1), 1);
Mdt = Mass / dt;
for it = 1:nsteps
    C = Lhs \ (Mdt * C + s);
end
end

```

A.7 `fk_calibrate.m`

```

function Cal = fk_calibrate(cfg, D, G, P, F)
%FK_CALIBRATE Estimate the nitrate loading field and its onset time.
%
% A first attempt used a single loading amplitude with the spatial shape of
% the DRASTIC index. It failed: the best fit over the eight exponents in
% cfg.cal_kappas reached R2 = 0.04 and a negative leave-one-out R2, and the
% onset time collapsed onto the shortest step in the scan. The reason is
% visible in the data. Nitrate in this aquifer is organised north to south
% (correlation with northing -0.57 across the 31 instrumented wells) far more
% strongly than it is organised by vulnerability (correlation with the
% DRASTIC index +0.30). The pattern is set by where the nitrogen enters, not
% by how the aquifer transports it, so a loading field whose shape is fixed a
% priori cannot reproduce it.
%
% The loading is therefore expanded in a small set of large-scale spatial
% functions and its coefficients estimated from the observations:
%

```

```

%     m_s(x) = sum_j beta_j psi_j(x),
%     psi = { 1, xi, eta, xi^2, eta^2, xi*eta, V }
%
% with xi and eta the along- and across-flow coordinates normalised to
% [-1, 1] and V the normalised DRASTIC index. The transport equation is
% linear in the source, so the response to each psi_j is computed once and
% the coefficients follow from linear least squares at every candidate onset
% time. The DRASTIC coefficient is then a result rather than an assumption:
% it answers whether vulnerability explains any of the loading once the
% regional trend is removed.

rng(cfg.seed);

% ----- observation locations ---
ob = D.inside & D.has_NO3;
Cal.obs_id = D.well_id(ob);
xo = D.x(ob);  yo = D.y(ob);  Cal.y_obs = D.NO3(ob);
Cal.n_obs = numel(Cal.y_obs);
cell_of = zeros(Cal.n_obs, 1);
for k = 1:Cal.n_obs
    [~, cell_of(k)] = min(hypot(G.xa - xo(k), G.ya - yo(k)));
end
Cal.cell_of = cell_of;
Cal.x_obs = xo;  Cal.y_obs_coord = yo;
fprintf('[cal] %d observation wells inside the domain, max well-to-cell %.0f m\n'
, ...
        Cal.n_obs, max(hypot(G.xa(cell_of) - xo, G.ya(cell_of) - yo)));

% ----- the null model that failed, run and reported
% A loading whose spatial shape is fixed to a power of the DRASTIC index, with
% one amplitude and one onset time. It is kept in the pipeline because the
% paper reports that it fails; the numbers below are that failure.
[A0, M0] = F.build(P.ne);
Lhs0 = decomposition(M0 / cfg.cal_dt + A0, 'lu');
Mdt0 = M0 / cfg.cal_dt;
tg0 = (1:cfg.cal_nsteps)' * cfg.cal_dt;
Vr = P.V / mean(P.V);
sst0 = sum((Cal.y_obs - mean(Cal.y_obs)) .^ 2);
Cal.null_kappa = cfg.cal_kappas;
Cal.null_R2 = zeros(size(cfg.cal_kappas));
Cal.null_T0 = zeros(size(cfg.cal_kappas));
for kk = 1:numel(cfg.cal_kappas)
    s0 = (cfg.dx * cfg.dy) * (Vr .^ cfg.cal_kappas(kk));
    C0 = zeros(G.n_active, 1);
    H0 = zeros(cfg.cal_nsteps, Cal.n_obs);
    for it = 1:cfg.cal_nsteps
        C0 = Lhs0 \ (Mdt0 * C0 + s0);
        H0(it, :) = C0(cell_of)';
    end
    num0 = H0 * Cal.y_obs;  den0 = sum(H0 .^ 2, 2);
    sse0 = sum(Cal.y_obs .^ 2) - (num0 .^ 2) ./ max(den0, realmin);
    [b0, i0] = min(sse0);

```

```

    Cal.null_R2(kk) = 1 - b0 / sst0;
    Cal.null_T0(kk) = tg0(i0);
end
[Cal.null_best_R2, ib] = max(Cal.null_R2);
Cal.null_best_kappa = cfg.cal_kappas(ib);
Cal.null_best_T0 = Cal.null_T0(ib);
fprintf(['[cal] null model with the loading shaped as DRASTIC^kappa: best ' ...
        'R2 %.3f at kappa %.2f, T0 %.0f d\n'], Cal.null_best_R2, ...
        Cal.null_best_kappa, Cal.null_best_T0);

% ----- source basis -----
xi_a = G.xa * G.exi(1) + G.ya * G.exi(2);
eta_a = G.xa * G.eta(1) + G.ya * G.eta(2);
nz = @(v) 2 * (v - min(v)) / (max(v) - min(v)) - 1;
xs = nz(xi_a); es = nz(eta_a); vs = nz(P.V);
Psi = [ones(G.n_active, 1), xs, es, xs.^2, es.^2, xs.*es, vs];
Cal.psi_names = {'1', 'xi', 'eta', 'xi^2', 'eta^2', 'xi*eta', 'V_DRASTIC'};
nb = size(Psi, 2);

% ----- marching -----
[A, Mass] = F.build(P.ne);
dt = cfg.cal_dt; nstep = cfg.cal_nsteps;
Lhs = decomposition(Mass / dt + A, 'lu');
Mdt = Mass / dt;
tgrid = (1:nstep)' * dt;

Hobs = zeros(nstep, Cal.n_obs, nb); % response at the wells per unit beta
tS = tic;
for j = 1:nb
    s = (cfg.dx * cfg.dy) * Psi(:, j);
    C = zeros(G.n_active, 1);
    for it = 1:nstep
        C = Lhs \ (Mdt * C + s);
        Hobs(it, :, j) = C(cell_of)';
    end
end
fprintf('[cal] %d source-basis marches of %d steps in %.0f s\n', nb, nstep, toc(tS));

% ----- least squares at every onset time
y = Cal.y_obs;
sst = sum((y - mean(y)) .^ 2);
sse_t = inf(nstep, 1);
beta_t = zeros(nb, nstep);
for it = 1:nstep
    X = squeeze(Hobs(it, :, :));
    if rcond(X' * X) < 1e-14, continue; end
    b = (X' * X) \ (X' * y);
    beta_t(:, it) = b;
    sse_t(it) = sum((y - X * b) .^ 2);
end
[sse, it_best] = min(sse_t);

```

```

Cal.T0 = tgrid(it_best);
Cal.beta = beta_t(:, it_best);
Cal.rmse = sqrt(sse / Cal.n_obs);
Cal.R2 = 1 - sse / sst;
Cal.n_par = nb + 1; % the basis plus the onset time
Cal.R2_adj = 1 - (1 - Cal.R2) * (Cal.n_obs - 1) / (Cal.n_obs - Cal.n_par - 1);
Cal.sse_profile = sse_t;
Cal.tgrid = tgrid;

fprintf(['[cal] onset time T0 = %.0f d (%.1f yr), RMSE %.3f mg/L, ' ...
        'R2 %.3f, adjusted R2 %.3f\n'], Cal.T0, Cal.T0 / 365.25, ...
        Cal.rmse, Cal.R2, Cal.R2_adj);
for j = 1:nb
    fprintf(['[cal]   beta(%-9s) = %.4e\n', Cal.psi_names{j}, Cal.beta(j));
end

% ----- source field -----
Cal.source_density = Psi * Cal.beta; % mg/L/day
Cal.frac_negative = mean(Cal.source_density < 0);
fprintf(['[cal] fitted loading %.3e to %.3e mg/L/d, negative over %.1f%% of the do
main\n', ...
        min(Cal.source_density), max(Cal.source_density), ...
        100 * Cal.frac_negative);
% A loading field must not be a sink. Where least squares drives it below
% zero it is clipped and the amplitude refitted on the clipped shape.
shape = max(Cal.source_density, 0);
s = (cfg.dx * cfg.dy) * shape;
C = zeros(G.n_active, 1);
Hc = zeros(nstep, Cal.n_obs);
for it = 1:nstep
    C = Lhs \ (Mdt * C + s);
    Hc(it, :) = C(cell_of)';
end
num = Hc * y; den = sum(Hc.^2, 2);
a_t = num ./ max(den, realmin);
sse_c = sum(y.^2) - (num.^2) ./ max(den, realmin);
[sse2, it2] = min(sse_c);
Cal.T0_clipped = tgrid(it2);
Cal.amp_clipped = a_t(it2);
Cal.rmse_clipped = sqrt(sse2 / Cal.n_obs);
Cal.R2_clipped = 1 - sse2 / sst;
fprintf(['[cal] non-negative loading: T0 %.0f d (%.1f yr), RMSE %.3f mg/L, ' ...
        'R2 %.3f\n'], Cal.T0_clipped, Cal.T0_clipped / 365.25, ...
        Cal.rmse_clipped, Cal.R2_clipped);

Cal.source = (cfg.dx * cfg.dy) * Cal.amp_clipped * shape;
Cal.T0 = Cal.T0_clipped;
Cal.rmse = Cal.rmse_clipped;
Cal.R2 = Cal.R2_clipped;

C = zeros(G.n_active, 1);
for it = 1:it2

```

```

    C = Lhs \ (Mdt * C + Cal.source);
end
Cal.C = C;
Cal.y_fit = C(cell_of);
Cal.resid = y - Cal.y_fit;
fprintf('[cal] modelled nitrate %.2f - %.2f mg/L (observed %.2f - %.2f)\n', ...
        min(C), max(C), min(y), max(y));

% ----- what DRASTIC alone explains
% Same fit with the vulnerability column removed, to isolate its contribution.
keep = 1:(nb - 1);
sse_t2 = inf(nstep, 1);
for it = 1:nstep
    X = squeeze(Hobs(it, :, keep));
    if rcond(X' * X) < 1e-14, continue; end
    b = (X' * X) \ (X' * y);
    sse_t2(it) = sum((y - X * b) .^ 2);
end
Cal.R2_noV = 1 - min(sse_t2) / sst;
fprintf(['[cal] dropping the DRASTIC column changes R2 from %.3f to %.3f ' ...
        '(partial contribution %.3f)\n'], ...
        1 - sse / sst, Cal.R2_noV, (1 - sse / sst) - Cal.R2_noV);

% ----- L00-CV -----
pred = zeros(Cal.n_obs, 1);
for k = 1:Cal.n_obs
    kp = true(Cal.n_obs, 1); kp(k) = false;
    best = inf; bp = 0;
    for it = 1:nstep
        X = squeeze(Hobs(it, kp, :));
        if rcond(X' * X) < 1e-14, continue; end
        b = (X' * X) \ (X' * y(kp));
        e = sum((y(kp) - X * b) .^ 2);
        if e < best
            best = e;
            bp = squeeze(Hobs(it, k, :))' * b;
        end
    end
    pred(k) = bp;
end
Cal.loo_pred = pred;
Cal.loo_rmse = sqrt(mean((y - pred) .^ 2));
Cal.loo_R2 = 1 - sum((y - pred) .^ 2) / sst;
fprintf('[cal] leave-one-out: RMSE %.3f mg/L, R2 %.3f\n', ...
        Cal.loo_rmse, Cal.loo_R2);
end

```

A.8 fk_sensitivity.m

```

function S = fk_sensitivity(cfg, G, P, F, Cal)
%FK_SENSITIVITY Jacobian of the nitrate field with respect to the porosity
%                  field, expressed in the Karhunen-Loeve basis of the prior.
%

```

```

% The porosity field is written  $n_e(x) = n_{\text{prior}}(x) + \sum_k \theta_k \phi_k(x)$ 
% with  $\theta_k \sim N(0, \text{diag}(\lambda_k))$ . Differentiating the discrete transport
% equation with respect to  $\theta_k$  gives
%
%  $(\text{Mass}/dt + A) C_k^n = (\text{Mass}/dt) C_k^{(n-1)}$ 
%  $- \text{diag}(\phi_k * \text{cellVolume}) (C^n - C^{(n-1)}) / dt$ 
%
% which is the same left-hand side as the base march. One sparse
% factorisation therefore serves the base solve and all KL modes, and the
% derivative is exact for the discrete system rather than a finite
% difference. A finite-difference check on a random mode is run and
% reported.

na = G.n_active;
m = numel(P.lam);
dt = cfg.cal_dt;
nstep = round(Cal.T0 / dt);
fprintf('[sens] %d KL modes, %d time steps to T0 = %.0f d\n', m, nstep, Cal.T0);

[A, Mass] = F.build(P.ne);
Lhs = decomposition(Mass / dt + A, 'lu');
Mdt = Mass / dt;
s = Cal.source;

% ----- base march ---
Cbase = zeros(na, nstep + 1);
C = zeros(na, 1);
for it = 1:nstep
    C = Lhs \ (Mdt * C + s);
    Cbase(:, it + 1) = C;
end
S.C = C;
fprintf('[sens] base field at T0: %.3f - %.3f mg/L\n', min(C), max(C));

% ----- mode sensitivity --
vol = F.cell_volume;
J = zeros(na, m);
tS = tic;
for k = 1:m
    wk = P.Phi(:, k) * vol / dt; % diag(phi_k * V) / dt
    Ck = zeros(na, 1);
    for it = 1:nstep
        rhs = Mdt * Ck - wk .* (Cbase(:, it + 1) - Cbase(:, it));
        Ck = Lhs \ rhs;
    end
    J(:, k) = Ck;
    if mod(k, 25) == 0
        fprintf('[sens] mode %d/%d, %.0f s elapsed\n', k, m, toc(tS));
    end
end
S.J = J;
fprintf('[sens] Jacobian built in %.0f s\n', toc(tS));

```

```

% ----- finite-difference check
% Run at three step sizes. The derivative is exact for the discrete system, so
% the residual should fall linearly with the step, which is what distinguishes
% a correct derivative from a coincidence at one step.
kchk = min(3, m);
S.fd_steps = [0.25 0.05 0.01] * sqrt(P.lam(kchk));
S.fd_rel = zeros(size(S.fd_steps));
ad = J(:, kchk);
for q = 1:numel(S.fd_steps)
    eps_k = S.fd_steps(q);
    [A2, Mass2] = F.build(P.ne + eps_k * P.Phi(:, kchk));
    Lhs2 = decomposition(Mass2 / dt + A2, 'lu');
    Mdt2 = Mass2 / dt;
    C2 = zeros(na, 1);
    for it = 1:nstep
        C2 = Lhs2 \ (Mdt2 * C2 + s);
    end
    fd = (C2 - S.C) / eps_k;
    S.fd_rel(q) = norm(fd - ad) / max(norm(ad), realmin);
    fprintf(['[sens] finite-difference check on mode %d, step %.3e: ' ...
            'relative difference %.3e\n'], kchk, eps_k, S.fd_rel(q));
end
S.fd_check_rel = S.fd_rel(end);
S.fd_slope = log(S.fd_rel(1) / S.fd_rel(end)) / log(S.fd_steps(1) / S.fd_steps(end));
fprintf(['[sens] residual falls with step as a power of %.2f (first order is 1)\n'
        , ...
        S.fd_slope]);

% ----- Fisher information -----
S.Iprior = diag(1 ./ P.lam);
Jo = J(Cal.cell_of, :);
S.Jobs = Jo;
S.Iobs = (Jo' * Jo) / cfg.sigma_obs^2;
S.Itot = S.Iprior + S.Iobs;
S.Sigma = inv(S.Itot);
S.Sigma = (S.Sigma + S.Sigma') / 2;

S.tr_prior = trace(S.Iprior);
S.tr_obs = trace(S.Iobs);
S.frac_prior = S.tr_prior / (S.tr_prior + S.tr_obs);
S.frac_obs = 1 - S.frac_prior;

% Degrees of freedom for signal: how many directions of the parameter space
% the observations actually constrain. This is the measure that should be
% quoted. The trace ratio above is reported as well because earlier work on
% this aquifer uses it, but it is biased: the trace of the prior information
% is dominated by the smallest prior variances, that is by the directions the
% prior already pins down, so it flatters the prior by construction. The
% degrees of freedom are scale free and bounded by the number of modes.
S.dofs = trace(S.Iobs * S.Sigma);

```

```

S.n_modes = m;
fprintf(['[sens] degrees of freedom for signal %.4f out of %d modes, ' ...
        'so the observations constrain the equivalent of %.4f directions\n'], ..
.
        S.dofs, m, S.dofs);
S.logdet_prior = sum(log(1 ./ P.lam));
S.logdet_tot = 2 * sum(log(diag(chol(S.Itot))));
S.info_gain_nats = 0.5 * (S.logdet_tot - S.logdet_prior);
fprintf(['[sens] Fisher trace: prior %.3e (%.1f%%), observations %.3e ' ...
        '(%1f%%)\n'], S.tr_prior, 100*S.frac_prior, S.tr_obs, 100*S.frac_obs);
fprintf(['[sens] log|I_total| - log|I_prior| = %.3f, so the 24 nitrate wells ' ..
.
        'carry %.3f nats\n'], S.logdet_tot - S.logdet_prior, S.info_gain_nats);

% Posterior sd of the porosity field, back in space. The posterior covariance
% is not diagonal in the KL basis, so the pointwise variance is the diagonal of
% Phi * Sigma * Phi', computed row by row rather than from diag(Sigma) alone.
S.var_post = sum((P.Phi * S.Sigma) .* P.Phi, 2);
S.sd_post = sqrt(max(S.var_post, 0));
S.var_prior_field = sum((P.Phi.^2) .* (P.lam'), 2);
S.sd_prior_field = sqrt(max(S.var_prior_field, 0));
red = 1 - mean(S.sd_post) / mean(S.sd_prior_field);
fprintf(['[sens] mean prior sd in the KL subspace %.5f -> posterior %.5f ' ...
        '(%1f%% reduction)\n'], mean(S.sd_prior_field), mean(S.sd_post), ...
        100 * red);
S.sd_reduction = red;
end

```

A.9 fk_design.m

```

function Des = fk_design(cfg, D, G, P, F, Cal, S)
%FK_DESIGN Classical D-optimal and goal-oriented monitoring network design.
%
% Two criteria are compared.
%
% (1) Classical D-optimality. Adding one concentration observation at x
%     updates the Fisher information by the rank-one term  $\sigma^{-2} J(x)'J(x)$ ,
%     and by the matrix determinant lemma
%          $\log \det I+ - \log \det I = \log(1 + \sigma^{-2} J(x) \Sigma J(x)')$ .
%     This is the objective maximised by the classical criterion, and it is
%     monotone submodular in the chosen set, so the greedy sequence carries
%     the standard  $(1 - 1/e)$  guarantee (Krause et al. 2008).
%
% (2) Goal-oriented design for the vulnerability-weighted arrival time.
%     Protection-zone delineation does not depend on the porosity field as
%     such; it depends on how long a contaminant takes to reach the discharge
%     boundary, and it depends on that most where the aquifer is vulnerable.
%     With a uniform Darcy flux the residual travel time from x is
%      $\tau(x) = L(x) n_e(x) / q_0$ , where  $L(x)$  is the remaining distance along
%     the flow direction, so a porosity error maps linearly onto an arrival
%     time error. Define the vulnerability-weighted mean arrival-time error
%          $g(\theta) = \sum_x w(x) L(x) dn_e(x) / (q_0 \sum_x w(x))$ ,
%          $w(x) = (V(x)/\text{mean } V)^\gamma$ ,

```

```

%      a linear functional  $g = c' \theta$ . The design minimises  $\text{Var}(g) = c' \Sigma c$ ,
%      and one observation at  $x$  reduces it by
%       $\Delta_g(x) = \sigma^{-2} (c' \Sigma J(x))^{-2} / (1 + \sigma^{-2} J(x) \Sigma J(x)')$ .
%
% Note on the guarantee. Submodularity is a property of the log-determinant
% objective. It is not claimed for the goal-oriented variance reduction,
% where the greedy sequence is used as a heuristic and reported as such.

na = G.n_active;
sig2 = cfg.sigma_obs^2;

% ----- candidate set: every active cell ----
% Sites already occupied by a nitrate well are excluded.
cand = true(na, 1);
cand(Cal.cell_of) = false;
Des.n_candidates = nnz(cand);

% ----- goal functional coefficients --
% Remaining distance to the downstream boundary along the flow direction.
xi_a = G.xa * G.exi(1) + G.ya * G.exi(2);
Lrem = max(xi_a) - xi_a;
w = (P.V / mean(P.V)) .^ cfg.gamma_Vdesign;
cw = w .* Lrem;
c_goal = (P.Phi' * cw) / (F.q0 * sum(w)); % days per unit theta
Des.c_goal = c_goal;
Des.Lrem = Lrem;
Des.w = w;
Des.tau_mean_prior = sum(w .* Lrem .* P.ne) / (F.q0 * sum(w));
fprintf(['[design] vulnerability-weighted mean residual travel time ' ...
        '%.0f d (%.1f yr)\n'], Des.tau_mean_prior, Des.tau_mean_prior/365.25);

% ----- information horizon -----
% Computed before the placement, because it is what the minimum separation
% should be set from. The kernel through which one observation sees the field
% is  $\kappa_x(x') = \sum_k \lambda_k J(x, k) \phi_k(x')$ , the prior covariance
% between the concentration at  $x$  and the porosity elsewhere; the horizon is the
% radius holding a given fraction of its absolute mass. Reference sites are
% the single best one-shot gain and the first three existing nitrate wells.
one_shot = sum((S.J * S.Sigma) .* S.J, 2);
one_shot(~cand) = -inf;
[~, best_single] = max(one_shot);
Des.horizon_sites = [best_single; Cal.cell_of(1:3)];
Des.horizon = zeros(numel(Des.horizon_sites), 1);
for t = 1:numel(Des.horizon_sites)
    ci = Des.horizon_sites(t);
    a = abs(P.Phi * (P.lam .* S.J(ci, :))');
    rr = hypot(G.xa - G.xa(ci), G.ya - G.ya(ci));
    [rs, ord] = sort(rr);
    cum = cumsum(a(ord)) / sum(a);
    Des.horizon(t) = rs(find(cum >= cfg.horizon_frac, 1));
end

```

```

Des.horizon_median = median(Des.horizon);
fprintf(['[design] information horizon at %.0f%% of kernel mass: %.0f - %.0f m '
...
      '(median %.0f m), against a variogram range of %.0f m\n'], ...
      100*cfg.horizon_frac, min(Des.horizon), max(Des.horizon), ...
      Des.horizon_median, P.vrange);

% ----- greedy placement ---
% Unconstrained greedy first, then the same search under two minimum
% separations. The unconstrained version is kept because what it does is a
% result in its own right: on a 100 m grid whose information field varies over
% kilometres, the rank-one update removes so little of the information at the
% chosen cell that the next choice is its neighbour, and the sequence collapses
% into a cluster of mutually redundant wells. The two constraints test which
% length scale has to be forbidden, the variogram range or the information
% horizon.
Des.gammaV = cfg.gamma_Vdesign;
Des.min_sep = P.vrange;
Des.min_sep_horizon = Des.horizon_median;
Des.D_hor = local_greedy(S.Sigma, S.J, c_goal, cand, sig2, cfg.n_new_wells, ...
      'D', Des.min_sep_horizon, G);
Des.G_hor = local_greedy(S.Sigma, S.J, c_goal, cand, sig2, cfg.n_new_wells, ...
      'goal', Des.min_sep_horizon, G);
Des.D_free = local_greedy(S.Sigma, S.J, c_goal, cand, sig2, cfg.n_new_wells, ...
      'D', 0, G);
Des.G_free = local_greedy(S.Sigma, S.J, c_goal, cand, sig2, cfg.n_new_wells, ...
      'goal', 0, G);
Des.D = local_greedy(S.Sigma, S.J, c_goal, cand, sig2, cfg.n_new_wells, ...
      'D', Des.min_sep, G);
Des.G = local_greedy(S.Sigma, S.J, c_goal, cand, sig2, cfg.n_new_wells, ...
      'goal', Des.min_sep, G);

Des.free_span_m = max(hypot(G.xa(Des.D_free.cells) - G.xa(Des.D_free.cells)'), ...
      G.ya(Des.D_free.cells) - G.ya(Des.D_free.cells)'), ..
.
      [], 'all');
fprintf(['[design] unconstrained greedy D-optimal spreads its %d wells over ' ...
      '%.0f m; minimum separation set to %.0f m\n'], ...
      cfg.n_new_wells, Des.free_span_m, Des.min_sep);

for nm = {'D', 'G'}
    r = Des.(nm{1});
    fprintf('[design] %s-optimal sequence:\n', nm{1});
    for j = 1:numel(r.cells)
        ci = r.cells(j);
        fprintf(['    %d: x %.0f y %.0f DRASTIC %.0f ' ...
            'logdet gain %.4f Var(g) %.4g -> sd(g) %.1f d\n'], ...
            j, G.xa(ci), G.ya(ci), P.V(ci), r.logdet_gain(j), ...
            r.var_goal(j + 1), sqrt(r.var_goal(j + 1)));
    end
end
end

```

```

Des.var_goal_prior = c_goal' * S.Sigma * c_goal;
Des.sd_goal_prior = sqrt(Des.var_goal_prior);
fprintf(['[design] sd of the vulnerability-weighted arrival time: ' ...
        '%.1f d before augmentation\n'], Des.sd_goal_prior);
fprintf(['[design] after 5 wells: D-optimal %.1f d, goal-oriented %.1f d ' ...
        '(%%.1f%% better)\n'], sqrt(Des.D.var_goal(end)), ...
        sqrt(Des.G.var_goal(end)), ...
        100 * (1 - sqrt(Des.G.var_goal(end)) / sqrt(Des.D.var_goal(end))));

% separation between the two designs
dd = zeros(cfg.n_new_wells, 1);
for j = 1:cfg.n_new_wells
    dd(j) = min(hypot(G.xa(Des.G.cells(j)) - G.xa(Des.D.cells), ...
                    G.ya(Des.G.cells(j)) - G.ya(Des.D.cells)));
end
Des.separation_m = dd;
Des.V_D = P.V(Des.D.cells);
Des.V_G = P.V(Des.G.cells);
fprintf(['[design] mean DRASTIC at the chosen sites: D-optimal %.1f, ' ...
        'goal-oriented %.1f (domain mean %.1f)\n'], ...
        mean(Des.V_D), mean(Des.V_G), mean(P.V));
fprintf('[design] median distance between the two designs %.0f m\n', median(dd));

% ----- information exclusion diagnostic --
% Correlation of the information carried by two observations. A greedy
% sequence that keeps this small is placing non-redundant wells.
nn = cfg.n_new_wells;
function Rm = infocorr(cells)
    Rm = eye(nn);
    for a = 1:nn
        for b = a+1:nn
            Ja = S.J(cells(a), :); Jb = S.J(cells(b), :);
            Rm(a, b) = (Ja * S.Sigma * Jb') / ...
                sqrt((Ja * S.Sigma * Ja') * (Jb * S.Sigma * Jb'));
            Rm(b, a) = Rm(a, b);
        end
    end
end
Des.info_corr = infocorr(Des.G.cells);
Des.info_corr_free = infocorr(Des.G_free.cells);
Des.info_corr_hor = infocorr(Des.G_hor.cells);
mx = @(Rm) max(abs(Rm(~eye(nn))));
Des.info_corr_max = mx(Des.info_corr);
Des.info_corr_free_max = mx(Des.info_corr_free);
Des.info_corr_hor_max = mx(Des.info_corr_hor);
fprintf(['[design] maximum pairwise information correlation: unconstrained ' ...
        '%.3f, at one variogram range %.3f, at one information horizon %.3f\n'],
    ...
        Des.info_corr_free_max, Des.info_corr_max, Des.info_corr_hor_max);

% ----- concentration versus pump test -
% A pumping test constrains the porosity at its own cell through the

```

```

% published log10 K to porosity relation used for the prior mean. Its Fisher
% contribution is rank one in the KL basis with variance sigma_nK^2.
sd_lK = std(log10(P.Kw));
sd_nK = P.ne_network_sd * (cfg.sigma_K_rel / log(10)) / sd_lK;
Des.sd_pump_ne = sd_nK;
gain_pump = zeros(na, 1);
gain_conc = zeros(na, 1);
SigC = S.Sigma * c_goal;
for a = 1:na
    Ja = S.J(a, :);
    q = Ja * SigC;
    d = 1 + (Ja * S.Sigma * Ja') / sig2;
    gain_conc(a) = (q^2 / sig2) / d;
    pa = P.Phi(a, :);
    qp = pa * SigC;
    dp = 1 + (pa * S.Sigma * pa') / sd_nK^2;
    gain_pump(a) = (qp^2 / sd_nK^2) / dp;
end
Des.gain_conc = gain_conc;
Des.gain_pump = gain_pump;
Des.gain_conc_best = max(gain_conc(cand));
Des.gain_pump_best = max(gain_pump(cand));
Des.horizon_frac = cfg.horizon_frac;
Des.R_equiv = Des.gain_conc_best / Des.gain_pump_best;
fprintf(['[design] best single observation reduces Var(g) by %.4g d^2 ' ...
        '(concentration) vs %.4g d^2 (pumping test)\n'], ...
        max(gain_conc(cand)), max(gain_pump(cand)));
fprintf(['[design] information equivalence factor R = %.3f ' ...
        '(concentration observations per pumping test)\n'], Des.R_equiv);
end

% =====
function r = local_greedy(Sigma, J, c, cand, sig2, nsel, mode, min_sep, G)
sel = [];
logdet_gain = zeros(nsel, 1);
var_goal = zeros(nsel + 1, 1);
var_goal(1) = c' * Sigma * c;
Sg = Sigma;
for j = 1:nsel
    SgC = Sg * c;
    Jc = J * Sg; % na x m
    quad = sum(Jc .* J, 2); % J Sg J'
    switch mode
        case 'D'
            score = log(1 + quad / sig2);
        case 'goal'
            num = (J * SgC) .^ 2 / sig2;
            score = num ./ (1 + quad / sig2);
    end
    score(~cand) = -inf;
    [~, best] = max(score);
    sel(end + 1, 1) = best; %#ok<AGROW>
end

```

```

cand(best) = false;
if min_sep > 0
    cand(hypot(G.xa - G.xa(best), G.ya - G.ya(best)) < min_sep) = false;
end
Jb = J(best, :);
u = Sg * Jb';
denom = sig2 + Jb * u;
logdet_gain(j) = log(1 + (Jb * u) / sig2);
Sg = Sg - (u * u') / denom;
Sg = (Sg + Sg') / 2;
var_goal(j + 1) = c' * Sg * c;
end
r.cells = sel;
r.logdet_gain = logdet_gain;
r.var_goal = var_goal;
r.Sigma_after = Sg;
end

```

A.10 oed_local.m

```

function R = oed_local(Phi, lam, w, sd_loc, cand, nsel, xa, ya, min_sep)
if nargin < 9, min_sep = 0; end
%OED_LOCAL D-optimal and goal-oriented placement for a local observation.
%
% A characterisation point placed at x measures the field there directly, so
% its Jacobian row in the KL basis is simply phi(x) and the two criteria take
% the same rank-one form as for a transport observation:
%
%      D-optimal    log(1 + phi Sigma phi' / sd^2)
%      goal         (c' Sigma phi')^2 / sd^2 / (1 + phi Sigma phi' / sd^2)
%
% with c the coefficient vector of the vulnerability-weighted mean of the
% field, g = sum_x w(x) dn(x) / sum_x w(x).
%
% Using the same operator at both sites is what makes the two comparable:
% it removes the transport model, and with it every site-specific assumption
% about flow, from the comparison.

Sigma0 = diag(lam);
c = (Phi' * w) / sum(w);
R.c = c;
R.var_prior = c' * Sigma0 * c;

for mode = {'D', 'goal'}
    md = mode{1};
    Sg = Sigma0;
    cc = cand;
    sel = zeros(nsel, 1);
    vg = zeros(nsel + 1, 1); vg(1) = R.var_prior;
    ld = zeros(nsel, 1);
    for j = 1:nsel
        SgC = Sg * c;
        PS = Phi * Sg;
    end
end

```

```

quad = sum(PS .* Phi, 2);
switch md
    case 'D'
        score = log(1 + quad / sd_loc^2);
    case 'goal'
        score = ((Phi * SgC) .^ 2 / sd_loc^2) ./ (1 + quad / sd_loc^2);
end
score(~cc) = -inf;
[~, b] = max(score);
sel(j) = b; cc(b) = false;
if min_sep > 0
    cc(hypot(xa - xa(b), ya - ya(b)) < min_sep) = false;
end
pb = Phi(b, :);
u = Sg * pb';
ld(j) = log(1 + (pb * u) / sd_loc^2);
Sg = Sg - (u * u') / (sd_loc^2 + pb * u);
Sg = (Sg + Sg') / 2;
vg(j + 1) = c' * Sg * c;
end
R.(md).cells = sel;
R.(md).var_goal = vg;
R.(md).logdet_gain = ld;
end

% how far apart the two designs are, and how they differ in vulnerability
d = zeros(nsel, 1);
for j = 1:nsel
    d(j) = min(hypot(xa(R.goal.cells(j)) - xa(R.D.cells), ...
        ya(R.goal.cells(j)) - ya(R.D.cells)));
end
R.separation_m = d;
R.median_separation_m = median(d);
R.var_reduction_D = 1 - R.D.var_goal(end) / R.var_prior;
R.var_reduction_goal = 1 - R.goal.var_goal(end) / R.var_prior;
R.goal_advantage = (R.D.var_goal(end) - R.goal.var_goal(end)) / R.D.var_goal(end);
end

```

A.11 fk_particles.m

```

function Par = fk_particles(cfg, G, P, F, Cal, S)
%FK_PARTICLES Posterior characterisation by implicit sampling, against SIR.
%
% The map from the porosity coefficients theta to the nitrate field is not
% linear: theta enters through the storage term, so it changes how far the
% front has travelled, and the observation operator is a nonlinear function
% of it. The posterior is therefore not Gaussian and a sampler is needed.
%
% Cost function
%     F(theta) = 0.5 theta' diag(1/lambda) theta
%               + 0.5 || y - h(theta) ||^2 / sigma_obs^2
%

```

```
% Implicit sampling (Chorin & Tu 2009; Chorin, Morzfeld & Tu 2010; Morzfeld
% et al. 2012) minimises F, builds the quadratic F0 around the minimiser from
% the Gauss-Newton Hessian, maps standard normal draws through the Cholesky
% factor and corrects with the weight exp(-(F - F0)). The comparison is with
% sequential importance resampling drawing from the prior.
%
% Two things are measured and reported: the effective sample size of each
% sampler, and the number of iterations the minimisation takes under the
% natural gradient of Amari (1998) versus the plain Euclidean gradient.
```

```
rng(cfg.seed);
m = numel(P.lam);
y = Cal.y_obs;
sig2 = cfg.sigma_obs ^ 2;
dt = cfg.cal_dt;
nstep = round(Cal.T0 / dt);
```

```
% forward evaluation at the observation wells for a given theta
```

```
function c = hfun(th)
    ne = P.ne + P.Phi * th;
    ne = max(ne, 0.05);
    [A, Mass] = F.build(ne);
    Lh = decomposition(Mass / dt + A, 'lu');
    Md = Mass / dt;
    C = zeros(G.n_active, 1);
    for it = 1:nstep
        C = Lh \ (Md * C + Cal.source);
    end
    c = C(Cal.cell_of);
end
function v = Ffun(th)
    r = y - hfun(th);
    v = 0.5 * sum((th .^ 2) ./ P.lam) + 0.5 * sum(r .^ 2) / sig2;
end
```

```
nfwd = 0;
```

```
% ----- Gauss-Newton MAP -----
```

```
% The Jacobian is held at its prior-state value, which is the Gauss-Newton
% approximation used throughout; the residual is recomputed each step.
```

```
th = zeros(m, 1);
Hgn = S.Itot;
Rch = chol(Hgn, 'lower');
Par.F_path = zeros(0, 1);
for k = 1:cfg.gn_iters
    r = y - hfun(th); nfwd = nfwd + 1;
    g = th ./ P.lam - (S.Jobs' * r) / sig2;
    dth = -(Rch' \ (Rch \ g));
    % backtracking on the true cost
    a = 1; f0 = Ffun(th); nfwd = nfwd + 1;
    for bt = 1:8
        if Ffun(th + a * dth) < f0, break; end
```

```

        nfwd = nfwd + 1;
        a = a / 2;
    end
    th = th + a * dth;
    Par.F_path(end + 1, 1) = Ffun(th); nfwd = nfwd + 1;
    if norm(dth) < 1e-10 * norm(th) + 1e-12, break; end
end
Par.theta_map = th;
Par.phi_F = Ffun(th); nfwd = nfwd + 1;
Par.gn_iters_used = numel(Par.F_path);
fprintf('[part] Gauss-Newton MAP reached in %d iterations, F = %.4f\n', ...
        Par.gn_iters_used, Par.phi_F);

% ----- natural gradient versus Euclidean -----
% Both descend the same quadratic model of F around the prior state, so the
% comparison isolates the effect of the metric. The Euclidean step uses the
% largest stable constant step, 1/L with L the largest eigenvalue.
Hs = (S.Itot + S.Itot') / 2;
ev = eig(Hs);
Par.cond_H = max(ev) / min(ev);
g0 = -(S.Jobs' * (y - hfun(zeros(m, 1)))) / sig2; nfwd = nfwd + 1;
tol = 1e-6;
% natural gradient: preconditioned by the inverse Fisher information
xg = zeros(m, 1); it_nat = 0;
while it_nat < cfg.max_desc
    gk = Hs * xg + g0;
    if sqrt(gk' * (Hs \ gk)) < tol * sqrt(g0' * (Hs \ g0)), break; end
    xg = xg - (Hs \ gk);
    it_nat = it_nat + 1;
end
% Euclidean gradient
xe = zeros(m, 1); it_euc = 0; step = 1 / max(ev);
while it_euc < cfg.max_desc
    gk = Hs * xe + g0;
    if sqrt(gk' * (Hs \ gk)) < tol * sqrt(g0' * (Hs \ g0)), break; end
    xe = xe - step * gk;
    it_euc = it_euc + 1;
end
Par.iters_natural = it_nat;
Par.iters_euclidean = it_euc;
Par.speedup = it_euc / max(it_nat, 1);
fprintf('[part] condition number of the Gauss-Newton Hessian %.3e\n', Par.cond_H);
fprintf('[part] descent to a relative Riemannian gradient of %.0e: ' ...
        'natural %d iterations, Euclidean %d, ratio %.1f (sqrt(kappa) = %.1f)\n'
], ...
        tol, it_nat, it_euc, Par.speedup, sqrt(Par.cond_H));

% ----- implicit sampling --
L = chol(Hgn, 'lower');
M = cfg.M_particles;
Xi = randn(m, M);

```

```

W = zeros(M, 1);
Fv = zeros(M, 1); F0v = zeros(M, 1);
for k = 1:M
    xk = Par.theta_map + L' \ Xi(:, k);
    Fv(k) = Ffun(xk); nfwd = nfwd + 1;
    dx = xk - Par.theta_map;
    F0v(k) = Par.phi_F + 0.5 * (dx' * (Hgn * dx));
    W(k) = exp(-(Fv(k) - F0v(k)));
end
Par.w_implicit = W;
Par.ess_implicit = (sum(W) ^ 2) / (M * sum(W.^ 2));
fprintf('[part] implicit sampling with M = %d: normalised ESS %.3f\n', ...
        M, Par.ess_implicit);

% posterior mean and spread of the goal-relevant field
Xs = Par.theta_map + (L' \ Xi);
wn = W / sum(W);
Par.theta_mean = Xs * wn;
dev = Xs - Par.theta_mean;
Par.theta_cov_diag = (dev.^ 2) * wn;

% ----- SIR -----
Ms = cfg.M_sir;
Ws = zeros(Ms, 1);
for k = 1:Ms
    xk = sqrt(P.lam) .* randn(m, 1);
    r = y - hfun(xk); nfwd = nfwd + 1;
    Ws(k) = exp(-0.5 * sum(r.^ 2) / sig2);
end
Ws = Ws / max(Ws);
Par.ess_sir = (sum(Ws) ^ 2) / (Ms * sum(Ws.^ 2));
fprintf('[part] SIR from the prior with M = %d: normalised ESS %.3f\n', ...
        Ms, Par.ess_sir);
Par.n_forward = nfwd;
fprintf('[part] %d forward transport solves in total\n', nfwd);
end

```

A.12 mrva_replicate.m

```

function M = mrva_replicate(cfg)
%MRVA_REPLICATE Independent replication on a public USGS dataset.
%
% Data: 44 wells of the Mississippi Alluvial Plain with hydraulic
% conductivity and transmissivity from slug tests (Pugh, USGS Scientific
% Investigations Report 2023-5101, doi:10.3133/sir20235101), together with
% the DRASTIC ratings assigned to each well in the project file. The
% DRASTIC index of every well is recomputed here from its seven ratings and
% the standard weights, and the recomputation must match the stored value
% exactly or the run stops.
%
% What is replicated is the design comparison, not the transport model. No
% flow field, boundary or nitrate record is available for this site, and
% none is invented. Both sites are therefore compared with the same local

```

```

% observation operator, in which a new characterisation point measures the
% field at its own location. Every reported quantity is dimensionless or a
% distance, so nothing depends on the absolute porosity scale, which this
% dataset does not constrain.

T = readtable(fullfile(cfg.dir_data, 'mrva_44.csv'));
M.n_wells = height(T);

% ----- recompute DRASTIC indices --
W = struct('D', 5, 'R', 4, 'A', 3, 'S', 2, 'T', 1, 'I', 5, 'C', 3);
V = W.D * T.D_rating + W.R * T.R_rating + W.A * T.A_rating + ...
    W.S * T.S_rating + W.T * T.T_rating + W.I * T.I_rating + W.C * T.C_rating;
M.drastic_mismatch = sum(abs(V - T.V_DRASTIC) > 1e-9);
if M.drastic_mismatch > 0
    error('mrva: %d DRASTIC indices do not reproduce from their ratings', ...
        M.drastic_mismatch);
end
fprintf('[mrva] %d wells, all DRASTIC indices reproduce from their ratings\n', ..
.
        M.n_wells);

x = T.x_km * 1000; y = T.y_km * 1000; K = T.K_m_per_d;
M.K_range = [min(K) max(K)]; M.K_mean = mean(K); M.K_sd = std(K);
M.V_range = [min(V) max(V)]; M.V_mean = mean(V); M.V_sd = std(V);
fprintf('[mrva] K %.2f - %.1f m/day (mean %.1f, sd %.1f); DRASTIC %g - %g (mean %
.1f)\n', ...
        M.K_range(1), M.K_range(2), M.K_mean, M.K_sd, ...
        M.V_range(1), M.V_range(2), M.V_mean);

% ----- variogram ---
lk = log10(K);
nb = 12;
[I, J] = find(triu(ones(numel(x)), 1));
h = hypot(x(I) - x(J), y(I) - y(J));
g = 0.5 * (lk(I) - lk(J)) .^ 2;
edges = linspace(0, 0.6 * max(h), nb + 1);
hb = zeros(nb, 1); gb = nan(nb, 1); nn = zeros(nb, 1);
for k = 1:nb
    s = h >= edges(k) & h < edges(k + 1);
    nn(k) = sum(s);
    if nn(k) > 0, hb(k) = mean(h(s)); gb(k) = mean(g(s)); end
end
ok = nn >= 10 & isfinite(gb);
s0 = var(lk);
obj = @(p) sum(nn(ok) .* (gb(ok) - (abs(p(1)) + abs(p(2)) * ...
    (1 - exp(-hb(ok) / abs(p(3))))) .^ 2);
p = fminsearch(obj, [0.1 * s0, s0, 0.3 * max(hb)], ...
    optimset('Display', 'off', 'MaxFunEvals', 5000));
M.nugget = abs(p(1)); M.sill = abs(p(2)); M.range = abs(p(3));
fprintf('[mrva] exponential variogram on log10 K: nugget %.4f, sill %.4f, range %
.0f m\n', ...
        M.nugget, M.sill, M.range);

```

```

% ----- grid ----
hull = convhull(x, y);
dx = cfg.mrva_dx;
xs = min(x):dx:max(x);
ys = min(y):dx:max(y);
[XX, YY] = meshgrid(xs, ys);
in = inpolygon(XX, YY, x(hull), y(hull));
M.xa = XX(in); M.ya = YY(in); M.n_active = nnz(in);
M.area_km2 = M.n_active * dx^2 / 1e6;
fprintf('[mrva] convex hull of the wells: %d cells of %g m, %.0f km2\n', ...
        M.n_active, dx, M.area_km2);

% ----- vulnerability field ----
M.V = local_krige(x, y, V, M.xa, M.ya, M.nugget, M.sill, M.range);
fprintf('[mrva] kriged DRASTIC %.1f - %.1f (mean %.1f)\n', ...
        min(M.V), max(M.V), mean(M.V));

% ----- KL basis ----
% Unit prior standard deviation: every reported quantity is a ratio or a
% distance, so the absolute scale cancels.
land = 1:cfg.nystrom_stride:M.n_active;
Xl = M.xa(land); Yl = M.ya(land);
Cll = exp(-hypot(Xl - Xl', Yl - Yl') / M.range);
Cll = (Cll + Cll') / 2;
[U, Lam] = eig(Cll, 'vector');
[Lam, o] = sort(Lam, 'descend'); U = U(:, o);
m = min(cfg.n_kl, sum(Lam > 0));
Lam = Lam(1:m); U = U(:, 1:m);
Phi = zeros(M.n_active, m);
for a = 1:20000:M.n_active
    b = min(a + 19999, M.n_active);
    Cxl = exp(-hypot(M.xa(a:b) - Xl', M.ya(a:b) - Yl') / M.range);
    Phi(a:b, :) = Cxl * (U ./ Lam');
end
for j = 1:m
    nr = norm(Phi(:, j));
    if nr > 0, Phi(:, j) = Phi(:, j) / nr; end
end
M.Phi = Phi;
M.lam = Lam * (M.n_active / numel(land));
M.kl_var_captured = sum(M.lam) / M.n_active;
fprintf('[mrva] KL basis: %d modes, %.1f%% of the prior variance\n', ...
        m, 100 * M.kl_var_captured);

% ----- design ----
cand = true(M.n_active, 1);
for k = 1:numel(x)
    [~, ci] = min(hypot(M.xa - x(k), M.ya - y(k)));
    cand(ci) = false;
end
w = (M.V / mean(M.V)) .^ cfg.gamma_Vdesign;

```

```

M.oed = oed_local(M.Phi, M.lam, w, cfg.sd_local, cand, cfg.n_new_wells, ...
                 M.xa, M.ya, M.range);
fprintf(['[mrva] variance of the vulnerability-weighted mean reduced by ' ...
        '%.1f%% (D-optimal) and %.1f%% (goal-oriented)\n'], ...
        100 * M.oed.var_reduction_D, 100 * M.oed.var_reduction_goal);
fprintf(['[mrva] goal-oriented design leaves %.1f%% less residual variance ' ...
        'than D-optimal; median separation %.0f m\n'], ...
        100 * M.oed.goal_advantage, M.oed.median_separation_m);
M.V_D = M.V(M.oed.D.cells);
M.V_goal = M.V(M.oed.goal.cells);
fprintf(['[mrva] mean DRASTIC at chosen sites: D-optimal %.1f, goal-oriented %.1f\n'
        '\n'], ...
        mean(M.V_D), mean(M.V_goal));
end

% =====
function zg = local_krige(x, y, z, xg, yg, nugget, sill, range)
n = numel(x);
Dd = hypot(x - x', y - y');
Gm = nugget + sill * (1 - exp(-Dd / range));
Gm(1:n+1:end) = 0;
A = [Gm, ones(n, 1); ones(1, n), 0] + 1e-10 * eye(n + 1);
zg = zeros(numel(xg), 1);
for a = 1:5000:numel(xg)
    b = min(a + 4999, numel(xg));
    Dg = hypot(xg(a:b) - x', yg(a:b) - y');
    Gg = nugget + sill * (1 - exp(-Dg / range));
    w = A \ [Gg'; ones(1, b - a + 1)];
    zg(a:b) = w(1:n, :) * z;
end
end

```

A.13 fk_sweep.m

```

function Sw = fk_sweep(cfg, D, G, P, F, Cal, S, Des)
%FK_SWEEP Do the conclusions survive the assumptions they rest on?
%
% Three quantities are followed: the fraction of the total Fisher information
% contributed by the 24 nitrate observations, the information-equivalence
% factor between a concentration observation and a pumping test, and the
% separation between the classical and goal-oriented designs. Each is
% recomputed as the assumptions marked [ASSUM] in fk_config are varied one at
% a time. The dispersivity and the flow azimuth require a new Jacobian and
% are therefore the expensive ones.

Sw = struct();
sig2 = cfg.sigma_obs ^ 2;

% ----- measurement error of the nitrate data
Sw.sigma_obs = [0.5 1.0 1.5 2.0];
Sw.frac_obs_vs_sigma = zeros(size(Sw.sigma_obs));
Sw.R_vs_sigma = zeros(size(Sw.sigma_obs));
for k = 1:numel(Sw.sigma_obs)

```

```

s2 = Sw.sigma_obs(k) ^ 2;
Iobs = (S.Jobs' * S.Jobs) / s2;
Sw.frac_obs_vs_sigma(k) = trace(Iobs) / (trace(S.Iprior) + trace(Iobs));
Sg = inv(S.Iprior + Iobs);
Sg = (Sg + Sg') / 2;
Sw.R_vs_sigma(k) = local_R(cfg, P, S, Des, Sg, s2);
fprintf(['[sweep] sigma_obs %.1f mg/L : observation share of the Fisher ' ...
        'trace %.2e, R = %.3f\n'], Sw.sigma_obs(k), ...
        Sw.frac_obs_vs_sigma(k), Sw.R_vs_sigma(k));
end

% ----- relative error of a pumping test
Sw.sigma_K_rel = [0.05 0.10 0.15 0.20];
Sw.R_vs_sigmaK = zeros(size(Sw.sigma_K_rel));
for k = 1:numel(Sw.sigma_K_rel)
    c2 = cfg; c2.sigma_K_rel = Sw.sigma_K_rel(k);
    Sw.R_vs_sigmaK(k) = local_R(c2, P, S, Des, S.Sigma, sig2);
    fprintf(['[sweep] pumping-test relative error %.2f : R = %.3f\n', ...
            Sw.sigma_K_rel(k), Sw.R_vs_sigmaK(k));
end

% ----- weighting exponent of the goal functional
Sw.gammaV = [0 0.5 1.0 1.5 2.0 3.0];
Sw.sep_vs_gamma = zeros(size(Sw.gammaV));
Sw.adv_vs_gamma = zeros(size(Sw.gammaV));
Sw.V_goal_vs_gamma = zeros(size(Sw.gammaV));
xi_a = G.xa * G.exi(1) + G.ya * G.exi(2);
Lrem = max(xi_a) - xi_a;
cand = true(G.n_active, 1); cand(Cal.cell_of) = false;
for k = 1:numel(Sw.gammaV)
    w = (P.V / mean(P.V)) .^ Sw.gammaV(k);
    c = (P.Phi' * (w .* Lrem)) / (F.q0 * sum(w));
    rD = local_greedy(S.Sigma, S.J, c, cand, sig2, cfg.n_new_wells, 'D', P.vrange
, G);
    rG = local_greedy(S.Sigma, S.J, c, cand, sig2, cfg.n_new_wells, 'goal', P.vra
nge, G);
    d = zeros(cfg.n_new_wells, 1);
    for j = 1:cfg.n_new_wells
        d(j) = min(hypot(G.xa(rG.cells(j)) - G.xa(rD.cells), ...
                        G.ya(rG.cells(j)) - G.ya(rD.cells)));
    end
    Sw.sep_vs_gamma(k) = median(d);
    Sw.adv_vs_gamma(k) = (rD.var_goal(end) - rG.var_goal(end)) / rD.var_goal(end)
;
    Sw.V_goal_vs_gamma(k) = mean(P.V(rG.cells));
    fprintf(['[sweep] gamma_V %.1f : median separation %.0f m, goal-oriented ' ..
.
        'advantage %.1f%%, mean DRASTIC at sites %.1f\n'], ...
        Sw.gammaV(k), Sw.sep_vs_gamma(k), 100 * Sw.adv_vs_gamma(k), ...
        Sw.V_goal_vs_gamma(k));
end

```

```

% ----- dispersivity
Sw.alphaL = [100 250 500 1000];
Sw.frac_obs_vs_alphaL = zeros(size(Sw.alphaL));
Sw.R_vs_alphaL = zeros(size(Sw.alphaL));
Sw.sep_vs_alphaL = zeros(size(Sw.alphaL));
for k = 1:numel(Sw.alphaL)
    c2 = cfg; c2.alphaL = Sw.alphaL(k);
    F2 = fk_forward_quiet(c2, G, P);
    S2 = fk_sensitivity_quiet(c2, G, P, F2, Cal);
    Sw.frac_obs_vs_alphaL(k) = S2.frac_obs;
    Sw.R_vs_alphaL(k) = local_R(c2, P, S2, Des, S2.Sigma, c2.sigma_obs^2);
    w = (P.V / mean(P.V)) .^ cfg.gamma_Vdesign;
    c = (P.Phi' * (w .* Lrem)) / (F2.q0 * sum(w));
    rD = local_greedy(S2.Sigma, S2.J, c, cand, c2.sigma_obs^2, cfg.n_new_wells, '
D', P.vrange, G);
    rG = local_greedy(S2.Sigma, S2.J, c, cand, c2.sigma_obs^2, cfg.n_new_wells, '
goal', P.vrange, G);
    d = zeros(cfg.n_new_wells, 1);
    for j = 1:cfg.n_new_wells
        d(j) = min(hypot(G.xa(rG.cells(j)) - G.xa(rD.cells), ...
                        G.ya(rG.cells(j)) - G.ya(rD.cells)));
    end
    Sw.sep_vs_alphaL(k) = median(d);
    fprintf(['[sweep] alphaL %4d m : observation share %.2e, R = %.3f, ' ...
            'median separation %.0f m\n'], Sw.alphaL(k), ...
            Sw.frac_obs_vs_alphaL(k), Sw.R_vs_alphaL(k), Sw.sep_vs_alphaL(k));
end
end

```

```

% =====
function R = local_R(cfg, P, S, Des, Sigma, sig2)
sd_lK = std(log10(P.Kw));
sd_nK = P.ne_network_sd * (cfg.sigma_K_rel / log(10)) / sd_lK;
c = Des.c_goal;
SigC = Sigma * c;
q = S.J * SigC;
qd = sum((S.J * Sigma) .* S.J, 2);
gc = (q .^ 2 / sig2) ./ (1 + qd / sig2);
qp = P.Phi * SigC;
qpd = sum((P.Phi * Sigma) .* P.Phi, 2);
gp = (qp .^ 2 / sd_nK^2) ./ (1 + qpd / sd_nK^2);
R = max(gc) / max(gp);
end

```

```

% =====
function r = local_greedy(Sigma, J, c, cand, sig2, nsel, mode, min_sep, G)
sel = zeros(nsel, 1);
var_goal = zeros(nsel + 1, 1);
var_goal(1) = c' * Sigma * c;
Sg = Sigma;
for j = 1:nsel
    SgC = Sg * c;

```

```

quad = sum((J * Sg) .* J, 2);
switch mode
    case 'D',    score = log(1 + quad / sig2);
    case 'goal', score = ((J * SgC) .^ 2 / sig2) ./ (1 + quad / sig2);
end
score(~cand) = -inf;
[~, b] = max(score);
sel(j) = b; cand(b) = false;
if min_sep > 0
    cand(hypot(G.xa - G.xa(b), G.ya - G.ya(b)) < min_sep) = false;
end
Jb = J(b, :); u = Sg * Jb';
Sg = Sg - (u * u') / (sig2 + Jb * u);
Sg = (Sg + Sg') / 2;
var_goal(j + 1) = c' * Sg * c;
end
r.cells = sel; r.var_goal = var_goal;
end

% =====
function F = fk_forward_quiet(cfg, G, P)
ev = evalc('F = fk_forward(cfg, G, P);'); %#ok<NASGU>
end

function S = fk_sensitivity_quiet(cfg, G, P, F, Cal)
ev = evalc('S = fk_sensitivity(cfg, G, P, F, Cal);'); %#ok<NASGU>
end

```

A.14 fk_write_results.m

```

function fk_write_results(cfg, R)
%FK_WRITE_RESULTS Write results/results.json, the only source the manuscript
%                  quotes numbers from.

r = struct();
r.run_date = R.run_date;
r.matlab_version = R.matlab_version;
r.runtime_s = R.runtime_s;
r.seed = cfg.seed;

% ----- data ---
r.data.n_wells_total      = R.D.n_total;
r.data.n_wells_inside     = R.D.n_inside;
r.data.n_with_K           = R.D.n_K;
r.data.n_with_NO3         = R.D.n_NO3;
r.data.n_inside_with_K    = R.D.n_inside_K;
r.data.n_inside_with_NO3  = R.D.n_inside_NO3;
r.data.n_boundary_vertices = R.D.n_boundary;
r.data.K_min = min(R.D.K(R.D.has_K));
r.data.K_max = max(R.D.K(R.D.has_K));
r.data.K_mean = mean(R.D.K(R.D.has_K));
r.data.K_sd   = std(R.D.K(R.D.has_K));
r.data.NO3_min = min(R.D.NO3(R.D.has_NO3));

```

```

r.data.NO3_max = max(R.D.NO3(R.D.has_NO3));
r.data.NO3_mean = mean(R.D.NO3(R.D.has_NO3));
r.data.NO3_sd = std(R.D.NO3(R.D.has_NO3));
r.data.DRASTIC_min = min(R.D.DRASTIC);
r.data.DRASTIC_max = max(R.D.DRASTIC);
r.data.DRASTIC_mean = mean(R.D.DRASTIC);
r.data.DRASTIC_sd = std(R.D.DRASTIC);
r.data.facies_counts = [sum(R.D.has_K & R.D.facies == 1), ...
                        sum(R.D.has_K & R.D.facies == 2), ...
                        sum(R.D.has_K & R.D.facies == 3)];
r.data.facies_K_eta2 = R.P.eta2_facies_K;
r.data.facies_K_F = R.P.F_facies_K;
r.data.facies_K_p = R.P.p_facies_K;

% ----- predictability ---
r.predictability.n = R.Pr.n;
r.predictability.idw_power = R.Pr.idw_power;
r.predictability.idw_R2 = R.Pr.idw_R2;
r.predictability.kriging_range_m = R.Pr.krig_range;
r.predictability.kriging_R2 = R.Pr.krig_R2;
r.predictability.linear_names = R.Pr.lin_name;
r.predictability.linear_R2 = R.Pr.lin_R2;
r.predictability.best_R2 = R.Pr.best_R2;
r.predictability.corr_northing = R.Pr.corr_northing;
r.predictability.corr_easting = R.Pr.corr_easting;
r.predictability.corr_alongflow = R.Pr.corr_alongflow;
r.predictability.corr_drastic = R.Pr.corr_drastic;
r.predictability.corr_K = R.Pr.corr_K;
r.predictability.corr_depth = R.Pr.corr_depth;

% ----- grid, prior ----
r.grid.dx_m = cfg.dx;
r.grid.nx = R.G.nx; r.grid.ny = R.G.ny;
r.grid.n_active = R.G.n_active;
r.grid.area_km2 = R.G.area_km2;
r.grid.area_polygon_km2 = R.G.area_polygon_km2;
r.grid.flow_azimuth_deg = cfg.azim_flow_deg;

r.prior.sd_independent = R.P.sd_iv;
r.prior.sd_facies_correlated = R.P.sd_facies;
r.prior.rho_used = R.P.rho_used;
r.prior.sd_used = R.P.sd_facies(2);
r.prior.sd_inflation_pct = 100 * (R.P.sd_facies(2) / R.P.sd_iv - 1);
r.prior.network_mean = R.P.ne_network_mean;
r.prior.network_sd = R.P.ne_network_sd;
r.prior.ne_grid_min = min(R.P.ne);
r.prior.ne_grid_max = max(R.P.ne);
r.prior.ne_grid_mean = mean(R.P.ne);
r.prior.variogram_nugget = R.P.nugget;
r.prior.variogram_sill = R.P.sill;
r.prior.variogram_range_m = R.P.vrange;
r.prior.n_kl = numel(R.P.lam);

```

```

r.prior.kl_variance_captured = R.P.kl_var_captured;

% ----- forward ---
r.forward.gradient = R.F.i_gradient;
r.forward.darcy_flux_m_per_day = R.F.q0;
r.forward.alphaL_m = cfg.alphaL;
r.forward.alphaT_m = cfg.alphaT_ratio * cfg.alphaL;
r.forward.Gamma_min = min(R.F.Gam);
r.forward.Gamma_max = max(R.F.Gam);

% ----- calibration ---
r.calibration.n_obs = R.Cal.n_obs;
r.calibration.basis = R.Cal.psi_names;
r.calibration.T0_days = R.Cal.T0;
r.calibration.T0_years = R.Cal.T0 / 365.25;
r.calibration.rmse_mg_per_L = R.Cal.rmse;
r.calibration.R2 = R.Cal.R2;
r.calibration.R2_without_DRASTIC = R.Cal.R2_noV;
r.calibration.loo_rmse_mg_per_L = R.Cal.loo_rmse;
r.calibration.loo_R2 = R.Cal.loo_R2;
r.calibration.modelled_min = min(R.Cal.C);
r.calibration.modelled_max = max(R.Cal.C);
r.calibration.frac_source_negative_before_clipping = R.Cal.frac_negative;
r.calibration.null_kappa = R.Cal.null_kappa;
r.calibration.null_R2 = R.Cal.null_R2;
r.calibration.null_best_R2 = R.Cal.null_best_R2;
r.calibration.null_best_kappa = R.Cal.null_best_kappa;
r.calibration.null_best_T0_days = R.Cal.null_best_T0;

% ----- Fisher information ----
r.fisher.trace_prior = R.S.tr_prior;
r.fisher.trace_observations = R.S.tr_obs;
r.fisher.observation_share = R.S.frac_obs;
r.fisher.dofs = R.S.dofs;
r.fisher.n_modes = R.S.n_modes;
r.fisher.logdet_prior = R.S.logdet_prior;
r.fisher.logdet_total = R.S.logdet_tot;
r.fisher.information_gain_nats = R.S.info_gain_nats;
r.fisher.sd_reduction_fraction = R.S.sd_reduction;
r.fisher.finite_difference_check = R.S.fd_check_rel;
r.fisher.fd_steps = R.S.fd_steps;
r.fisher.fd_relative = R.S.fd_rel;
r.fisher.fd_convergence_order = R.S.fd_slope;
r.fisher.sd_prior_mean = mean(R.S.sd_prior_field);
r.fisher.sd_post_mean = mean(R.S.sd_post);

% ----- design ---
r.design.n_candidates = R.Des.n_candidates;
r.design.n_new_wells = cfg.n_new_wells;
r.design.sd_local = cfg.sd_local;
r.design.gamma_V = R.Des.gammaV;
r.design.tau_weighted_mean_days = R.Des.tau_mean_prior;

```

```

r.design.sd_goal_prior_days = R.Des.sd_goal_prior;
r.design.sd_goal_after_D_days = sqrt(R.Des.D.var_goal(end));
r.design.sd_goal_after_goal_days = sqrt(R.Des.G.var_goal(end));
r.design.D_cells_xy = [R.G.xa(R.Des.D.cells), R.G.ya(R.Des.D.cells)];
r.design.goal_cells_xy = [R.G.xa(R.Des.G.cells), R.G.ya(R.Des.G.cells)];
r.design.D_DRASTIC = R.Des.V_D;
r.design.goal_DRASTIC = R.Des.V_G;
r.design.domain_mean_DRASTIC = mean(R.P.V);
r.design.separation_m = R.Des.separation_m;
r.design.median_separation_m = median(R.Des.separation_m);
r.design.logdet_gain_D = R.Des.D.logdet_gain;
r.design.information_horizon_m = R.Des.horizon;
r.design.information_correlation_max = R.Des.info_corr_max;
r.design.information_correlation_unconstrained_max = R.Des.info_corr_free_max;
r.design.information_correlation_horizon_max = R.Des.info_corr_hor_max;
r.design.information_horizon_median_m = R.Des.horizon_median;
r.design.min_separation_horizon_m = R.Des.min_sep_horizon;
r.design.sd_goal_after_D_horizon_days = sqrt(R.Des.D_hor.var_goal(end));
r.design.sd_goal_after_goal_horizon_days = sqrt(R.Des.G_hor.var_goal(end));
r.design.unconstrained_span_m = R.Des.free_span_m;
r.design.min_separation_m = R.Des.min_sep;
r.design.horizon_fraction = R.Des.horizon_frac;
r.design.gain_conc_best = R.Des.gain_conc_best;
r.design.gain_pump_best = R.Des.gain_pump_best;
r.design.sd_goal_after_D_free_days = sqrt(R.Des.D_free.var_goal(end));
r.design.sd_goal_after_goal_free_days = sqrt(R.Des.G_free.var_goal(end));
r.design.R_equivalence = R.Des.R_equiv;
r.design.sd_pump_test_on_ne = R.Des.sd_pump_ne;

```

```

% ----- local operator --

```

```

r.local.var_reduction_D = R.Loc.var_reduction_D;
r.local.var_reduction_goal = R.Loc.var_reduction_goal;
r.local.goal_advantage = R.Loc.goal_advantage;
r.local.median_separation_m = R.Loc.median_separation_m;

```

```

% ----- sampling ----

```

```

r.sampling.gn_iterations = R.Par.gn_iters_used;
r.sampling.F_at_map = R.Par.phi_F;
r.sampling.condition_number = R.Par.cond_H;
r.sampling.iters_natural = R.Par.iters_natural;
r.sampling.iters_euclidean = R.Par.iters_euclidean;
r.sampling.speedup = R.Par.speedup;
r.sampling.sqrt_condition = sqrt(R.Par.cond_H);
r.sampling.M_implicit = cfg.M_particles;
r.sampling.ess_implicit = R.Par.ess_implicit;
r.sampling.M_sir = cfg.M_sir;
r.sampling.ess_sir = R.Par.ess_sir;
r.sampling.n_forward_solves = R.Par.n_forward;

```

```

% ----- MRVA ---

```

```

r.mrva.n_wells = R.M.n_wells;
r.mrva.drastic_mismatch = R.M.drastic_mismatch;

```

```

r.mrva.K_min = R.M.K_range(1); r.mrva.K_max = R.M.K_range(2);
r.mrva.K_mean = R.M.K_mean; r.mrva.K_sd = R.M.K_sd;
r.mrva.V_min = R.M.V_range(1); r.mrva.V_max = R.M.V_range(2);
r.mrva.V_mean = R.M.V_mean; r.mrva.V_sd = R.M.V_sd;
r.mrva.variogram_range_m = R.M.range;
r.mrva.area_km2 = R.M.area_km2;
r.mrva.n_active = R.M.n_active;
r.mrva.var_reduction_D = R.M.oed.var_reduction_D;
r.mrva.var_reduction_goal = R.M.oed.var_reduction_goal;
r.mrva.goal_advantage = R.M.oed.goal_advantage;
r.mrva.median_separation_m = R.M.oed.median_separation_m;
r.mrva.mean_DRASTIC_D = mean(R.M.V_D);
r.mrva.mean_DRASTIC_goal = mean(R.M.V_goal);

% ----- sweep ---
r.sweep = R.Sw;

txt = jsonencode(r, 'PrettyPrint', true);
fid = fopen(fullfile(cfg.dir_results, 'results.json'), 'w');
fwrite(fid, txt);
fclose(fid);
fprintf('[out] results/results.json written (%d characters)\n', numel(txt));
end

```

A.15 fk_figures.m

```

function fk_figures(R)
%FK_FIGURES Every figure of the manuscript, from results/run_all.mat.
%
% Figures are drawn at the width they are printed at (90 mm one column,
% 190 mm two columns), with Position equal to PaperPosition so that nothing
% is rescaled on export, and written at 600 dpi as PNG and TIFF.

if nargin < 1
    cfg0 = fk_config();
    R = load(fullfile(cfg0.dir_results, 'run_all.mat'));
end
cfg = R.cfg; D = R.D; G = R.G; P = R.P; Cal = R.Cal; S = R.S;
Des = R.Des; Loc = R.Loc; Par = R.Par; M = R.M; Sw = R.Sw; Pr = R.Pr;
if ~exist(cfg.dir_figures, 'dir'), mkdir(cfg.dir_figures); end

set(0, 'DefaultAxesFontName', 'Arial', 'DefaultTextFontName', 'Arial', ...
    'DefaultAxesFontSize', 8, 'DefaultTextFontSize', 8, ...
    'DefaultAxesLineWidth', 0.5, 'DefaultLineLineWidth', 0.9);
W2 = 19.0;
CB = [0.00 0.45 0.74; 0.85 0.33 0.10; 0.47 0.67 0.19; 0.49 0.18 0.56];

% ===== Figure 1 =====
f = newfig(W2, 10.0);
subplot(1, 3, 1);
plot(D.bx/1000, D.by/1000, 'k-', 'LineWidth', 0.6); hold on
plot(D.x(~D.has_K)/1000, D.y(~D.has_K)/1000, '.', 'Color', [.65 .65 .65], ...
    'MarkerSize', 4);

```

```

plot(D.x(D.has_K)/1000, D.y(D.has_K)/1000, 'o', 'MarkerSize', 3, ...
     'MarkerFaceColor', CB(1,:), 'MarkerEdgeColor', 'none');
plot(D.x(D.has_NO3)/1000, D.y(D.has_NO3)/1000, '^', 'MarkerSize', 3.4, ...
     'MarkerFaceColor', CB(2,:), 'MarkerEdgeColor', 'none');
axis equal tight; box on
xlabel('Easting (km)'); ylabel('Northing (km)');
title(sprintf('(a) %d points, %d with K, %d with NO_3', ...
             D.n_total, D.n_K, D.n_NO3), 'FontWeight', 'normal');
legend({'boundary', 'DRASTIC only', 'pumping test', 'nitrate'}, ...
      'Location', 'southoutside', 'Box', 'off');

subplot(1, 3, 2);
scatterfield(G, P.V); hold on
plot(D.bx/1000, D.by/1000, 'k-', 'LineWidth', 0.4);
quiverflow(G);
c = colorbar('southoutside'); c.Label.String = 'DRASTIC index';
title('(b) vulnerability and regional flow', 'FontWeight', 'normal');

subplot(1, 3, 3);
scatterfield(G, P.ne);
c = colorbar('southoutside'); c.Label.String = 'effective porosity prior';
title('(c) porosity prior', 'FontWeight', 'normal');
printfig(f, cfg, 'fig1_study_area');

% ===== Figure 2 =====
f = newfig(W2, 7.5);
subplot(1, 2, 1);
names = [arrayfun(@(p) sprintf('IDW %d', p), Pr.idw_power, 'uni', 0), ...
         arrayfun(@(r) sprintf('kriging %g km', r/1000), Pr.krig_range, 'uni', 0)
, ...
         Pr.lin_name, {'transport model'}];
vals = [Pr.idw_R2, Pr.krig_R2, Pr.lin_R2, Cal.loo_R2];
barh(vals, 'FaceColor', CB(1,:), 'EdgeColor', 'none'); hold on
set(gca, 'YTick', 1:numel(vals), 'YTickLabel', names, 'YDir', 'reverse', ...
     'TickLabelInterpreter', 'none');
xline(0, 'k-', 'LineWidth', 0.9);
xlabel('leave-one-out R^2'); box on
title('(a) nothing predicts a held-out well', 'FontWeight', 'normal');

subplot(1, 2, 2);
plot(Cal.y_obs, Cal.y_fit, 'o', 'MarkerSize', 4, ...
     'MarkerFaceColor', CB(1,:), 'MarkerEdgeColor', 'none'); hold on
plot(Cal.y_obs, Cal.loo_pred, 's', 'MarkerSize', 4, ...
     'MarkerFaceColor', CB(2,:), 'MarkerEdgeColor', 'none');
lim = [0 max([Cal.y_obs; Cal.y_fit; Cal.loo_pred]) * 1.15];
plot(lim, lim, 'k--', 'LineWidth', 0.6);
xlim(lim); ylim(lim); axis square; box on
xlabel('observed NO_3 (mg L^{-1})'); ylabel('modelled NO_3 (mg L^{-1})');
legend({sprintf('in sample, R^2 = %.2f', Cal.R2), ...
       sprintf('leave one out, R^2 = %.2f', Cal.loo_R2), '1:1'}, ...
      'Location', 'northwest', 'Box', 'off');
title('(b) in sample against out of sample', 'FontWeight', 'normal');

```

```

printfig(f, cfg, 'fig2_predictability');

% ===== Figure 3 =====
f = newfig(W2, 7.0);
subplot(1, 3, 1);
semilogy(P.lam / P.lam(1), 'k-'); box on
xlabel('Karhunen-Loeve mode'); ylabel('\lambda_k / \lambda_1');
title(sprintf('(a) %d modes, %.0f%% of prior variance', ...
    numel(P.lam), 100*P.kl_var_captured), 'FontWeight', 'normal');
subplot(1, 3, 2);
plot(P.vario_h/1000, P.vario_g, 'o', 'MarkerSize', 3.5, ...
    'MarkerFaceColor', CB(1,:), 'MarkerEdgeColor', 'none'); hold on
hh = linspace(0, max(P.vario_h), 200);
plot(hh/1000, P.nugget + P.sill*(1 - exp(-hh/P.vrange)), 'k-');
xlabel('lag (km)'); ylabel('\gamma of log_{10}K'); box on
title(sprintf('(b) exponential range %.0f m', P.vrange), 'FontWeight', 'normal');
subplot(1, 3, 3);
scatterfield(G, Cal.source / (cfg.dx*cfg.dy));
c = colorbar('southoutside'); c.Label.String = 'loading (mg L^{-1} d^{-1})';
title(sprintf('(c) fitted loading, T_0 = %.0f yr', Cal.T0/365.25), ...
    'FontWeight', 'normal');
printfig(f, cfg, 'fig3_prior_and_loading');

% ===== Figure 4 =====
f = newfig(W2, 7.0);
subplot(1, 3, 1);
scatterfield(G, Cal.C);
c = colorbar('southoutside'); c.Label.String = 'NO_3 (mg L^{-1})';
title('(a) modelled nitrate at T_0', 'FontWeight', 'normal');
subplot(1, 3, 2);
scatterfield(G, S.sd_prior_field);
c = colorbar('southoutside'); c.Label.String = 'prior sd of n_e';
title('(b) prior standard deviation', 'FontWeight', 'normal');
subplot(1, 3, 3);
scatterfield(G, S.sd_prior_field - S.sd_post);
c = colorbar('southoutside'); c.Label.String = 'sd reduction';
title(sprintf('(c) what the %d nitrate wells buy', Cal.n_obs), ...
    'FontWeight', 'normal');
printfig(f, cfg, 'fig4_information');

% ===== Figure 5 =====
% The transport-operator design, and the greedy clustering pathology.
f = newfig(W2, 9.0);
subplot(1, 3, 1);
scatterfield(G, log10(max(Des.gain_conc, realmin))); hold on
plot(G.xa(Des.D_free.cells)/1000, G.ya(Des.D_free.cells)/1000, 'ko', ...
    'MarkerSize', 5, 'LineWidth', 1);
c = colorbar('southoutside'); c.Label.String = 'log_{10} variance gain';
title(sprintf('(a) unconstrained greedy: %d wells in %.0f m', ...
    cfg.n_new_wells, Des.free_span_m), 'FontWeight', 'normal');

subplot(1, 3, 2);

```

```

scatterfield(G, P.V); hold on
plot(G.xa(Des.D.cells)/1000, G.ya(Des.D.cells)/1000, 'ks', ...
     'MarkerSize', 6, 'LineWidth', 1);
plot(G.xa(Des.G.cells)/1000, G.ya(Des.G.cells)/1000, 'rp', ...
     'MarkerSize', 8, 'LineWidth', 1);
c = colorbar('southoutside'); c.Label.String = 'DRASTIC index';
title(sprintf('(b) separated by %.0f m', Des.min_sep), 'FontWeight', 'normal');

subplot(1, 3, 3);
plot(0:cfg.n_new_wells, sqrt(Des.D.var_goal), 'ks-', 'MarkerSize', 4, ...
     'MarkerFaceColor', 'k'); hold on
plot(0:cfg.n_new_wells, sqrt(Des.G.var_goal), 'r^--', 'MarkerSize', 4, ...
     'MarkerFaceColor', 'r');
xlabel('concentration wells added');
ylabel('sd of weighted arrival time (d)');
legend({'D-optimal', 'goal-oriented'}, 'Box', 'off', 'Location', 'northeast');
box on; title('(c) almost nothing to allocate', 'FontWeight', 'normal');
printfig(f, cfg, 'fig5_transport_design');

% ===== Figure 6 =====
% The local-operator design, where the criterion does matter, at both sites.
f = newfig(W2, 9.0);
subplot(1, 3, 1);
scatterfield(G, P.V); hold on
plot(G.xa(Loc.D.cells)/1000, G.ya(Loc.D.cells)/1000, 'ks', ...
     'MarkerSize', 6, 'LineWidth', 1);
plot(G.xa(Loc.goal.cells)/1000, G.ya(Loc.goal.cells)/1000, 'rp', ...
     'MarkerSize', 8, 'LineWidth', 1);
c = colorbar('southoutside'); c.Label.String = 'DRASTIC index';
title('(a) Fushe-Kuqe, characterisation points', 'FontWeight', 'normal');

subplot(1, 3, 2);
nw = cfg.n_new_wells;
plot(0:nw, Loc.D.var_goal / Loc.var_prior, 'ks-', 'MarkerSize', 4, ...
     'MarkerFaceColor', 'k'); hold on
plot(0:nw, Loc.goal.var_goal / Loc.var_prior, 'r^--', 'MarkerSize', 4, ...
     'MarkerFaceColor', 'r');
plot(0:nw, M.oed.D.var_goal / M.oed.var_prior, 'ks:', 'MarkerSize', 4);
plot(0:nw, M.oed.goal.var_goal / M.oed.var_prior, 'r^:', 'MarkerSize', 4);
xlabel('points added'); ylabel('Var(g) relative to prior'); ylim([0 1]);
legend({'Fushe-Kuqe D', 'Fushe-Kuqe goal', 'MRVA D', 'MRVA goal'}, ...
     'Box', 'off', 'Location', 'southwest');
box on; title('(b) goal orientation pays', 'FontWeight', 'normal');

subplot(1, 3, 3);
scatter(M.xa/1000, M.ya/1000, 4, M.V, 'filled', 'Marker', 's'); hold on
plot(M.xa(M.oed.D.cells)/1000, M.ya(M.oed.D.cells)/1000, 'ks', ...
     'MarkerSize', 6, 'LineWidth', 1);
plot(M.xa(M.oed.goal.cells)/1000, M.ya(M.oed.goal.cells)/1000, 'rp', ...
     'MarkerSize', 8, 'LineWidth', 1);
axis equal tight; box on
xlabel('x (km)'); ylabel('y (km)');

```

```

c = colorbar('southoutside'); c.Label.String = 'DRASTIC index';
title(sprintf('(c) MRVA, %d USGS wells', M.n_wells), 'FontWeight', 'normal');
printfig(f, cfg, 'fig6_local_design_and_replication');

% ===== Figure 7 =====
f = newfig(W2, 6.5);
subplot(1, 4, 1);
bar([Par.ess_implicit, Par.ess_sir], 'FaceColor', CB(1,:), 'EdgeColor', 'none');
set(gca, 'XTickLabel', {sprintf('implicit\nM=%d', cfg.M_particles), ...
                        sprintf('SIR\nM=%d', cfg.M_sir)});
ylabel('normalised ESS'); box on; ylim([0 1]);
title('(a) both samplers are easy', 'FontWeight', 'normal');

subplot(1, 4, 2);
plot(Sw.sigma_obs, Sw.R_vs_sigma, 'ko-', 'MarkerFaceColor', 'k', ...
     'MarkerSize', 4); hold on
yline(1, 'r--');
xlabel('\sigma_{obs} (mg L^{-1})'); ylabel('R'); box on
title('(b) R against noise', 'FontWeight', 'normal');

subplot(1, 4, 3);
semilogx(Sw.alpha_L, Sw.R_vs_alpha_L, 'ko-', 'MarkerFaceColor', 'k', ...
         'MarkerSize', 4); hold on
yline(1, 'r--');
xlabel('\alpha_L (m)'); ylabel('R'); box on
title('(c) R against dispersivity', 'FontWeight', 'normal');

subplot(1, 4, 4);
yyaxis left; plot(Sw.gammaV, Sw.sep_vs_gamma/1000, 'o-', 'MarkerSize', 4);
ylabel('median separation (km)');
yyaxis right; plot(Sw.gammaV, 100*Sw.adv_vs_gamma, 's-', 'MarkerSize', 4);
ylabel('goal advantage (%)'); xlabel('\gamma_V'); box on
title('(d) weighting exponent', 'FontWeight', 'normal');
printfig(f, cfg, 'fig7_sampling_and_sensitivity');

% ===== graphical abstract =====
% Elsevier asks for 531 x 1328 px at 96 dpi minimum; 13.3 x 5.3 cm at 600 dpi
% is comfortably above that.
f = newfig(13.3, 5.3);
ax1 = axes('Position', [0.04 0.18 0.28 0.66]);
scatterfield(G, P.V); hold on
plot(D.x(D.has_NO3)/1000, D.y(D.has_NO3)/1000, '^', 'MarkerSize', 3.2, ...
     'MarkerFaceColor', 'k', 'MarkerEdgeColor', 'none');
set(ax1, 'XTickLabel', [], 'YTickLabel', []);
xlabel(''); ylabel('');
title(sprintf('%d nitrate wells', Cal.n_obs), 'FontWeight', 'normal', ...
     'FontSize', 8);

ax2 = axes('Position', [0.345 0.12 0.265 0.76]);
% Typography rather than a chart: the quantity is a fraction of 150 so small
% that no shared axis can show it and be read at abstract size.
axis off; xlim([0 1]); ylim([0 1]); hold on

```

```

text(0.5, 0.92, 'the network already has', 'HorizontalAlignment', 'center', ...
    'FontSize', 8.5);
text(0.5, 0.74, sprintf('%.3f', S.dofs), 'HorizontalAlignment', 'center', ...
    'FontSize', 26, 'FontWeight', 'bold', 'Color', CB(2,:));
text(0.5, 0.57, sprintf('of %d directions constrained', numel(P.lam)), ...
    'HorizontalAlignment', 'center', 'FontSize', 8.5);
plot([0.12 0.88], [0.46 0.46], 'k-', 'LineWidth', 0.6);
text(0.5, 0.35, 'one nitrate sample buys', 'HorizontalAlignment', 'center', ...
    'FontSize', 8.5);
text(0.5, 0.20, sprintf('%.2f', Des.R_equiv), 'HorizontalAlignment', 'center', ..
    'FontSize', 22, 'FontWeight', 'bold', 'Color', CB(2,:));
text(0.5, 0.05, 'of a pumping test', 'HorizontalAlignment', 'center', ...
    'FontSize', 8.5);

ax3 = axes('Position', [0.735 0.24 0.245 0.58]);
plot(0:cfg.n_new_wells, Loc.D.var_goal / Loc.var_prior, 'ks-', ...
    'MarkerSize', 4, 'MarkerFaceColor', 'k'); hold on
plot(0:cfg.n_new_wells, Loc.goal.var_goal / Loc.var_prior, 'r^--', ...
    'MarkerSize', 4, 'MarkerFaceColor', 'r');
xlabel('characterisation points'); ylabel('Var(g) / prior');
ylim([0 1]); box on
legend({'D-optimal', 'goal-oriented'}, 'Box', 'off', 'FontSize', 7, ...
    'Location', 'southwest');
title('where design pays', 'FontWeight', 'normal', 'FontSize', 8);
printfig(f, cfg, 'graphical_abstract');

fprintf('[fig] seven figures and a graphical abstract written to %s\n', ...
    cfg.dir_figures);
end

% =====
function f = newfig(wcm, hcm)
f = figure('Units', 'centimeters', 'Position', [2 2 wcm hcm], 'Color', 'w');
set(f, 'PaperUnits', 'centimeters', 'PaperSize', [wcm hcm], ...
    'PaperPosition', [0 0 wcm hcm], 'PaperPositionMode', 'manual');
end

function printfig(f, cfg, name)
print(f, fullfile(cfg.dir_figures, [name '.png']), '-dpng', '-r600');
print(f, fullfile(cfg.dir_figures, [name '.tif']), '-dtiff', '-r600');
close(f);
end

function scatterfield(G, v)
scatter(G.xa/1000, G.ya/1000, 1.5, v, 'filled', 'Marker', 's');
axis equal tight; box on
xlabel('Easting (km)'); ylabel('Northing (km)');
end

function quiverflow(G)
xc = mean(G.xa)/1000; yc = mean(G.ya)/1000;

```

```

L = 0.15 * (max(G.xa) - min(G.xa)) / 1000;
quiver(xc, yc, G.exi(1)*L, G.exi(2)*L, 0, 'k', 'LineWidth', 1.2, ...
    'MaxHeadSize', 2);
text(xc + G.exi(1)*L, yc + G.exi(2)*L, ' flow', 'FontSize', 7);
end

```

A.16 fk_prepare_data.m

```

function fk_prepare_data()
%FK_PREPARE_DATA Extract the analysis tables from the sources into flat CSVs.
%
% Inputs, read only and never modified:
%   Fushe_Kuqe_All_Data.xlsx           the Desktop copy, which carries three
%                                       sheets transcribed from Cenameri and
%                                       Beqiraj (2016) besides the well and
%                                       boundary tables
%   Pugh2023_MRVA_44wells-REAL.csv    the USGS Mississippi Alluvial Plain set
%
% Outputs, into ../data:
%   wells_all.csv                     180 wells, every column of the well sheet
%   wells_K.csv                       the wells carrying a pumping-test conductivity
%   wells_NO3.csv                     the wells carrying a nitrate measurement
%   boundary.csv                      the aquifer boundary vertices
%   swi_chemistry.csv                 major ions at four wells
%   swi_cl_trend.csv                  chloride in 1984, 1999 and 2001
%   swi_facts.csv                     flow direction, heads, thicknesses, wedge geometry
%   mrva_44.csv                       the USGS set
%   manifest.json                     the counts, so a reader can check the extraction
%
% No value is altered, imputed or rounded: rows are copied or dropped only.

cfg = fk_config();
out = cfg.dir_data;
if ~exist(out, 'dir'), mkdir(out); end

xlsx = 'C:\Users\d_zeq\OneDrive\Desktop\Fushe_Kuqe_All_Data.xlsx';
mrva = ['C:\Users\d_zeq\OneDrive\Desktop\PROJEKTE 2 ARTIKUJ\' ...
    'AKUIFER TE DHENA TE NDRYSHME FUSHE KUQE TE DHENA\' ...
    'Pugh2023_MRVA_44wells-REAL.csv'];
for f = {xlsx, mrva}
    if ~exist(f{1}, 'file')
        error('fk_prepare_data:source', 'missing source %s', f{1});
    end
end

% ----- wells -----
T = readtable(xlsx, 'Sheet', '2_All_Wells', 'VariableNamingRule', 'preserve');
T = T(~ismissing(T.(1)), :);
writetable(T, fullfile(out, 'wells_all.csv'));

hasK = ~ismissing(T.('K_m_per_day'));
hasN = ~ismissing(T.('NO3_mg_per_L'));
writetable(T(hasK, :), fullfile(out, 'wells_K.csv'));

```

```

writetable(T(hasN, :), fullfile(out, 'wells_NO3.csv'));

B = readtable(xlsx, 'Sheet', '3_Boundary_Coords', 'VariableNamingRule', 'preserve');
B = B(~ismissing(B.(1)), :);
writetable(B, fullfile(out, 'boundary.csv'));

% ----- the three seawater-intrusion sheets --
% Each carries two banner lines and a blank line before the real header, so
% the header row is located rather than assumed.
swi = {'5_Cenameri2016_Chemistry', 'swi_chemistry.csv'; ...
      '6_Cenameri2016_Cl_Trend', 'swi_cl_trend.csv'; ...
      '7_Cenameri2016_SWI_Facts', 'swi_facts.csv'};
sheets = sheetnames(xlsx);
nswi = zeros(size(swi, 1), 1);
for k = 1:size(swi, 1)
    if ~any(strcmp(sheets, swi{k, 1})), continue; end
    C = readcell(xlsx, 'Sheet', swi{k, 1});
    hdr = local_header_row(C);
    S = cell2table(local_clean(C(hdr+1:end, :)), ...
                  'VariableNames', local_names(C(hdr, :)));
    S = S(~local_blank(S), :);
    writetable(S, fullfile(out, swi{k, 2}));
    nswi(k) = height(S);
end

% ----- MRVA -----
M = readtable(mrva, 'VariableNamingRule', 'preserve');
writetable(M, fullfile(out, 'mrva_44.csv'));

% ----- manifest -----
inside = strcmp(string(T('Inside_Aquifer')), "YES");
lith = string(T('Lithology'));
man = struct();
man.source_workbook = xlsx;
man.source_mrva = mrva;
man.n_wells_total = height(T);
man.n_wells_with_K = sum(hasK);
man.n_wells_with_NO3 = sum(hasN);
man.n_wells_inside = sum(inside);
man.n_inside_with_K = sum(inside & hasK);
man.n_inside_with_NO3 = sum(inside & hasN);
man.n_boundary_vertices = height(B);
man.n_mrva = height(M);
man.swi_rows = nswi;
u = unique(lith(hasK));
lc = struct();
for k = 1:numel(u)
    lc.(matlab.lang.makeValidName(char(u(k)))) = sum(hasK & lith == u(k));
end
man.lithology_of_K_wells = lc;

```

```

fid = fopen(fullfile(out, 'manifest.json'), 'w');
fwrite(fid, jsonencode(man, 'PrettyPrint', true));
fclose(fid);

fprintf(['[prep] %d wells, %d with K, %d with NO3, %d inside, ' ...
        '%d boundary vertices, %d MRVA\n'], man.n_wells_total, ...
        man.n_wells_with_K, man.n_wells_with_NO3, man.n_wells_inside, ...
        man.n_boundary_vertices, man.n_mrva);
end

% =====
function h = local_header_row(C)
h = 0;
for k = 2:size(C, 1)
    row = C(k, :);
    filled = sum(cellfun(@(v) ~local_ismissing(v), row));
    if filled >= 2
        h = k;
        return
    end
end
error('fk_prepare_data:header', 'no header row found');
end

function tf = local_ismissing(v)
tf = isa(v, 'missing') || (ischar(v) && isempty(v)) || ...
    (isnumeric(v) && isscalar(v) && isnan(v)) || ...
    (isstring(v) && (ismissing(v) || v == ""));
end

function C = local_clean(C)
for k = 1:numel(C)
    if local_ismissing(C{k}), C{k} = ''; end
    if isstring(C{k}), C{k} = char(C{k}); end
    if isnumeric(C{k}) && isscalar(C{k}), C{k} = num2str(C{k}); end
    if ~ischar(C{k}), C{k} = ''; end
end
end

function nm = local_names(row)
nm = cell(1, numel(row));
for k = 1:numel(row)
    v = row{k};
    if local_ismissing(v), v = sprintf('Var%d', k); end
    if ~ischar(v), v = char(string(v)); end
    nm{k} = matlab.lang.makeValidName(v);
end
nm = matlab.lang.makeUniqueStrings(nm);
end

function tf = local_blank(S)
C = table2cell(S);

```

```

tf = all(cellfun(@(v) isempty(v), C), 2);
first = C(:, 1);
tf = tf | cellfun(@(v) ischar(v) && (startsWith(v, 'Note') || ...
    startsWith(v, '- ')), first);

end

```

A.17 fk_fill_manuscript.m

```

function fk_fill_manuscript(github_url, zenodo_doi)
%FK_FILL_MANUSCRIPT Put the run's numbers into the manuscript.
%
% fk_fill_manuscript()
% fk_fill_manuscript(github_url, zenodo_doi)
%
% Substitutes every {{placeholder}} in manuscript/manuscript.md with a value
% taken from results/results.json and writes manuscript/manuscript_filled.md.
% It refuses to write anything if a placeholder has no value behind it, and
% reports values that were computed and never cited, which catches a claim
% dropped from the text but not from the code.
%
% This is what keeps the manuscript honest: a number reaches the text only by
% coming out of a run.
%
% The output is also checked for control characters. That check is here
% because one crept into the source on 12 September 2026 and turned
% $\alpha_L$ into $\x07lpha_L$; the converter dropped the byte silently and
% the equation reached a delivered document as the word "lpha".

cfg = fk_config();
man = fullfile(cfg.dir_code, '..', 'manuscript');
src = fullfile(man, 'manuscript.md');
dst = fullfile(man, 'manuscript_filled.md');
res = fullfile(cfg.dir_results, 'results.json');

if nargin < 1 || isempty(github_url)
    github_url = 'https://github.com/dulianzeqiraj/fushe-kuqe-oed';
end
if nargin < 2 || isempty(zenodo_doi)
    zenodo_doi = 'https://doi.org/10.5281/zenodo.PLACEHOLDER';
end

r = jsondecode(fileread(res));
m = local_values(r, github_url, zenodo_doi);

txt = fileread(src);
keys = fieldnames(m);
used = false(numel(keys), 1);
for k = 1:numel(keys)
    tag = ['{' keys{k} '}'];
    if contains(txt, tag)
        used(k) = true;
        txt = strrep(txt, tag, m.(keys{k}));
    end
end

```

```

end

left = regexp(txt, '\\{\\{\\(\\w+\\)\\}\\}', 'tokens');
if ~isempty(left)
    names = unique(cellfun(@(c) c{1}, left, 'UniformOutput', false));
    error('fk_fill_manuscript:unresolved', ...
        'no value for: %s', strjoin(names, ', '));
end

ctrl = double(txt) < 32 & ~ismember(double(txt), [9 10 13]);
if any(ctrl)
    j = find(ctrl, 1);
    error('fk_fill_manuscript:control', ...
        'control character U+%04X at offset %d, context "%s"', ...
        double(txt(j)), j, txt(max(1, j-40):min(numel(txt), j+20)));
end

fid = fopen(dst, 'w', 'n', 'UTF-8');
fwrite(fid, unicode2native(txt, 'UTF-8'));
fclose(fid);

fprintf('[fill] %s written, %d placeholders resolved\n', dst, sum(used));
if any(~used)
    fprintf('[fill] computed but unused: %s\n', ...
        strjoin(keys(~used), ', '));
end
end

% =====
function m = local_values(r, github_url, zenodo_doi)
d = r.data; p = r.predictability; g = r.grid; pr = r.prior;
fw = r.forward; cal = r.calibration; fi = r.fisher; des = r.design;
loc = r.local; sam = r.sampling; mr = r.mrva; sw = r.sweep;
m = struct();
f = @(fmt, v) sprintf(fmt, v);
pct = @(x) [sprintf('%.1f', 100*x) ' per cent'];
rng2 = @(v, fmt) [sprintf(fmt, min(v)) ' to ' sprintf(fmt, max(v))];

% ----- data ---
m.n_total = f('%d', d.n_wells_total);
m.n_inside = f('%d', d.n_wells_inside);
m.n_K = f('%d', d.n_with_K);
m.n_NO3 = f('%d', d.n_with_NO3);
m.n_inside_K = f('%d', d.n_inside_with_K);
m.n_inside_NO3 = f('%d', d.n_inside_with_NO3);
m.n_boundary = f('%d', d.n_boundary_vertices);
m.K_min = f('%.0f', d.K_min); m.K_max = f('%.0f', d.K_max);
m.K_mean = f('%.1f', d.K_mean); m.K_sd = f('%.1f', d.K_sd);
m.NO3_min = f('%.2f', d.NO3_min); m.NO3_max = f('%.2f', d.NO3_max);
m.NO3_mean = f('%.2f', d.NO3_mean); m.NO3_sd = f('%.2f', d.NO3_sd);
m.DR_min = f('%.0f', d.DRASTIC_min); m.DR_max = f('%.0f', d.DRASTIC_max);
m.DR_mean = f('%.1f', d.DRASTIC_mean);

```

```

m.facies_c = f('%d', d.facies_counts(1));
m.facies_m = f('%d', d.facies_counts(2));
m.facies_f = f('%d', d.facies_counts(3));
m.eta2 = f('%.3f', d.facies_K_eta2);
m.Fstat = f('%.2f', d.facies_K_F);
m.pval = f('%.2f', d.facies_K_p);
m.df1 = '2';
m.df2 = f('%d', d.n_with_K - 3);

% ----- predictability ---
m.best_pred_R2 = f('%+.2f', p.best_R2);
m.krig_min = f('%+.2f', min(p.kriging_R2));
m.krig_max = f('%+.2f', max(p.kriging_R2));
m.idw_min = f('%+.2f', min(p.idw_R2));
m.idw_max = f('%+.2f', max(p.idw_R2));
lin = local_lookup(p.linear_names, p.linear_R2);
m.lin_xy = f('%+.2f', lin('X,Y'));
m.lin_drastic = f('%+.2f', lin('DRASTIC'));
m.corr_north = f('%+.2f', p.corr_northing);
m.corr_along = f('%+.2f', p.corr_alongflow);
m.corr_drastic = f('%+.2f', p.corr_drastic);
m.corr_K = f('%+.2f', p.corr_K);

% ----- grid and prior ---
m.nx = f('%d', g.nx); m.ny = f('%d', g.ny);
m.dx = f('%g', g.dx_m);
m.n_active = f('%d', g.n_active);
m.area_grid = f('%.1f', g.area_km2);
m.area_poly = f('%.1f', g.area_polygon_km2);
m.ne_mean = f('%.3f', pr.network_mean);
m.ne_sd = f('%.3f', pr.network_sd);
m.sd_iv = f('%.4f', pr.sd_independent);
m.sd_c = f('%.4f', pr.sd_facies_correlated(1));
m.sd_m = f('%.4f', pr.sd_facies_correlated(2));
m.sd_f = f('%.4f', pr.sd_facies_correlated(3));
m.sd_used = f('%.4f', pr.sd_used);
m.sd_inflation = f('%.1f', pr.sd_inflation_pct);
m.nugget = f('%.4f', pr.variogram_nugget);
m.sill = f('%.4f', pr.variogram_sill);
m.vrange = f('%.0f', pr.variogram_range_m);
m.n_kl = f('%d', pr.n_kl);
m.kl_var = f('%.0f', 100*pr.kl_variance_captured);

% ----- forward ---
m.gradient = f('%.2e', fw.gradient);
m.q0 = f('%.4f', fw.darcy_flux_m_per_day);
m.gam_min = f('%.2f', fw.Gamma_min);
m.gam_max = f('%.2f', fw.Gamma_max);

% ----- calibration ---
m.T0_yr = f('%.1f', cal.T0_years);
m.cal_rmse = f('%.2f', cal.rmse_mg_per_L);

```

```

m.cal_R2 = f('%.2f', cal.R2);
m.cal_loo_rmse = f('%.2f', cal.loo_rmse_mg_per_L);
m.cal_loo_R2 = f('%.2f', cal.loo_R2);
m.drastic_contrib = f('%.3f', cal.R2 - cal.R2_without_DRASTIC);
m.null_R2 = f('%.2f', cal.null_best_R2);
m.null_kappa = f('%.2f', cal.null_best_kappa);
m.frac_neg = [sprintf('%.0f', 100*cal.frac_source_negative_before_clipping) ' per
cent'];

% ----- Fisher information ---
m.tr_obs = f('%.3e', fi.trace_observations);
m.tr_prior = f('%.3e', fi.trace_prior);
m.frac_obs_sci = f('%.1e', fi.observation_share);
m.dofs = f('%.4f', fi.dofs);
m.n_modes = f('%d', fi.n_modes);
m.info_gain = f('%.3f', fi.information_gain_nats);
m.sd_red_pct = f('%.2f', 100*fi.sd_reduction_fraction);
m.sd_prior_mean = f('%.5f', fi.sd_prior_mean);
m.sd_post_mean = f('%.5f', fi.sd_post_mean);
m.fd_check = f('%.1e', fi.finite_difference_check);
m.fd_order = f('%.2f', fi.fd_convergence_order);

% ----- design ---
m.horizon_frac = f('%.0f', 100*des.horizon_fraction);
m.n_new = f('%d', des.n_new_wells);
m.sd_local = f('%.3f', des.sd_local);
m.tau_mean_yr = f('%.1f', des.tau_weighted_mean_days/365.25);
m.sd_goal_prior = f('%.1f', des.sd_goal_prior_days);
m.sd_goal_D = f('%.1f', des.sd_goal_after_D_days);
m.sd_goal_G = f('%.1f', des.sd_goal_after_goal_days);
m.sd_goal_D_hor = f('%.1f', des.sd_goal_after_D_horizon_days);
m.sd_goal_G_hor = f('%.1f', des.sd_goal_after_goal_horizon_days);
m.sd_goal_red_pct = f('%.1f', 100*(1 - des.sd_goal_after_goal_days/des.sd_goal_pr
ior_days));
m.median_sep_m = f('%.0f', des.median_separation_m);
m.V_goal = f('%.1f', mean(des.goal_DRASTIC));
m.V_D = f('%.1f', mean(des.D_DRASTIC));
m.V_domain = f('%.1f', des.domain_mean_DRASTIC);
adv = (des.sd_goal_after_D_days^2 - des.sd_goal_after_goal_days^2) / ...
    des.sd_goal_after_D_days^2;
m.goal_adv_pct = pct(adv);
m.horizon_med = f('%.0f', median(des.information_horizon_m));
m.horizon_min = f('%.0f', min(des.information_horizon_m));
m.horizon_max = f('%.0f', max(des.information_horizon_m));
m.horizon_over_range = f('%.1f', des.information_horizon_median_m / pr.variogram_
range_m);
m.corr_constrained = f('%.3f', des.information_correlation_max);
m.corr_horizon = f('%.3f', des.information_correlation_horizon_max);
m.corr_free = f('%.3f', des.information_correlation_unconstrained_max);
m.free_span = f('%.0f', des.unconstrained_span_m);
m.min_sep = f('%.0f', des.min_separation_m);
m.R_equiv = f('%.2f', des.R_equivalence);

```

```

m.R_inverse = f('%0f', 1/des.R_equivalence);
m.sd_pump_ne = f('%4f', des.sd_pump_test_on_ne);
m.gain_conc = f('%3g', des.gain_conc_best);
m.gain_pump = f('%3g', des.gain_pump_best);

% ----- local operator ---
m.loc_D_pct = pct(loc.var_reduction_D);
m.loc_goal_pct = pct(loc.var_reduction_goal);
m.loc_ratio = f('%1f', loc.var_reduction_goal / loc.var_reduction_D);
m.loc_sep_m = f('%0f', loc.median_separation_m);
m.loc_sep_km = f('%1f', loc.median_separation_m/1000);

% ----- sampling ---
m.gn_iters = f('%d', sam.gn_iterations);
m.cond_H = f('%1f', sam.condition_number);
m.iters_nat = f('%d', sam.iters_natural);
m.iters_euc = f('%d', sam.iters_euclidean);
m.sqrt_kappa = f('%1f', sam.sqrt_condition);
m.M_imp = f('%d', sam.M_implicit);
m.ess_imp = f('%3f', sam.ess_implicit);
m.M_sir = f('%d', sam.M_sir);
m.ess_sir = f('%3f', sam.ess_sir);

% ----- MRVA ---
m.mrva_n = f('%d', mr.n_wells);
m.mrva_K_min = f('%2f', mr.K_min);
m.mrva_K_max = f('%1f', mr.K_max);
m.mrva_K_mean = f('%1f', mr.K_mean);
m.mrva_V_min = f('%0f', mr.V_min);
m.mrva_V_max = f('%0f', mr.V_max);
m.mrva_mismatch = f('%d', mr.drastic_mismatch);
m.mrva_sep_km = f('%1f', mr.median_separation_m/1000);
m.mrva_D_pct = pct(mr.var_reduction_D);
m.mrva_goal_pct = pct(mr.var_reduction_goal);
m.mrva_V_goal = f('%1f', mr.mean_DRASTIC_goal);
m.mrva_V_D = f('%1f', mr.mean_DRASTIC_D);

% ----- sweep ---
m.sw_frac_range = rng2(sw.frac_obs_vs_sigma, '%1e');
m.sw_R_sigma_range = rng2(sw.R_vs_sigma, '%2f');
m.sw_R_sigmaK_range = rng2(sw.R_vs_sigmaK, '%2f');
m.sw_sep_range = rng2(sw.sep_vs_gamma, '%0f');
m.sw_R_alpha_range = rng2(sw.R_vs_alphaL, '%2f');

% ----- tables ---
m.TABLE_INVENTORY = local_tbl_inventory(d, g, pr);
m.TABLE_LOO = local_tbl_loo(p, cal);
m.TABLE_DESIGN = local_tbl_design(des, loc, mr);
m.TABLE_SWEEP = local_tbl_sweep(sw);

m.GITHUB_URL = github_url;
m.ZENODO_DOI = zenodo_doi;

```

end

```
% =====  
function h = local_lookup(names, vals)  
h = containers.Map('KeyType', 'char', 'ValueType', 'double');  
for k = 1:numel(names)  
    nm = names{k};  
    if isstring(nm) || iscell(nm), nm = char(nm); end  
    h(nm) = vals(k);  
end  
end
```

```
% =====  
function t = local_tbl_inventory(d, g, pr)  
rows = {  
    'Monitoring points with a DRASTIC index', sprintf('%d', d.n_wells_total)  
    'Of which inside the digitised boundary', sprintf('%d', d.n_wells_inside)  
    'With a pumping-test hydraulic conductivity', sprintf('%d', d.n_with_K)  
    'With a nitrate concentration', sprintf('%d', d.n_with_NO3)  
    'Inside the boundary, with conductivity', sprintf('%d', d.n_inside_with_K)  
    'Inside the boundary, with nitrate', sprintf('%d', d.n_inside_with_NO3)  
    'Aquifer boundary vertices', sprintf('%d', d.n_boundary_vertices)  
    ['Digitised area (km' char(178) ')'], sprintf('%.1f', g.area_polygon_km2)  
    ['Hydraulic conductivity (m d' char(8315) char(185) ')'], ...  
        sprintf('%.0f to %.0f, mean %.1f, sd %.1f', d.K_min, d.K_max, d.K_mean, d.K_  
sd)  
    ['Nitrate (mg L' char(8315) char(185) ')'], ...  
        sprintf('%.2f to %.2f, mean %.2f, sd %.2f', d.NO3_min, d.NO3_max, d.NO3_mean  
, d.NO3_sd)  
    'DRASTIC index', ...  
        sprintf('%.0f to %.0f, mean %.1f, sd %.1f', d.DRASTIC_min, d.DRASTIC_max, d.  
DRASTIC_mean, d.DRASTIC_sd)  
    'Lithological classes of the K wells (coarse/medium/fine)', ...  
        sprintf('%d / %d / %d', d.facies_counts(1), d.facies_counts(2), d.facies_cou  
nts(3))  
    'ANOVA of K on the lithological label', ...  
        sprintf('%s%s = %.3f, F(2,%d) = %.2f, p = %.2f', char(951), char(178), ...  
            d.facies_K_eta2, d.n_with_K-3, d.facies_K_F, d.facies_K_p)  
    ['Exponential variogram of log' char(8321) char(8320) 'K'], ...  
        sprintf('nugget %.4f, partial sill %.4f, range %.0f m', ...  
            pr.variogram_nugget, pr.variogram_sill, pr.variogram_range_m)  
};  
t = local_mdtable({'Quantity', 'Value'}, rows);  
end
```

```
function t = local_tbl_loo(p, cal)  
rows = {};  
for k = 1:numel(p.idw_power)  
    rows(end+1, :) = {sprintf('Inverse distance, power %d', p.idw_power(k)), ...  
        sprintf('%.3f', p.idw_R2(k))}; %#ok<AGROW>  
end  
for k = 1:numel(p.kriging_range_m)
```

```

        rows(end+1, :) = {sprintf('Ordinary kriging, range %g km', p.kriging_range_m(
k)/1000), ...
                        sprintf('%+.3f', p.kriging_R2(k))}; %#ok<AGROW>
end
for k = 1:numel(p.linear_names)
    nm = p.linear_names{k};
    if isstring(nm), nm = char(nm); end
    rows(end+1, :) = {sprintf('Linear regression on %s', nm), ...
                    sprintf('%+.3f', p.linear_R2(k))}; %#ok<AGROW>
end
rows(end+1, :) = {'Transient transport model', sprintf('%+.3f', cal.loo_R2)};
rows(end+1, :) = {'Network mean (baseline)', '0.000'};
t = local_mdtable({'Method', 'Leave-one-out R2'}, rows);
end

```

```

function t = local_tbl_design(des, loc, mr)
rows = {
    'Transport observations, Fushe-Kuqe', ...
        sprintf('%+.1f', 100*(1 - (des.sd_goal_after_D_days/des.sd_goal_prior_days)^2))
, ...
    sprintf('%+.1f', 100*(1 - (des.sd_goal_after_goal_days/des.sd_goal_prior_days)^
2)), ...
    sprintf('%+.0f', des.median_separation_m)
    'Characterisation points, Fushe-Kuqe', ...
        sprintf('%+.1f', 100*loc.var_reduction_D), ...
        sprintf('%+.1f', 100*loc.var_reduction_goal), ...
        sprintf('%+.0f', loc.median_separation_m)
    'Characterisation points, MRVA', ...
        sprintf('%+.1f', 100*mr.var_reduction_D), ...
        sprintf('%+.1f', 100*mr.var_reduction_goal), ...
        sprintf('%+.0f', mr.median_separation_m)
};
t = local_mdtable({'Observation type and site', 'D-optimal (%)', ...
                  'Goal-oriented (%)', 'Median separation (m)'}, rows);
end

```

```

function t = local_tbl_sweep(sw)
rows = {
    ['Nitrate measurement sd (mg L' char(8315) char(185) ')'], ...
        sprintf('%g to %g', min(sw.sigma_obs), max(sw.sigma_obs)), ...
        sprintf('%+.3f to %+.3f', min(sw.R_vs_sigma), max(sw.R_vs_sigma)), ...
        sprintf('observation share %.1e to %.1e', min(sw.frac_obs_vs_sigma), max(sw.fr
ac_obs_vs_sigma))
    'Pumping-test relative error', ...
        sprintf('%g to %g', min(sw.sigma_K_rel), max(sw.sigma_K_rel)), ...
        sprintf('%+.3f to %+.3f', min(sw.R_vs_sigmaK), max(sw.R_vs_sigmaK)), '-'
    'Longitudinal dispersivity (m)', ...
        sprintf('%g to %g', min(sw.alphaL), max(sw.alphaL)), ...
        sprintf('%+.3f to %+.3f', min(sw.R_vs_alphaL), max(sw.R_vs_alphaL)), ...
        sprintf('median separation %.0f to %.0f m', min(sw.sep_vs_alphaL), max(sw.sep_
vs_alphaL))
    'Vulnerability exponent', ...

```

```

    sprintf('%g to %g', min(sw.gammaV), max(sw.gammaV)), '-', ...
    sprintf('median separation %.0f to %.0f m, goal advantage %.2f to %.2f%%', ...
        min(sw.sep_vs_gamma), max(sw.sep_vs_gamma), ...
        100*min(sw.adv_vs_gamma), 100*max(sw.adv_vs_gamma))
};
t = local_mdtbl({ 'Assumption varied', 'Range', 'R', 'Other effect'}, rows);
end

```

```

% =====
function t = local_mdtbl(head, rows)
lines = {[ ' ' strjoin(head, ' | ') ' |'], ...
    [ ' ' repmat('---|', 1, numel(head)) ]};
for k = 1:size(rows, 1)
    lines{end+1} = [ ' ' strjoin(rows(k, :), ' | ') ' |']; %#ok<AGROW>
end
t = strjoin(lines, newline);
end

```

A.18 fk_latex2omml.m

```

function xml = fk_latex2omml(tex, display)
%FK_LATEX2OMML Convert the LaTeX subset used in this manuscript to OMML.
%
% xml = fk_latex2omml(tex)          inline equation
% xml = fk_latex2omml(tex, true)    display equation, wrapped in m:oMathPara
%
% This is not a general LaTeX engine. It covers exactly the constructs the
% manuscript uses, which fk_check_omml.m enumerates from the source and
% asserts are all handled; anything outside that set raises an error rather
% than being silently dropped, because a silently dropped symbol in an
% equation is the kind of error that survives to print.
%
% Word stores equations as Office MathML. Writing it directly keeps the
% equations editable in Word, which rendering them as images would not.

if nargin < 2, display = false; end

toks = local_tokenise(tex);
[body, pos] = local_parse(toks, 1, numel(toks));
if pos <= numel(toks)
    error('fk_latex2omml:trailing', 'unconsumed input in "%s"', tex);
end

if display
    xml = ['<m:oMathPara><m:oMath>' body '</m:oMath></m:oMathPara>'];
else
    xml = ['<m:oMath>' body '</m:oMath>'];
end
end

```

```

% =====
function t = local_tokenise(s)
%LOCAL_TOKENISE Split into command, brace, script and character tokens.

```

```

t = {};
i = 1;
n = numel(s);
while i <= n
    c = s(i);
    if c == '\'
        if i == n, error('fk_latex2omml:dangling', 'trailing backslash'); end
        j = i + 1;
        if isletter(s(j))
            while j <= n && isletter(s(j)), j = j + 1; end
            t{end+1} = s(i:j-1); %#ok<AGROW>
            i = j;
        else
            t{end+1} = s(i:j); %#ok<AGROW>
            i = j + 1;
        end
    elseif any(c == '{}^_')
        t{end+1} = c; %#ok<AGROW>
        i = i + 1;
    elseif c == ' '
        i = i + 1; % spacing is Word's business
    else
        t{end+1} = c; %#ok<AGROW>
        i = i + 1;
    end
end
end
end

```

```

% =====
function [out, i] = local_parse(t, i, stop)
%LOCAL_PARSE Parse tokens i..stop into OMML, collapsing adjacent characters
% into single runs so Word does not get one run per letter.

```

```

out = '';
buf = '';
function flush()
    if ~isempty(buf)
        out = [out local_run(buf)];
        buf = '';
    end
end
end

```

```

while i <= stop
    tk = t{i};
    if strcmp(tk, '{}')
        break
    elseif strcmp(tk, '{')
        [inner, i] = local_parse(t, i + 1, stop);
        if i > stop || ~strcmp(t{i}, '{}')
            error('fk_latex2omml:brace', 'unbalanced brace');
        end
        i = i + 1;
        [inner, i] = local_scripts(t, i, stop, inner);
    end
end

```

```

        flush();
        out = [out inner];
    elseif strcmp(tk, '^') || strcmp(tk, '_')
        % a script attached to the characters accumulated so far
        if isempty(buf)
            error('fk_latex2omml:script', 'script with no base');
        end
        base = local_run(buf(end));
        buf(end) = [];
        flush();
        [sc, i] = local_scripts(t, i, stop, base);
        out = [out sc];
    else
        [node, i, isrun] = local_atom(t, i, stop);
        if isrun
            buf = [buf node];
        else
            [node, i] = local_scripts(t, i, stop, node);
            flush();
            out = [out node];
        end
    end
end
flush();
end

% =====
function [out, i] = local_scripts(t, i, stop, base)
%LOCAL_SCRIPTS Attach any ^ and _ that follow a completed base.
sup = ''; sub = '';
while i <= stop && (strcmp(t{i}, '^') || strcmp(t{i}, '_'))
    kind = t{i};
    [arg, i] = local_argument(t, i + 1, stop);
    if strcmp(kind, '^'), sup = arg; else, sub = arg; end
end
if ~isempty(sup) && ~isempty(sub)
    out = ['<m:sSubSup><m:e>' base '</m:e><m:sub>' sub ...
        '</m:sub><m:sup>' sup '</m:sup></m:sSubSup>'];
elseif ~isempty(sup)
    out = ['<m:sSup><m:e>' base '</m:e><m:sup>' sup '</m:sup></m:sSup>'];
elseif ~isempty(sub)
    out = ['<m:sSub><m:e>' base '</m:e><m:sub>' sub '</m:sub></m:sSub>'];
else
    out = base;
end
end

% =====
function [out, i] = local_argument(t, i, stop)
%LOCAL_ARGUMENT One braced group, or one atom.
if i > stop, error('fk_latex2omml:arg', 'missing argument'); end
if strcmp(t{i}, '{')

```

```

[out, i] = local_parse(t, i + 1, stop);
if i > stop || ~strcmp(t{i}, '{}')
    error('fk_latex2omml:brace', 'unbalanced brace in argument');
end
i = i + 1;
else
    [node, i, isrun] = local_atom(t, i, stop);
    if isrun, out = local_run(node); else, out = node; end
end
end

% =====
function [out, i, isrun] = local_atom(t, i, stop)
%LOCAL_ATOM One token, expanded. isrun true means "plain text, may be
% merged with neighbours into one run".
tk = t{i};
isrun = false;

% ---- structures taking arguments -----
switch tk
    case '\frac'
        [num, i] = local_argument(t, i + 1, stop);
        [den, i] = local_argument(t, i, stop);
        out = ['<m:f><m:fPr><m:type m:val="bar"/></m:fPr>' ...
            '<m:num>' num '</m:num><m:den>' den '</m:den></m:f>'];
        return
    case '\sqrt'
        [e, i] = local_argument(t, i + 1, stop);
        out = ['<m:rad><m:radPr><m:degHide m:val="1"/></m:radPr>' ...
            '<m:deg/><m:e>' e '</m:e></m:rad>'];
        return
    case {'\bar', '\overline'}
        [e, i] = local_argument(t, i + 1, stop);
        out = ['<m:bar><m:barPr><m:pos m:val="top"/></m:barPr>' ...
            '<m:e>' e '</m:e></m:bar>'];
        return
    case {'\text', '\mathcal', '\mathrm'}
        [e, i] = local_argument(t, i + 1, stop);
        out = e; % upright is Word's default for m:t
        return
    case '\operatorname'
        % An upright run, not an m:func. A function with an empty base is
        % valid OMML but Word reserves space for the missing argument and
        % leaves a gap before the bracket that follows.
        j = i + 1;
        if j <= stop && strcmp(t{j}, '{')
            k = j + 1; txt = '';
            while k <= stop && ~strcmp(t{k}, '}')
                txt = [txt t{k}]; %#ok<AGROW>
                k = k + 1;
            end
            i = k + 1;

```

```

else
    txt = t{j}; i = j + 1;
end
out = local_upright(txt);
return
case '\underbrace'
[e, i] = local_argument(t, i + 1, stop);
if i <= stop && strcmp(t{i}, '_')
    [lab, i] = local_argument(t, i + 1, stop);
else
    lab = '';
end
grp = ['<m:groupChr><m:groupChrPr><m:chr m:val="&#9183;" />' ...
        '<m:pos m:val="bot" /><m:vertJc m:val="top" /></m:groupChrPr>' ...
        '<m:e>' e '</m:e></m:groupChr>'];
if isempty(lab)
    out = grp;
else
    out = ['<m:limLow><m:e>' grp '</m:e><m:lim>' lab '</m:lim></m:limLow>
'];
end
return
case '\sum'
sub = ''; sup = '';
while i + 1 <= stop && (strcmp(t{i+1}, '_') || strcmp(t{i+1}, '^'))
    kind = t{i+1};
    [arg, i2] = local_argument(t, i + 2, stop);
    if strcmp(kind, '_'), sub = arg; else, sup = arg; end
    i = i2 - 1;
end
i = i + 1;
if isempty(sup)
    % A sum with only a lower index is set as the symbol with a
    % subscript. An n-ary with an empty base would be correct OMML
    % but Word reserves space for the missing summand and leaves a
    % visible gap before the terms that follow.
    base = local_run(char(8721));
    if isempty(sub)
        out = base;
    else
        out = ['<m:sSub><m:e>' base '</m:e><m:sub>' sub ...
                '</m:sub></m:sSub>'];
    end
else
    pr = ['<m:naryPr><m:chr m:val="&#8721;" />' ...
          '<m:limLoc m:val="undOvr" />'];
    if isempty(sub), pr = [pr '<m:subHide m:val="1" />']; end
    pr = [pr '</m:naryPr>'];
    out = ['<m:nary>' pr '<m:sub>' sub '</m:sub><m:sup>' sup ...
            '</m:sup><m:e></m:e></m:nary>'];
end
return

```

```

case '\left'
    i = i + 1;
    if i > stop, error('fk_latex2omml:left', '\\left with no delimiter'); end
    opench = local_delim(t{i});
    i = i + 1;
    depth = 0;
    j = i;
    while j <= stop
        if strcmp(t{j}, '\left'), depth = depth + 1; end
        if strcmp(t{j}, '\right')
            if depth == 0, break; end
            depth = depth - 1;
        end
        j = j + 1;
    end
    if j > stop, error('fk_latex2omml:right', 'missing \\right'); end
    [inner, ~] = local_parse(t, i, j - 1);
    closech = local_delim(t{j + 1});
    i = j + 2;
    out = ['<m:d><m:dPr><m:begChr m:val="" opench ""/><m:endChr m:val="" ...
        closech ""/></m:dPr><m:e>' inner '</m:e></m:d>'];
    return
end

% ---- functions rendered upright -----
fn = {'\log', '\ln', '\exp', '\det', '\max', '\min', '\inf', '\sup'};
if any(strcmp(tk, fn))
    name = tk(2:end);
    i = i + 1;
    sub = '';
    if i <= stop && strcmp(t{i}, '_')
        [sub, i] = local_argument(t, i + 1, stop);
    end
    out = local_upright([name char(8201)]); % thin space after the name
    if ~isempty(sub)
        out = ['<m:sSub><m:e>' local_upright(name) '</m:e><m:sub>' sub ...
            '</m:sub></m:sSub>' local_upright(char(8201))];
    end
    return
end

% ---- single symbols -----
% found and sym are separate because several commands map to the empty
% string: \! is negative thin space, which Word handles by itself.
[found, sym] = local_symbol(tk);
if found
    i = i + 1;
    out = sym;
    isrun = true;
    return
end

```

```

if numel(tk) == 1
    i = i + 1;
    out = tk;
    isrun = true;
    return
end

error('fk_latex2omml:unknown', 'unhandled LaTeX token "%s"', tk);
end

% =====
function d = local_delim(tk)
switch tk
    case {'(', ')'}, d = tk;
    case {'[', ']'}, d = tk;
    case {'\{'}, d = '{';
    case {'\}'}, d = '}';
    case {'|'}, d = '|';
    case {'.'}, d = '.';
    otherwise, error('fk_latex2omml:delim', 'unsupported delimiter "%s"', tk);
end
end

% =====
function [found, s] = local_symbol(tk)
%LOCAL_SYMBOL Unicode for the commands the manuscript uses. found is false
% for anything else, so local_atom can raise a clear error; it is
% separate from s because some commands map to the empty string.
persistent map
if isempty(map)
    map = containers.Map('KeyType', 'char', 'ValueType', 'char');
    g = { '\alpha',char(945); '\beta',char(946); '\gamma',char(947); ...
        '\Gamma',char(915); '\delta',char(948); '\Delta',char(916); ...
        '\epsilon',char(949); '\eta',char(951); '\theta',char(952); ...
        '\Theta',char(920); '\kappa',char(954); '\lambda',char(955); ...
        '\Lambda',char(923); '\mu',char(956); '\nu',char(957); ...
        '\xi',char(958); '\Xi',char(926); '\pi',char(960); ...
        '\Pi',char(928); '\rho',char(961); '\sigma',char(963); ...
        '\Sigma',char(931); '\tau',char(964); '\phi',char(966); ...
        '\Phi',char(934); '\chi',char(967); '\psi',char(968); ...
        '\Psi',char(936); '\omega',char(969); '\Omega',char(937); ...
        '\partial',char(8706); '\ell',char(8467); '\top',char(8868); ...
        '\in',char(8712); '\pm',char(177); '\sim',char(8764); ...
        '\mapsto',char(8614); '\square',char(9633); '\times',char(215); ...
        '\cdot',char(183); '\leq',char(8804); '\geq',char(8805); ...
        '\neq',char(8800); '\approx',char(8776); '\infty',char(8734); ...
        '\{','{'; '\}','}'; '\,',' ' ; '\;',' ' ; '\,',' ' ; ...
        '\!','!'; '\qquad',' ' ; '\quad',' ' ; '\%','% ' };
    for k = 1:size(g, 1)
        map(g{k, 1}) = g{k, 2};
    end
end
end

```

```

found = isKey(map, tk);
if found
    s = map(tk);
else
    s = '';
end
end

% =====
function r = local_run(txt)
if isempty(txt)
    r = '';
else
    r = ['<m:r><m:t xml:space="preserve">' fk_xmlesc(txt) '</m:t></m:r>'];
end
end

% =====
function r = local_upright(txt)
%LOCAL_UPRIGHT A run set in roman, for operator and function names, which
% Word would otherwise italicise as if they were variables.
r = ['<m:r><m:rPr><m:nor/></m:rPr><m:t xml:space="preserve">' ...
    fk_xmlesc(txt) '</m:t></m:r>'];
end

```

A.19 fk_xmlesc.m

```

function s = fk_xmlesc(s)
%FK_XMLESC Escape the five XML metacharacters, ampersand first.
s = strrep(s, '&', '&amp;');
s = strrep(s, '<', '&lt;');
s = strrep(s, '>', '&gt;');
s = strrep(s, '"', '&quot;');
s = strrep(s, "'", '&apos;');
end

```

A.20 fk_check_omml.m

```

function fk_check_omml(mdfile)
%FK_CHECK_OMML Convert every equation in the manuscript and report failures.
%
% The converter raises on anything it does not recognise rather than dropping
% it, so running it over the whole manuscript is the test that it covers the
% source. Called by fk_build_docx before it writes anything.

if nargin < 1
    cfg = fk_config();
    mdfile = fullfile(cfg.dir_code, '..', 'manuscript', 'manuscript_filled.md');
end
s = fileread(mdfile);

disp_eq = regexp(s, '\$\$(.*?)\$\$', 'tokens');
rest = regexprep(s, '\$\$.*?\$\$', '');
inl_eq = regexp(rest, '(?<!\$)\$([^\$\\n]+?)\$(?!\$)', 'tokens');

```

```

nd = numel(dispenq); ni = numel(inl_eq);
fprintf('[omml] %d display and %d inline equations\n', nd, ni);

bad = 0;
for k = 1:nd
    try
        x = fk_latex2omml(dispenq{k}{1}, true);
        assert(contains(x, 'm:oMathPara'));
    catch ME
        bad = bad + 1;
        fprintf('[omml] DISPLAY %d failed: %s\n    %s\n', k, ME.message, ...
            strtrim(dispenq{k}{1}));
    end
end
for k = 1:ni
    try
        fk_latex2omml(inl_eq{k}{1}, false);
    catch ME
        bad = bad + 1;
        fprintf('[omml] INLINE %d failed: %s\n    %s\n', k, ME.message, ...
            strtrim(inl_eq{k}{1}));
    end
end

if bad > 0
    error('fk_check_omml:failed', '%d of %d equations did not convert', ...
        bad, nd + ni);
end
fprintf('[omml] all %d equations convert\n', nd + ni);
end

```

A.21 [fk_md2docx.m](#)

```

function fk_md2docx(mdtext, template, outfile)
%FK_MD2DOCX Write a .docx from the manuscript's markdown, in MATLAB alone.
%
% fk_md2docx(mdtext, template, outfile)
%
% Handles the subset of markdown the manuscript uses: YAML title and author,
% ATX headings, paragraphs, bullet and numbered lists, pipe tables, fenced
% code blocks, bold and code spans, and inline and display mathematics, the
% last through fk_latex2omml.
%
% The template supplies every part of the package except word/document.xml:
% styles, numbering, theme, fonts, section setup. Only the body is written
% here, which is the same discipline used elsewhere in this project for
% editing a Word file without rebuilding it.
%
% Anything the parser does not recognise raises an error. Silently dropping
% a construct would produce a document that looks finished and is not.

W = ['xmlns:w="http://schemas.openxmlformats.org/wordprocessingml/2006/main"'];

```

```

M = ['xmlns:m="http://schemas.openxmlformats.org/officeDocument/2006/math"'];
R = ['xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationship
s"'];

lines = regexp(mdtext, '\r?\n', 'split');
n = numel(lines);
body = {};
i = 1;
after_heading = false;

% ----- YAML front matter
if n >= 1 && strcmp(strtrim(lines{1}), '---')
    j = 2;
    ttl = ''; aut = '';
    while j <= n && ~strcmp(strtrim(lines{j}), '---')
        tok = regexp(lines{j}, '^(\\w+):\\s*"?(.*?)"?\\s*$', 'tokens', 'once');
        if ~isempty(tok)
            switch tok{1}
                case 'title', ttl = tok{2};
                case 'author', aut = tok{2};
            end
        end
        j = j + 1;
    end
    if ~isempty(ttl)
        body{end+1} = local_para('Title', local_inline(ttl));
    end
    if ~isempty(aut)
        body{end+1} = local_para('Author', local_inline(aut));
    end
    i = j + 1;
end

% ----- block loop --
while i <= n
    ln = lines{i};
    s = strtrim(ln);

    if isempty(s)
        i = i + 1;
        continue
    end

    % ---- fenced code -----
    % One paragraph for the whole block, with a line break between lines. A
    % paragraph per line would carry the style's paragraph spacing into every
    % line and more than double the page count.
    if startsWith(s, '```')
        i = i + 1;
        code = {};
        while i <= n && ~startsWith(strtrim(lines{i}), '```')
            code{end+1} = lines{i}; %#ok<AGROW>

```

```

        i = i + 1;
    end
    i = i + 1;
    body{end+1} = local_code_block(code); %#ok<AGROW>
    after_heading = false;
    continue
end

% ---- pipe table -----
if startsWith(s, '|')
    rows = {};
    while i <= n && startsWith(strtrim(lines{i}), '|')
        rows{end+1} = strtrim(lines{i}); %#ok<AGROW>
        i = i + 1;
    end
    body{end+1} = local_table(rows); %#ok<AGROW>
    after_heading = false;
    continue
end

% ---- heading -----
h = regexp(s, '^(#{1,6})\s+(.*)$', 'tokens', 'once');
if ~isempty(h)
    lvl = numel(h{1});
    body{end+1} = local_para(sprintf('Heading%d', lvl), ...
                             local_inline(h{2})); %#ok<AGROW>
    i = i + 1;
    after_heading = true;
    continue
end

% ---- display equation on its own line -----
if startsWith(s, '$$') && endsWith(s, '$$') && numel(s) > 4
    tex = s(3:end-2);
    body{end+1} = ['<w:p>' fk_latex2omml(tex, true) '</w:p>']; %#ok<AGROW>
    i = i + 1;
    after_heading = false;
    continue
end

% ---- bullet item -----
if startsWith(s, '- ')
    body{end+1} = local_listpara(1001, local_inline(s(3:end))); %#ok<AGROW>
    i = i + 1;
    after_heading = false;
    continue
end

% ---- numbered item -----
nm = regexp(s, '^\d+\.\s+(.*)$', 'tokens', 'once');
if ~isempty(nm)
    body{end+1} = local_listpara(1002, local_inline(nm{1})); %#ok<AGROW>
end

```

```

        i = i + 1;
        after_heading = false;
        continue
    end

% ---- ordinary paragraph, possibly wrapped over several lines -----
buf = {s};
i = i + 1;
while i <= n
    s2 = strtrim(lines{i});
    if isempty(s2) || startsWith(s2, '|') || startsWith(s2, '#') || ...
        startsWith(s2, '- ') || startsWith(s2, '` ` `') || ...
        startsWith(s2, '$$') || ~isempty(regexp(s2, '^\\d+\\.\\.s', 'once'))
        break
    end
    buf{end+1} = s2; %#ok<AGROW>
    i = i + 1;
end
txt = strjoin(buf, ' ');
if after_heading, sty = 'FirstParagraph'; else, sty = 'BodyText'; end
body{end+1} = local_para(sty, local_inline(txt)); %#ok<AGROW>
after_heading = false;
end

% ----- assemble part --
sectPr = local_sectpr(template);
xml = ['<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' ...
    '<w:document ' W ' ' M ' ' R ' ><w:body>' ...
    strjoin(body, '') sectPr '</w:body></w:document>'];

fk_zip_replace(template, outfile, 'word/document.xml', xml);
fprintf('[docx] %s written, %d block elements\\n', outfile, numel(body));
end

% =====
function p = local_para(style, runs)
p = ['<w:p><w:pPr><w:pStyle w:val="" style ""/></w:pPr>' runs '</w:p>'];
end

function p = local_listpara(numid, runs)
p = ['<w:p><w:pPr><w:pStyle w:val="Compact"/><w:numPr>' ...
    '<w:ilvl w:val="0"/><w:numId w:val="" num2str(numid) ""/>' ...
    '</w:numPr></w:pPr>' runs '</w:p>'];
end

function p = local_code_block(codelines)
runs = '';
for k = 1:numel(codelines)
    if k > 1
        runs = [runs '<w:r><w:br/></w:r>']; %#ok<AGROW>
    end
    runs = [runs '<w:r><w:rPr><w:rStyle w:val="VerbatimChar"/></w:rPr>' ...

```

```

        '<w:t xml:space="preserve">' fk_xmlesc(codelines{k}) ...
        '</w:t></w:r>']; %#ok<AGROW>
end
p = ['<w:p><w:pPr><w:pStyle w:val="SourceCode"/></w:pPr>' runs '</w:p>'];
end

% =====
function out = local_inline(s)
%LOCAL_INLINE Split a paragraph into runs: maths, bold, code, plain.
out = '';
rest = s;
while ~isempty(rest)
    % earliest of the four markers
    iM = regexp(rest, '\$[^\$]+\$', 'once');
    iB = regexp(rest, '\*\*([^\*]+\*\*)', 'once');
    iC = regexp(rest, '`([`]+)`', 'once');
    iI = regexp(rest, '(?<!\*)\*([^\*]+\*(?!\*))', 'once');
    kinds = {'m', 'b', 'c', 'i'};
    keep = ~cellfun(@isempty, {iM, iB, iC, iI});
    if ~any(keep)
        out = [out local_run(rest, '')];
        return
    end
    cc = [iM iB iC iI];          % empties vanish, so cc lines up with kk
    kk = kinds(keep);
    [pos, w] = min(cc);
    kind = kk{w};

    if pos > 1
        out = [out local_run(rest(1:pos-1), '')];
    end
    rest = rest(pos:end);
    switch kind
        case 'm'
            tk = regexp(rest, '^\$([^\$]+\)\$', 'tokens', 'once');
            out = [out fk_latex2omml(tk{1}, false)];
            rest = rest(numel(tk{1}) + 3:end);
        case 'b'
            tk = regexp(rest, '^*\*([^\*]+\*\*)', 'tokens', 'once');
            out = [out local_run(tk{1}, '<w:b/>')];
            rest = rest(numel(tk{1}) + 5:end);
        case 'c'
            tk = regexp(rest, '^`([`]+)`', 'tokens', 'once');
            out = [out local_run(tk{1}, '<w:rStyle w:val="VerbatimChar"/>')];
            rest = rest(numel(tk{1}) + 3:end);
        case 'i'
            tk = regexp(rest, '^*\*([^\*]+\*)', 'tokens', 'once');
            out = [out local_run(tk{1}, '<w:i/>')];
            rest = rest(numel(tk{1}) + 3:end);
    end
end
end
end

```

```

function r = local_run(txt, rpr)
if isempty(txt), r = ''; return; end
if isempty(rpr)
    r = ['<w:r><w:t xml:space="preserve">' fk_xmlesc(txt) '</w:t></w:r>'];
else
    r = ['<w:r><w:rPr>' rpr '</w:rPr><w:t xml:space="preserve">' ...
        fk_xmlesc(txt) '</w:t></w:r>'];
end
end

% =====
function t = local_table(rows)
%LOCAL_TABLE A pipe table. The second row of a markdown table is the
% alignment rule and carries no content.
cells = cell(numel(rows), 1);
keep = true(numel(rows), 1);
for k = 1:numel(rows)
    r = rows{k};
    r = regexprep(r, '^\\|', '');
    r = regexprep(r, '\\|$', '');
    parts = strsplit(r, '|');
    parts = cellfun(@strtrim, parts, 'UniformOutput', false);
    if all(~cellfun(@isempty, regexp(parts, '^:?.-{2,}:?$', 'once'))))
        keep(k) = false;
    end
    cells{k} = parts;
end
cells = cells(keep);
ncol = max(cellfun(@numel, cells));
w = floor(9360 / ncol);

grid = '';
for c = 1:ncol
    grid = [grid '<w:gridCol w:w="' num2str(w) '" />']; %#ok<AGROW>
end

trs = '';
for k = 1:numel(cells)
    tcs = '';
    row = cells{k};
    for c = 1:ncol
        if c <= numel(row), txt = row{c}; else, txt = ''; end
        tcs = [tcs '<w:tc><w:tcPr/><w:p><w:pPr><w:pStyle w:val="Compact"/>' ...
            '</w:pPr>' local_inline(txt) '</w:p></w:tc>']; %#ok<AGROW>
    end
    if k == 1
        trs = [trs '<w:tr><w:trPr><w:tblHeader w:val="on"/></w:trPr>' tcs '</w:tr>']; %#ok<AGROW>
    else
        trs = [trs '<w:tr>' tcs '</w:tr>']; %#ok<AGROW>
    end
end

```

end

```
t = ['<w:tbl><w:tblPr><w:tblStyle w:val="Table"/>' ...  
    '<w:tblW w:type="pct" w:w="5000"/><w:tblLayout w:type="fixed"/>' ...  
    '<w:tblLook w:firstRow="1" w:lastRow="0" w:firstColumn="0" ' ...  
    'w:lastColumn="0" w:noHBand="0" w:noVBand="0" w:val="0020"/></w:tblPr>' ...  
    '<w:tblGrid>' grid '</w:tblGrid>' trs '</w:tbl>' ...  
    '<w:p><w:pPr><w:pStyle w:val="BodyText"/></w:pPr></w:p>'];
```

end

```
% =====  
function sp = local_sectpr(template)  
%LOCAL_SECTPR Reuse the template's page setup verbatim.  
tmp = tempname;  
mkdir(tmp);  
c = onCleanup(@() rmdir(tmp, 's'));  
unzip(template, tmp);  
d = fileread(fullfile(tmp, 'word', 'document.xml'));  
m = regexp(d, '<w:sectPr[^>]*>.*?</w:sectPr>', 'match', 'once');  
if isempty(m)  
    m = regexp(d, '<w:sectPr[^>]*>/>', 'match', 'once');  
end  
if isempty(m)  
    error('fk_md2docx:sectPr', 'template has no sectPr');  
end  
sp = m;  
end
```

A.22 fk_zip_replace.m

```
function fk_zip_replace(template, outfile, partname, content)  
%FK_ZIP_REPLACE Copy a zip package entry by entry, swapping one member.  
%  
% Used to write a .docx by replacing word/document.xml in a template while  
% every other part, styles, numbering, theme, fonts, is carried across  
% untouched. That is the same discipline this project uses when editing a  
% Word file rather than rebuilding it.  
%  
% Two details, both learned the hard way.  
%  
% Writing goes through the JVM that ships with MATLAB rather than through  
% zip(), because the package contains [Content_Types].xml and the brackets  
% are wildcard characters to MATLAB's file matching.  
%  
% Reading does NOT go through java.io.InputStream.read(byte[]). MATLAB  
% passes arrays to Java by value, so the JVM fills its own copy, the caller  
% gets its buffer back unchanged, and every copied part comes out as a run of  
% zero bytes. The document then looks structurally sound and Word refuses  
% it as corrupt. The parts are therefore extracted to a temporary folder and  
% read with fread, which is the direction that works.
```

```
outfile = local_abspath(outfile);  
template = local_abspath(template);
```

```

tmp = tempname;
mkdir(tmp);
c1 = onCleanup(@() rmdir(tmp, 's')); %#ok<NASGU>
unzip(template, tmp);

names = local_entry_names(template);
newbytes = unicode2native(content, 'UTF-8');

if exist(outfile, 'file'), delete(outfile); end
fos = java.io.FileOutputStream(outfile);
zos = java.util.zip.ZipOutputStream(fos);
c2 = onCleanup(@() local_close(zos, fos)); %#ok<NASGU>

seen = false;
for k = 1:numel(names)
    name = names{k};
    zos.putNextEntry(java.util.zip.ZipEntry(name));
    if strcmp(name, partname)
        bytes = newbytes;
        seen = true;
    else
        bytes = local_readbytes(fullfile(tmp, strrep(name, '/', filesep)));
    end
    if ~isempty(bytes)
        zos.write(typecast(bytes(:)', 'int8'), 0, numel(bytes));
    end
    zos.closeEntry();
end
if ~seen
    error('fk_zip_replace:missing', '%s is not in %s', partname, template);
end
end

% =====
function names = local_entry_names(zipfile)
zf = java.util.zip.ZipFile(zipfile);
c = onCleanup(@() zf.close()); %#ok<NASGU>
names = {};
en = zf.entries();
while en.hasMoreElements()
    e = en.nextElement();
    if ~e.isDirectory()
        names{end+1} = char(e.getName()); %#ok<AGROW>
    end
end
end

function b = local_readbytes(path)
fid = fopen(path, 'r');
if fid < 0
    error('fk_zip_replace:read', 'cannot read extracted part %s', path);
end
end

```

```

end
b = fread(fid, inf, '*uint8');
fclose(fid);
end

function local_close(zos, fos)
try, zos.close(); catch, end %#ok<CTCH>
try, fos.close(); catch, end %#ok<CTCH>
end

function p = local_abspath(p)
if isempty(regexp(p, '^([A-Za-z]:|\\\\\\\\|/)', 'once'))
    p = fullfile(pwd, p);
end
end

```

A.23 fk_make_template.m

```

function fk_make_template(src, dst)
%FK_MAKE_TEMPLATE Strip a .docx down to its styles, to serve as a template.
%
% fk_make_template(src, dst)
%
% Keeps every part of src except the body of word/document.xml, which is
% emptied apart from the section properties. The result carries the styles,
% numbering definitions, theme, fonts and page setup, and nothing else.
%
% The template in manuscript/template.docx was made this way and is committed
% so that a reader does not have to reproduce it; this function is here so
% that they can.

if nargin < 2
    cfg = fk_config();
    man = fullfile(cfg.dir_code, '..', 'manuscript');
    if nargin < 1
        src = fullfile(man, 'Zeqiraj_FusheKuke_OED_manuscript.docx');
    end
    dst = fullfile(man, 'template.docx');
end

tmp = tempname;
mkdir(tmp);
c = onCleanup(@() rmdir(tmp, 's')); %#ok<NASGU>
unzip(src, tmp);
d = fileread(fullfile(tmp, 'word', 'document.xml'));

hdr = regexp(d, '^.*?<w:body>', 'match', 'once');
if isempty(hdr)
    error('fk_make_template:body', '%s has no w:body', src);
end
sp = regexp(d, '<w:sectPr[^>]*>.*?</w:sectPr>', 'match', 'once');
if isempty(sp)
    sp = regexp(d, '<w:sectPr[^>]*>', 'match', 'once');
end

```

end

```
xml = [hdr sp '</w:body></w:document>'];
fk_zip_replace(src, dst, 'word/document.xml', xml);

z = java.util.zip.ZipFile(dst);
np = 0;
en = z.entries();
while en.hasMoreElements(), en.nextElement(); np = np + 1; end
z.close();
fprintf('[template] %s written, %d parts, body emptied\n', dst, np);
end
```

A.24 fk_build_docx.m

```
function fk_build_docx()
%FK_BUILD_DOCX Assemble the submission documents.
%
% Produces, in manuscript/:
%   Zeqiraj_FusheKuke_OED_manuscript.docx   the paper, MATLAB listings after
%                                              the references and nothing else
%   Highlights.docx
%   Cover_Letter.docx
%
% The listings are read from the .m files themselves, so the appendix cannot
% drift from the code that produced the numbers, and each is identified by a
% SHA-256 prefix that fk_verify_docx checks against the file on disk.
%
% Run fk_fill_manuscript first.

cfg = fk_config();
man = fullfile(cfg.dir_code, '..', 'manuscript');
src = fullfile(man, 'manuscript_filled.md');
tpl = fullfile(man, 'template.docx');
out = fullfile(man, 'Zeqiraj_FusheKuke_OED_manuscript.docx');

if ~exist(src, 'file')
    error('fk_build_docx:missing', 'run fk_fill_manuscript first: %s', src);
end
if ~exist(tpl, 'file')
    error('fk_build_docx:template', 'no template: %s', tpl);
end

fk_check_omml(src);

% ----- the listings, in order --
order = {'fk_config.m', 'fk_load_data.m', 'fk_predictability.m', ...
        'fk_build_grid.m', 'fk_prior_field.m', 'fk_forward.m', ...
        'fk_calibrate.m', 'fk_sensitivity.m', 'fk_design.m', ...
        'oed_local.m', 'fk_particles.m', 'mrva_replicate.m', ...
        'fk_sweep.m', 'fk_write_results.m', 'fk_figures.m', ...
        'fk_prepare_data.m', 'fk_fill_manuscript.m', 'fk_latex2omml.m', ...
        'fk_xmlesc.m', 'fk_check_omml.m', 'fk_md2docx.m', ...
```

```

        'fk_zip_replace.m', 'fk_make_template.m', 'fk_build_docx.m', ...
        'fk_verify_docx.m', 'fk_sha256.m', 'run_all.m'};

body = fileread(src);
parts = {strtrim(body), '', '', '## Appendix A. MATLAB code', '', ...
    ['The listings below are the complete pipeline, in the order it runs ' ...
    'them. They are reproduced verbatim from the released repository; the ' ...
    'sixteen-character SHA-256 prefix given with each file identifies the ' ...
    'exact version that produced every number in this paper. Nothing ' ...
    'follows the listings.'], ''};

tbl = {'| File | Lines | SHA-256 prefix |', '|---|---|---|'};
lst = {};
for k = 1:numel(order)
    path = fullfile(cfg.dir_code, order{k});
    if ~exist(path, 'file')
        error('fk_build_docx:listing', 'missing listing %s', order{k});
    end
    code = fileread(path);
    nl = numel(strfind(code, newline)) + 1;
    dg = fk_sha256(path);
    tbl{end+1} = sprintf('| `%s` | %d | `%s` |', order{k}, nl, dg); %#ok<AGROW>
    lst{end+1} = sprintf('### A.%d `%s`', k, order{k}); %#ok<AGROW>
    lst{end+1} = ''; %#ok<AGROW>
    lst{end+1} = '```\matlab'; %#ok<AGROW>
    lst{end+1} = strip_trailing(code); %#ok<AGROW>
    lst{end+1} = '```\n'; %#ok<AGROW>
    lst{end+1} = ''; %#ok<AGROW>
    fprintf('%-24s %5d lines %s\n', order{k}, nl, dg);
end

full = strjoin([parts, tbl, {''}, lst], newline);
fullmd = fullfile(man, 'manuscript_with_code.md');
fid = fopen(fullmd, 'w', 'n', 'UTF-8');
fwrite(fid, unicode2native(full, 'UTF-8'));
fclose(fid);

fk_md2docx(full, tpl, out);

% ----- highlights and cover letter --
hi = regexp(body, '## Highlights\s*(.*?)\n## ', 'tokens', 'once');
if ~isempty(hi)
    hmd = ['## Highlights' newline newline strtrim(hi{1}) newline];
    fk_md2docx(hmd, tpl, fullfile(man, 'Highlights.docx'));
end
cl = fullfile(man, 'cover_letter.md');
if exist(cl, 'file')
    fk_md2docx(fileread(cl), tpl, fullfile(man, 'Cover_Letter.docx'));
end
end

% =====

```

```

function s = strip_trailing(s)
while ~isempty(s) && (s(end) == newline || s(end) == char(13))
    s(end) = [];
end
end

```

A.25 fk_verify_docx.m

```

function fk_verify_docx(docx)
%FK_VERIFY_DOCX Check the submission document before it goes anywhere.
%
% Six checks, each of which has caught something at least once:
%
% 1. Nothing but the MATLAB listings follows the reference list. That is
%    the one structural requirement the document has.
% 2. No unresolved {{placeholder}} survived into the file.
% 3. No em dash anywhere.
% 4. No control character anywhere. This check exists because one reached
%    a delivered document on 12 September 2026: a stray byte turned
%    $\alpha_L$ into the word "lpha", and the converter dropped it in
%    silence.
% 5. No stray dollar sign, which is what an unconverted equation leaves.
% 6. Every listing in the appendix matches the file on disk, compared by
%    the SHA-256 prefix printed beside it.

if nargin < 1
    cfg = fk_config();
    docx = fullfile(cfg.dir_code, '..', 'manuscript', ...
        'Zeqiraj_FusheKuge_OED_manuscript.docx');
end
if ~exist(docx, 'file')
    error('fk_verify_docx:missing', 'no document at %s', docx);
end

[paras, styles, text] = local_paragraphs(docx);
fails = {};

% ---- 1. structure -----
iref = find(strcmpi(strtrim(paras), 'references'), 1);
iapp = find(startsWith(lower(strtrim(paras)), 'appendix a'), 1);
if isempty(iref)
    fails{end+1} = 'no References heading';
elseif isempty(iapp)
    fails{end+1} = 'no Appendix A heading';
elseif iapp < iref
    fails{end+1} = 'the appendix precedes the references';
else
    fprintf(['[verify] %d paragraphs, references at %d, appendix at %d, ' ...
        '%d after it\n'], numel(paras), iref, iapp, ...
        numel(paras) - iapp);
end

```

% The prose checks below apply to the paper, not to the code listings, which

```

% legitimately contain dollar signs and any character a MATLAB file may hold.
if ~isempty(iapp)
    prose = strjoin(paras(1:iapp), newline);
else
    prose = text;
end

% ---- 2. placeholders -----
ph = regexp(prose, '\\{\\{\\w+\\}\\}', 'match');
if ~isempty(ph)
    fails{end+1} = ['unresolved placeholders: ' strjoin(unique(ph), ', ')];
end

% ---- 3. em dashes -----
nem = sum(double(prose) == 8212);
if nem > 0
    fails{end+1} = sprintf('%d em dash(es)', nem);
end

% ---- 4. control characters -----
ctrl = double(text) < 32 & ~ismember(double(text), [9 10 13]);
if any(ctrl)
    j = find(ctrl, 1);
    fails{end+1} = sprintf('control character U+%04X near "%s"', ...
        double(text(j)), text(max(1, j-40):min(numel(text), j+20)));
end

% ---- 5. stray dollar signs -----
% Equations become OMML, which carries no dollar sign; one left in the text
% means an equation was not converted.
nd = sum(double(prose) == 36);
if nd > 0
    j = find(double(prose) == 36, 1);
    fails{end+1} = sprintf('%d dollar sign(s), first near "%s"', nd, ...
        prose(max(1, j-50):min(numel(prose), j+30)));
end

% ---- 6. listings match the code -----
cfg = fk_config();
tok = regexp(text, '([A-Za-z0-9_]+\\.m)\\s+(\\d+)\\s+([0-9a-f]{16})', 'tokens');
nchk = 0;
for k = 1:numel(tok)
    name = tok{k}{1};
    dg = tok{k}{3};
    path = fullfile(cfg.dir_code, name);
    if ~exist(path, 'file')
        fails{end+1} = ['listed file not on disk: ' name]; %#ok<AGROW>
        continue
    end
    real = fk_sha256(path);
    if ~strcmp(real, dg)
        fails{end+1} = sprintf('listing out of date: %s (document %s, disk %s)',

```

```

...
                                name, dg, real); %#ok<AGROW>
    end
    nchk = nchk + 1;
end
fprintf('[verify] %d listing checksums checked\n', nchk);
if nchk == 0
    fails{end+1} = 'no listing checksums found';
end

% ---- report -----
if ~isempty(fails)
    fprintf('\n[verify] FAILED\n');
    for k = 1:numel(fails)
        fprintf('  - %s\n', fails{k});
    end
    error('fk_verify_docx:failed', '%d check(s) failed', numel(fails));
end
fprintf('[verify] all checks passed\n');
end

% =====
function [paras, styles, text] = local_paragraphs(docx)
tmp = tempname;
mkdir(tmp);
c = onCleanup(@() rmdir(tmp, 's')); %#ok<NASGU>
unzip(docx, tmp);
x = fileread(fullfile(tmp, 'word', 'document.xml'));

pieces = regexp(x, '<w:p[ >].*?</w:p>|<w:p/>', 'match');
paras = cell(numel(pieces), 1);
styles = cell(numel(pieces), 1);
for k = 1:numel(pieces)
    s = pieces{k};
    t = regexp(s, '<(?:w|m):t[^>]*>(.*?)</(?:w|m):t>', 'tokens');
    txt = '';
    for j = 1:numel(t)
        txt = [txt local_unesc(t{j}{1})]; %#ok<AGROW>
    end
    paras{k} = txt;
    st = regexp(s, '<w:pStyle w:val="([^\"]+)"', 'tokens', 'once');
    if isempty(st), styles{k} = ''; else, styles{k} = st{1}; end
end
text = strjoin(paras, newline);
end

function s = local_unesc(s)
s = strrep(s, '&lt;', '<');
s = strrep(s, '&gt;', '>');
s = strrep(s, '&quot;', '"');
s = strrep(s, '&apos;', "'");

```

```
s = strrep(s, '&', '&');
end
```

A.26 fk_sha256.m

```
function h = fk_sha256(path, nchar)
%FK_SHA256 Hex SHA-256 of a file, truncated to nchar characters (default 16).
%
% Through the JVM that ships with MATLAB, so no toolbox and no shell call.

if nargin < 2, nchar = 16; end
fid = fopen(path, 'r');
if fid < 0, error('fk_sha256:open', 'cannot read %s', path); end
bytes = fread(fid, inf, '*uint8');
fclose(fid);

md = java.security.MessageDigest.getInstance('SHA-256');
md.update(typecast(bytes, 'int8'));
d = typecast(md.digest(), 'uint8');
h = lower(reshape(dec2hex(d, 2)', 1, []));
h = h(1:min(nchar, numel(h)));
end
```

A.27 run_all.m

```
function R = run_all()
%RUN_ALL Reproduce every number reported in the manuscript.
%
% Usage: cd code; matlab -batch "run_all"
%
% Writes results/results.json, which is the single file the manuscript
% quotes from, plus results/run_all.mat with the full state. Any number in
% the paper that is not in results.json is an error.

t_start = tic;
cfg = fk_config();
if ~exist(cfg.dir_results, 'dir'), mkdir(cfg.dir_results); end

fprintf('\n===== 1. data =====\n');
D = fk_load_data(cfg);

fprintf('\n===== 2. is the nitrate field predictable at all =====\n');
Pr = fk_predictability(cfg, D);

fprintf('\n===== 3. grid and prior =====\n');
G = fk_build_grid(cfg, D);
P = fk_prior_field(cfg, D, G);

fprintf('\n===== 4. forward model and calibration =====\n');
F = fk_forward(cfg, G, P);
Cal = fk_calibrate(cfg, D, G, P, F);

fprintf('\n===== 5. sensitivity and Fisher information =====\n');
S = fk_sensitivity(cfg, G, P, F, Cal);
```

```

fprintf('\n===== 6. monitoring network design =====\n');
Des = fk_design(cfg, D, G, P, F, Cal, S);

fprintf('\n===== 7. the same design with a local operator =====\n');
cand = true(G.n_active, 1);
cand(Cal.cell_of) = false;
w = (P.V / mean(P.V)) .^ cfg.gamma_Vdesign;
Loc = oed_local(P.Phi, P.lam, w, cfg.sd_local, cand, cfg.n_new_wells, ...
               G.xa, G.ya, P.vrange);
fprintf(['[local] Fushe-Kuqe, local operator: variance of the weighted mean ' ...
        'down %.1f%% (D) and %.1f%% (goal); median separation %.0f m\n'], ...
        100 * Loc.var_reduction_D, 100 * Loc.var_reduction_goal, ...
        Loc.median_separation_m);

fprintf('\n===== 8. posterior sampling =====\n');
Par = fk_particles(cfg, G, P, F, Cal, S);

fprintf('\n===== 9. independent replication on the USGS MRVA set =====\n');
M = mrva_replicate(cfg);

fprintf('\n===== 10. sensitivity of the conclusions =====\n');
Sw = fk_sweep(cfg, D, G, P, F, Cal, S, Des);

R = struct('cfg', cfg, 'D', D, 'Pr', Pr, 'G', G, 'P', P, 'F', F, ...
          'Cal', Cal, 'S', S, 'Des', Des, 'Loc', Loc, 'Par', Par, ...
          'M', M, 'Sw', Sw);
R.runtime_s = toc(t_start);
R.matlab_version = version;
R.run_date = datestr(now, 'yyyy-mm-dd HH:MM:SS');

save(fullfile(cfg.dir_results, 'run_all.mat'), '-struct', 'R', '-v7.3');
fk_write_results(cfg, R);
fprintf('\nfinished in %.0f s\n', R.runtime_s);
end

```